

嵌入式系统

第四讲 ARM硬件结构（四）

教师：郭玉臣

Mail:yuchenguo@tongji.edu.cn



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. **UART(0、1)**
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

- **Universal Asynchronous Receiver/Transmitter** 通用非同步收发传输器
- **UART**是一种通用串行数据总线，用于异步通信。该总线双向通信，可以实现全双工传输和接收。在嵌入式设计中，**UART**用来与**PC**进行通信，包括与监控调试器和其它器件，如**EEPROM**通信。

4.14 UART(0、1)

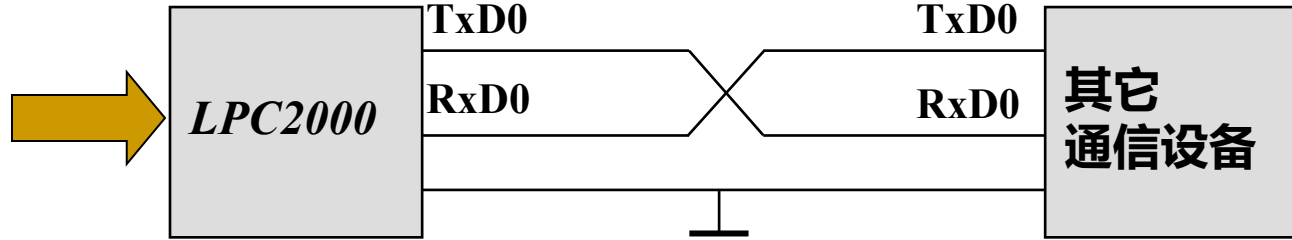
• 特性

LPC2000系列微控制器具有两个功能强大的UART，其特性如下：

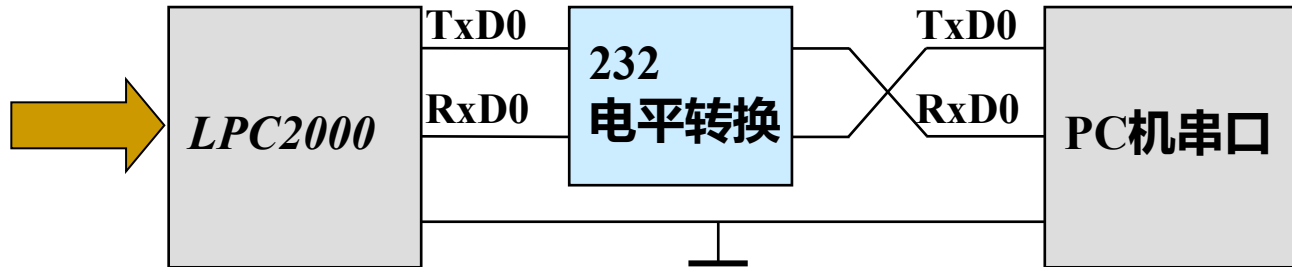
- 16字节接收FIFO和16字节发送FIFO；
- 寄存器位置符合16C550工业标准；
- 接收FIFO触发点可设置为1、4、8或14字节；
- 内置波特率发生器；
- UART1含有标准调制解调器接口信号。

应用示例

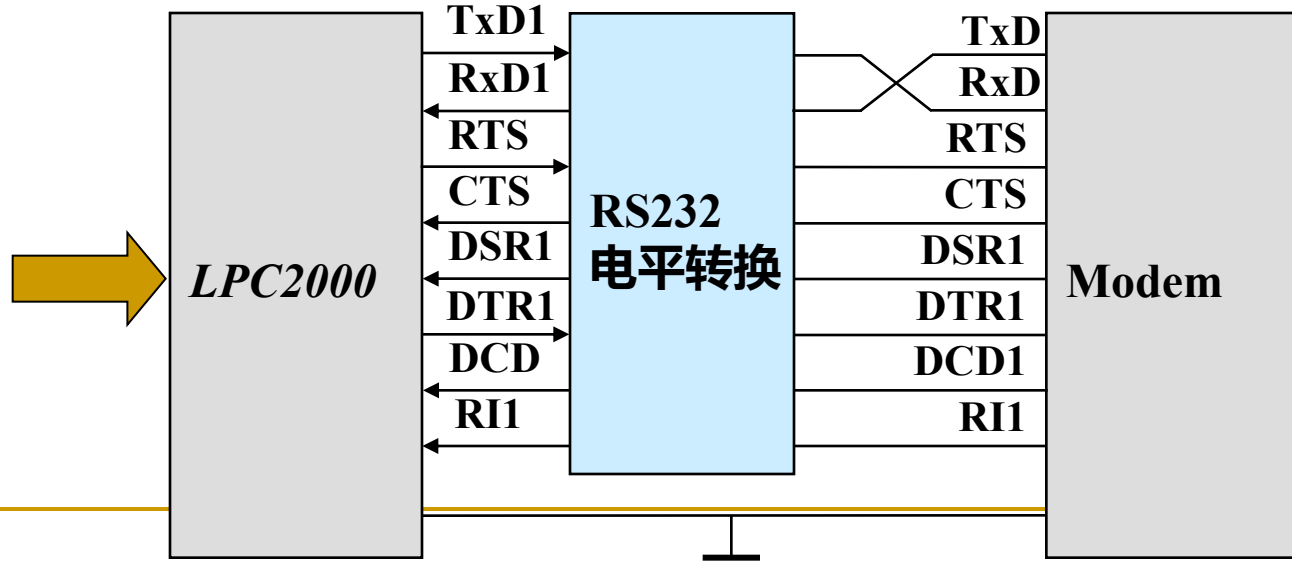
LPC2000的I/O电压为3.3V，连接时须注意电平的匹配。



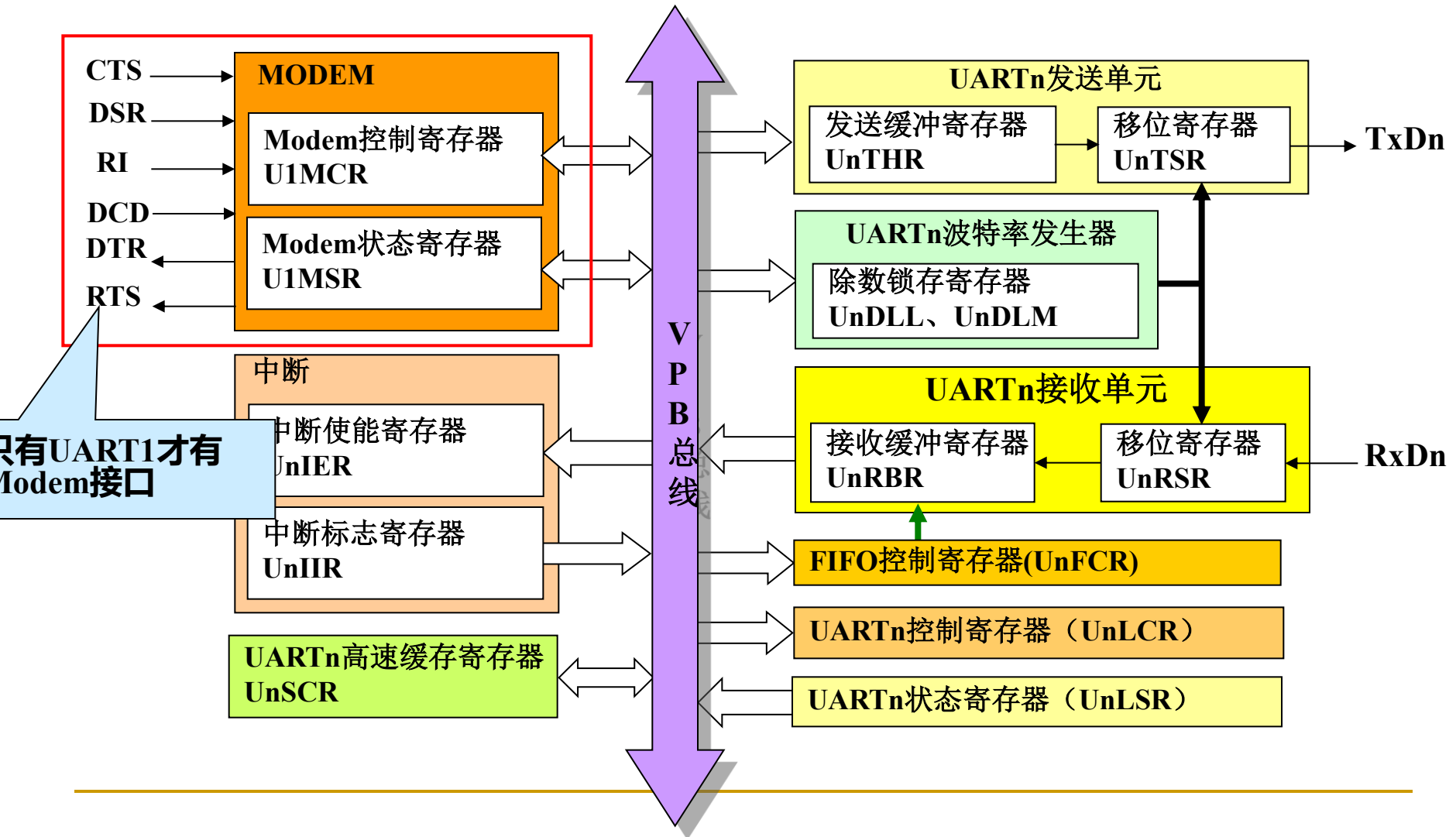
与PC机相连时，由于PC机串口是RS232电平，所以连接时需要使用RS232转换器。



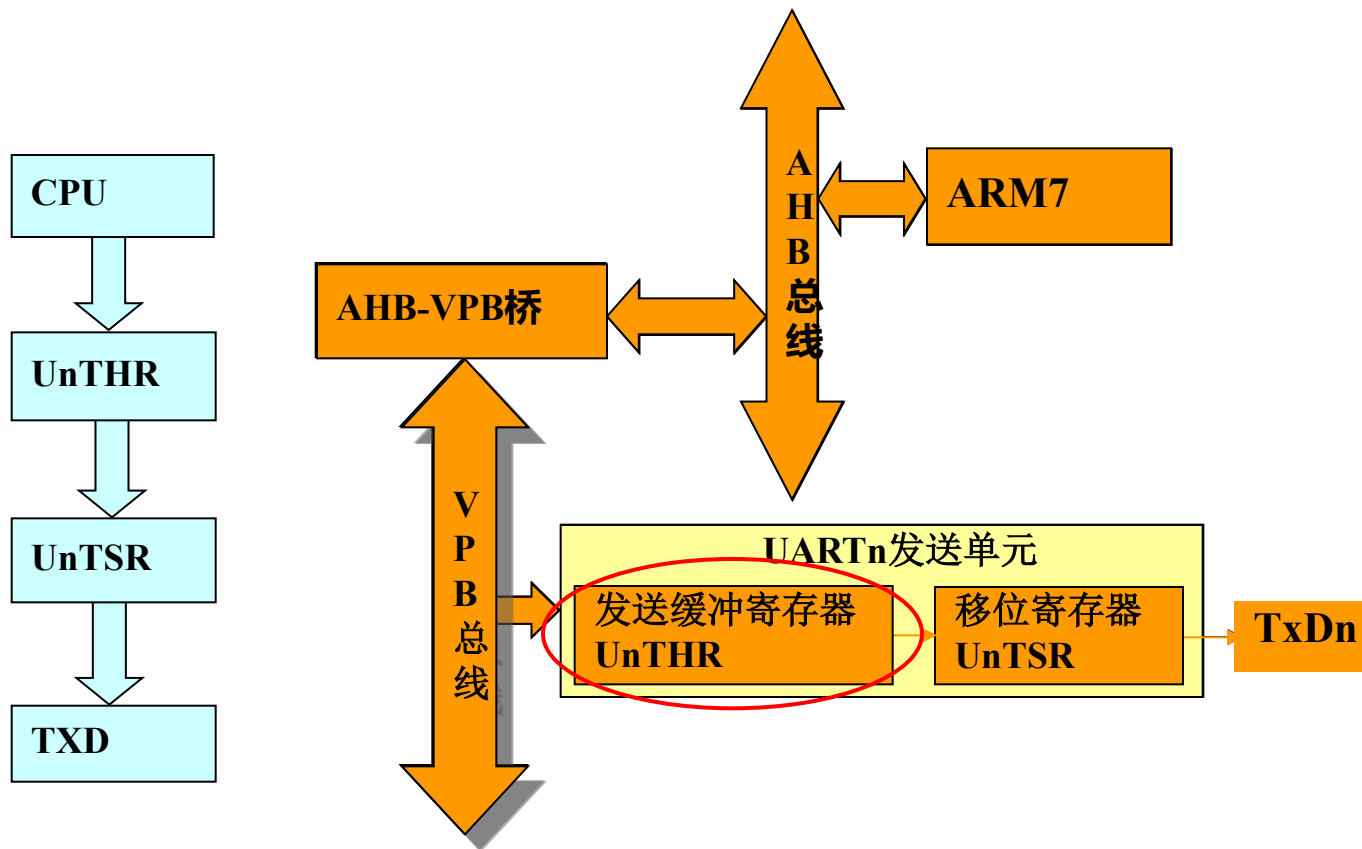
当使用Modem接口时，需要一个RS232转换器将信号转换为RS232电平后，才能与Modem连接。



UART结构图



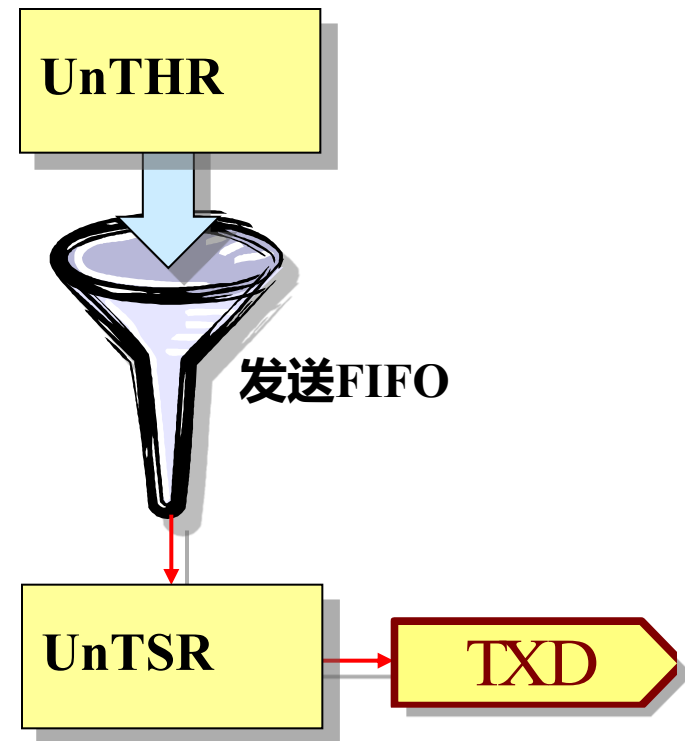
UART发送单元



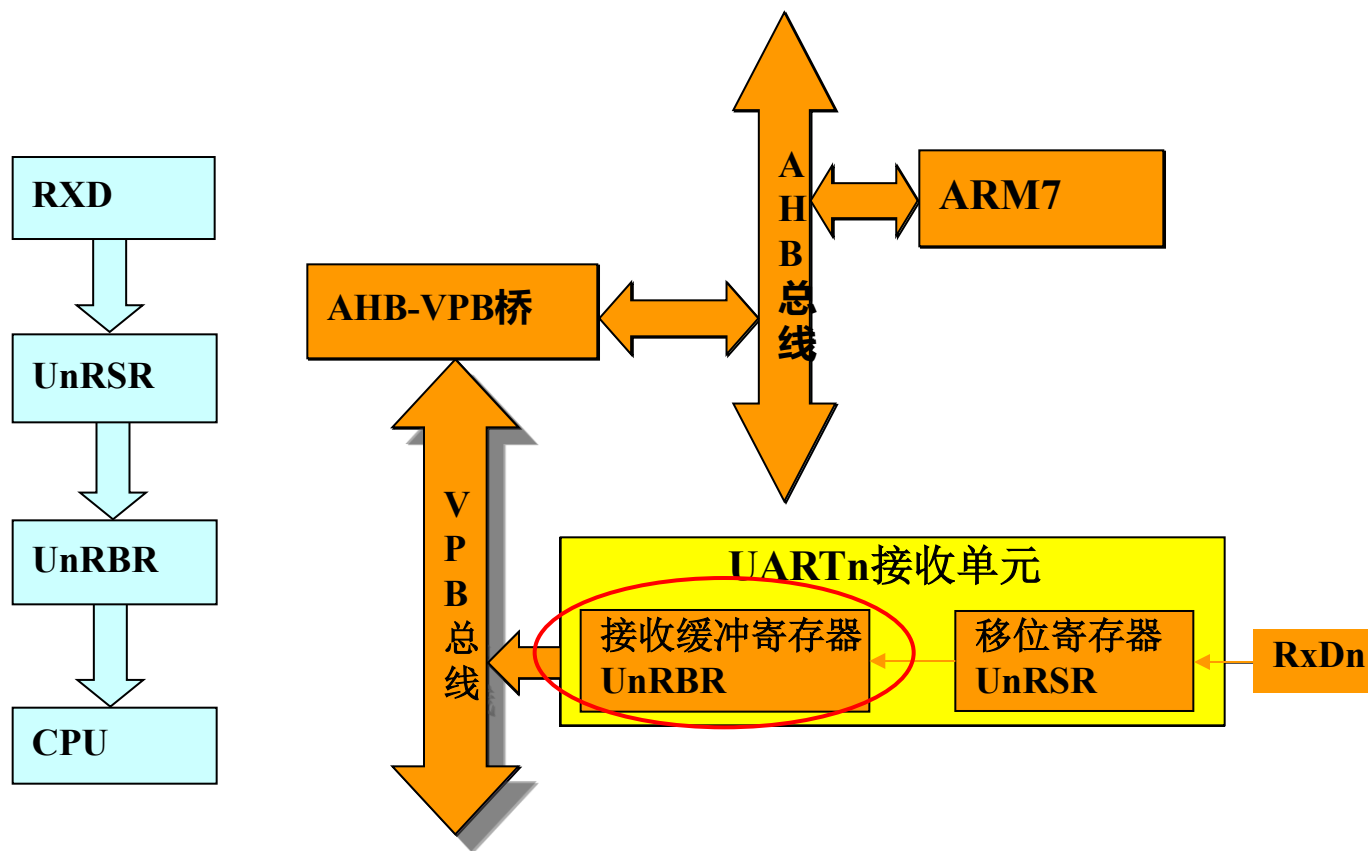
UnTHR	描述	复位值
7 : 0	发送器保持寄存器。写入该寄存器的值保存到发送FIFO中，当该字节到达FIFO底部时，它将被送入发送移位寄存器（UnTSR）进行发送。	未定义

• UART发送FIFO缓冲区

- UART0、UART1各含有1个16字节的发送FIFO缓冲区。
- UnTHR是UARTn发送FIFO的最高字节。
- UART的发送FIFO是一直使能的。



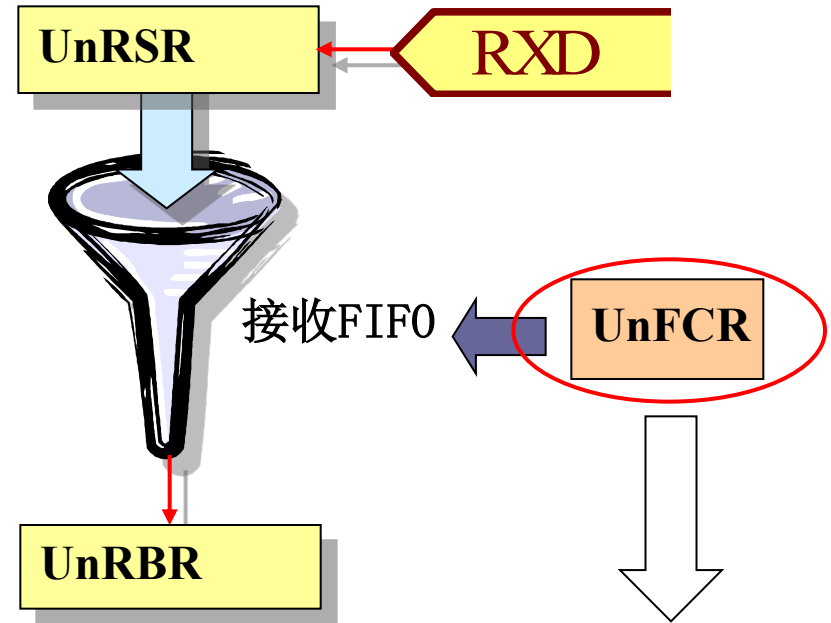
UART接收模块



UnRBR	描述	复位值
7 : 0	接收缓存寄存器，它包含了接收FIFO中最早接收到的字节	未定义

UART接收FIFO缓冲区

- UART0、UART1各含有1个16字节的接收FIFO缓冲区。
- 软件设置接收FIFO缓冲区的触发字节。



UART FIFO控制寄存器						
位	7	6	[5 : 3]	2	1	0
功能	Rx触发点设置		—	复位Tx FIFO	复位Rx FIFO	使能FIFO

4.14 UART(0、1)

- UART线状态寄存器——UnLSR

线状态寄存器 (UnLSR) 为只读寄存器，它提供UARTn发送和接收模块的状态信息。

UART线状态寄存器								
位	7	6	5	4	3	2	1	0
功能	RXFE	TEMT	THRE	BI	FE	PE	OE	RDR

4.14 UART(0、1)

• UART波特率发生器

UART0和UART1各含有一个单独的波特率发生器，两者的功能相同，且相互独立。

名称	描述		复位值
UnDLL[7:0]	除数锁存寄存器低字节	两个寄存器一起构成一个16位的除数，决定UARTn的波特率。	未定义
UnDLM[7:0]	除数锁存寄存器高字节		

这两个寄存器决定波特率时钟的频率，而波特率时钟必须是波特率的16倍。波特率计算公式如下：

$$\text{BaudRate} = F_{\text{PCLK}} / ([\text{U0DLM}, \text{U0DLL}] \times 16)$$

采样时钟

接收数据

空闲位

连续采样8次

起始位

0

数据位

1

0

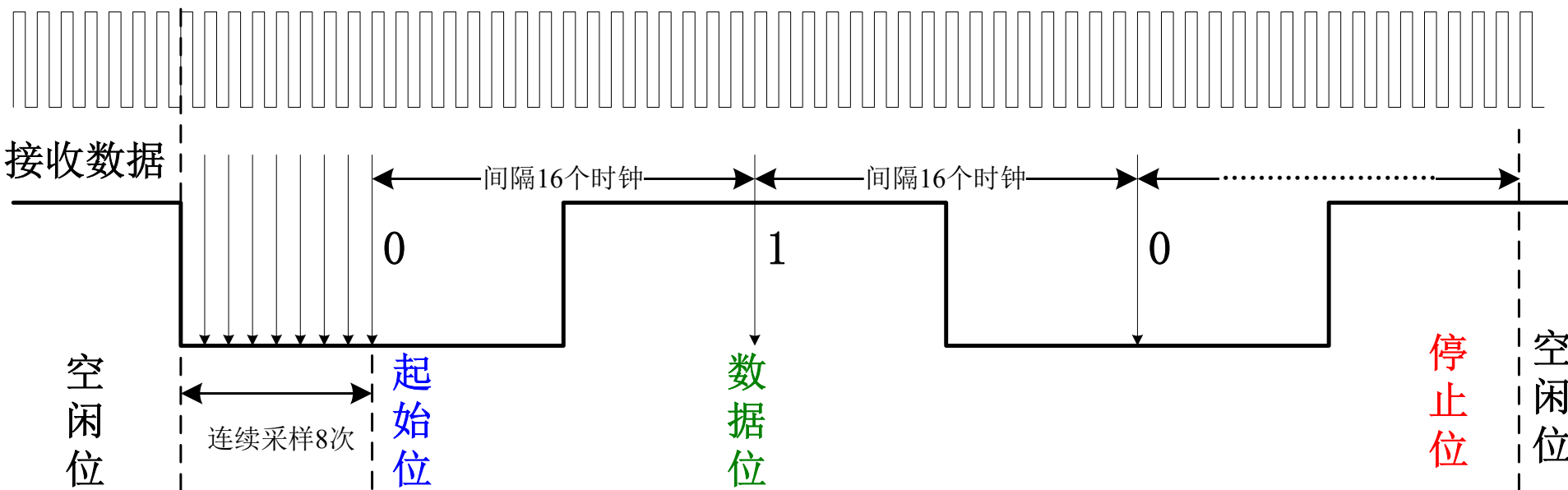
停止位

空闲位

间隔16个时钟

间隔16个时钟

.....



4.14 UART(0、1)

• Modem控制寄存器——U1MCR

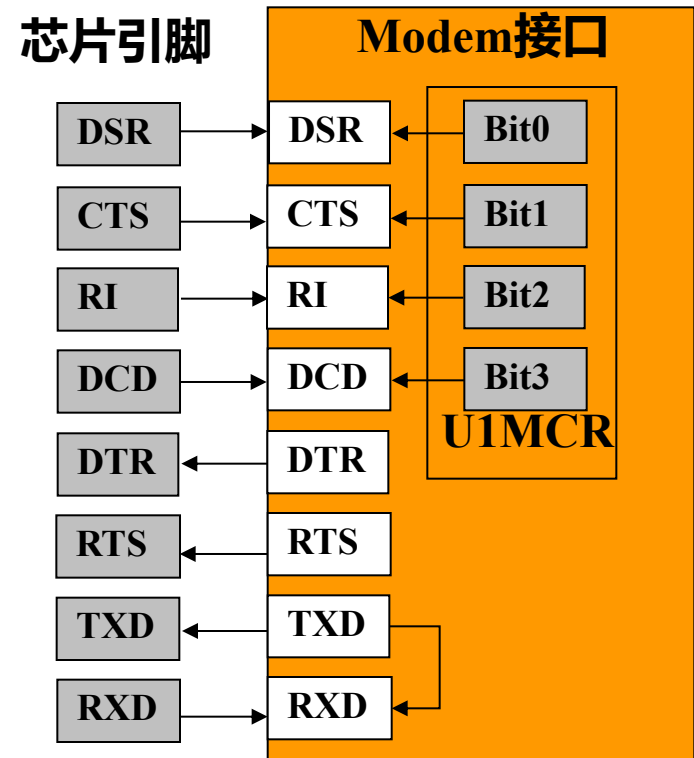
该寄存器使能Modem的回写模式，并控制Modem的输出信号。

位	功能	功能描述
0	DTR控制	选择Modem输出引脚DTR。该位在回写模式激活时读出为0。
1	RTS控制	选择Modem输出引脚RTS。该位在回写模式激活时读出为0。
3 : 2	保留	用户软件不要向该位写入1
4	回写模式	0：禁止modem回写模式 1：使能modem回写模式 modem回写模式提供了一个执行回写测试的诊断机制。
7 : 5	保留	用户软件不要向该位写入1

4.14 UART(0、1)

• 回写模式

在Modem回写模式下，发送器输出的串行数据在内部连接到接收器的串行输入端。输入脚RxD1对回写模式无影响，输出脚TxD1保持总为1的状态。4个Modem输入（CTS, DSR, RI和DCD）与外部断开。此时，U1MSR的高4位分别由U1MCR的低4位驱动。



4.14 UART(0、1)

• Modem状态寄存器

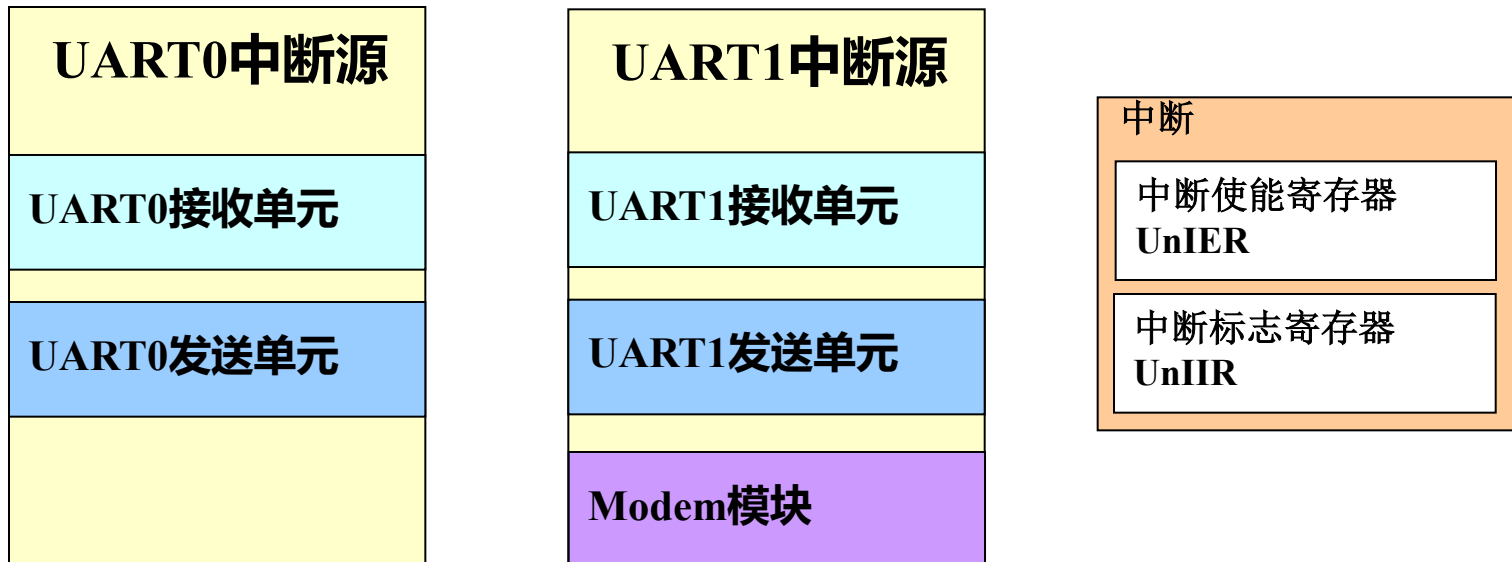
该寄存器为只读，它反映Modem输入信号的状态信息。需要注意的是，Mdoem信号对UART1的操作没有直接影响，Modem信号的操作是通过软件来实现的。

位	功能	功能描述
0	Delta CTS	检测到CTS状态发生变化时，该位置位。读取U1MSR时清零。
1	Delta DSR	检测到DSR状态发生变化时，该位置位。读取U1MSR时清零。
2	后沿 RI	检测到RI状态发生变化时，该位置位。读取U1MSR时清零。
3	Delta DCD	检测到DCD状态发生变化时，该位置位。读取U1MSR时清零。
4	CTS	反映输入信号CTS的补码。回写模式下该位连接U1MCR的bit1。
5	DSR	反映输入信号DSR的补码。回写模式下该位连接U1MCR的bit0。
6	RI	反映输入信号RI的补码。回写模式下该位连接U1MCR的bit2。
7	DCD	反映输入信号DCD的补码。回写模式下该位连接U1MCR的bit1。

4.14 UART(0、1)

- 中断接口

UART0和UART1的中断接口包含中断使能寄存器(UnIER) 和中断标识寄存器(UnIIR)。



4.14 UART(0、1)

• 中断使能寄存器

UnIER	描述	复位值
0	RBR中断使能。1：使能RDA中断；0：禁止RDA中断。	0
1	THRE中断使能。1：使能THRE中断；0：禁止THRE中断。	0
2	Rx线状态中断使能。1：使能Rx线状态中断；0：禁止Rx线状态中断；该中断状态可从UnLSR[4:1]读出。	0
3	Modem中断使能。1：禁止Modem中断；0：禁止Rx线状态中断；该中断状态可从UIMSR[3:0]读取。 注：只有UART1具有。	0
7 : 4	保留，用户软件不要向这些位写入1。	未定义

UnIER可以控制UARTn的4个中断源。其中RBR中断使能包括两个中断，一个是接收数据可用（RDA）中断，一个是接收超时中断（CTI）。稍后将对各中断源作详细介绍。



4.14 UART(0、1)

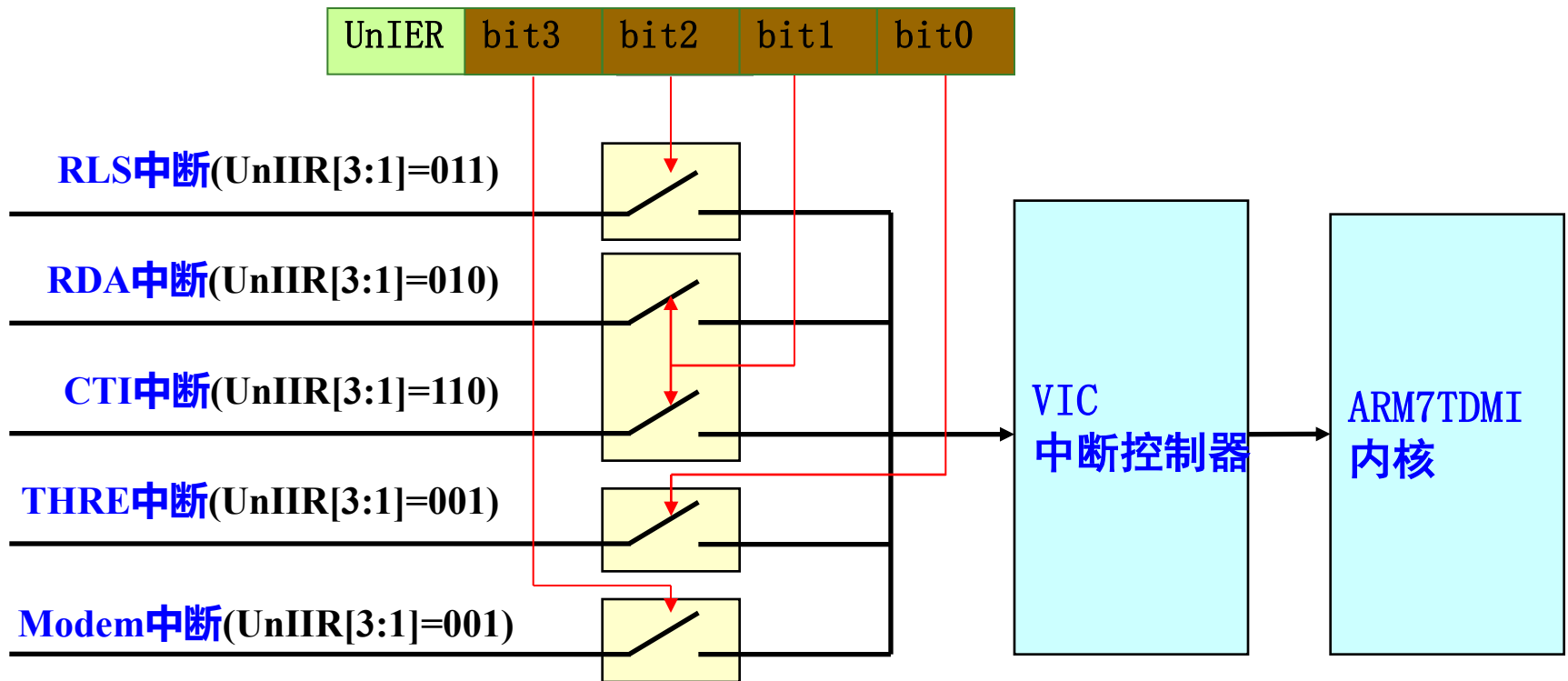
• 中断标识寄存器

UnIIR	描述		复位值
0	中断挂起。1：没有挂起的中断；0：至少有一个中断被挂起。		0
3 : 1	中断标识。 这3位表示了对应于UARTn Rx FIFO的中断。未列出的其它组合作为保留值。	011: 1. 接收线状态中断 (RLS)	0
		010: 2a. 接收数据可用中断 (RDA)	0
		110: 2b. 字符超时指示 (CTI)	
		001: 3. 发送中断 (THRE)	
		000: 4. Modem中断 注：只有UART1具有	
7 : 3	保留，用户软件不要向这些位写入1。		未定义

UnIIR提供状态代码用于指示一个挂起中断的中断源和优先级。在访问UnIIR过程中，中断被冻结。如果在访问UnIIR时产生了中断，该中断将被记录，在下次访问UnIIR时可以读出，避免了中断的丢失。

4.14 UART(0、1)

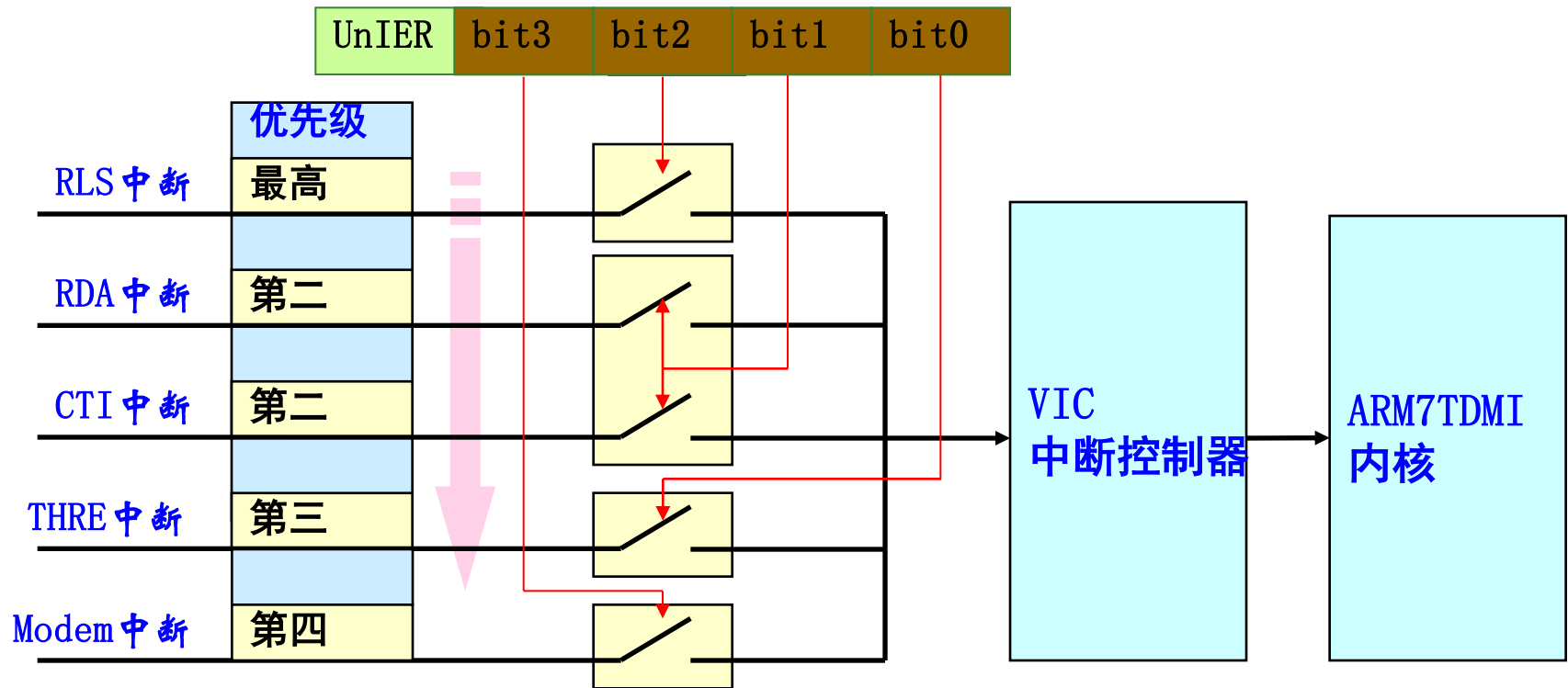
• UART中断示意图



注意：只有UART1才有Modem中断。

4.14 UART(0、1)

• UART中断优先级



注意：只有UART1才有Modem中断。

4.14 UART(0、1)

• 中断源说明

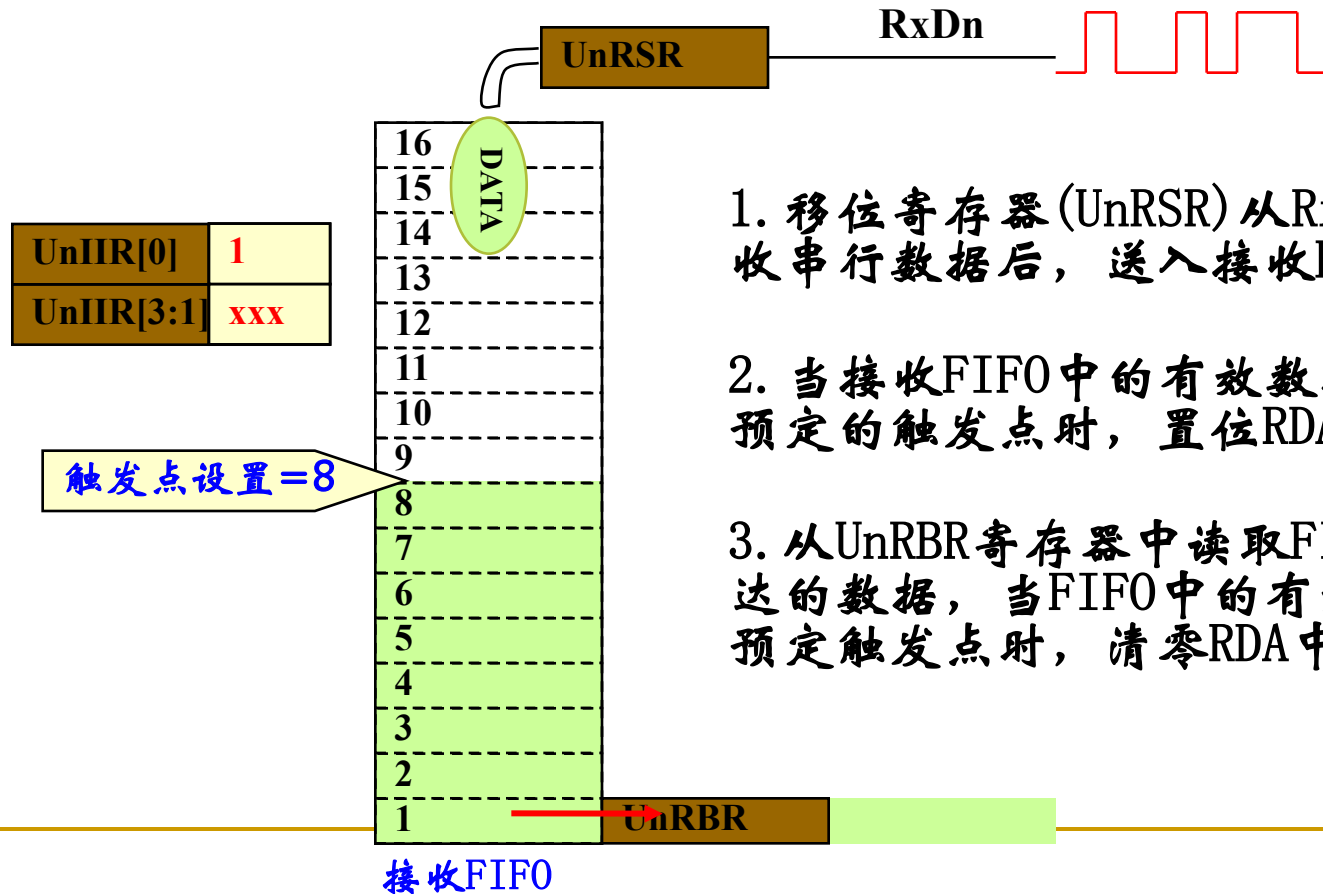
- **RLS中断**：该中断为最高优先级。它在UARTn发生下面的错误时产生中断：

- 1、溢出错误(OE)
- 2、奇偶错误(PE)
- 3、帧错误(FE)
- 4、间隔中断(BI)

通过查看UnLSR[4:1]可以了解到产生该中断的错误条件。读取UnLSR时清除该中断；

中断源说明

- **RDA中断**: 该中断与CTI中断并列为第二优先级。当接收的有效数据到达接收FIFO设置寄存器(UnFCR)中设置的触发点时, RDA被激活。当接收FIFO中的有效数据少于触发点时, RDA复位;



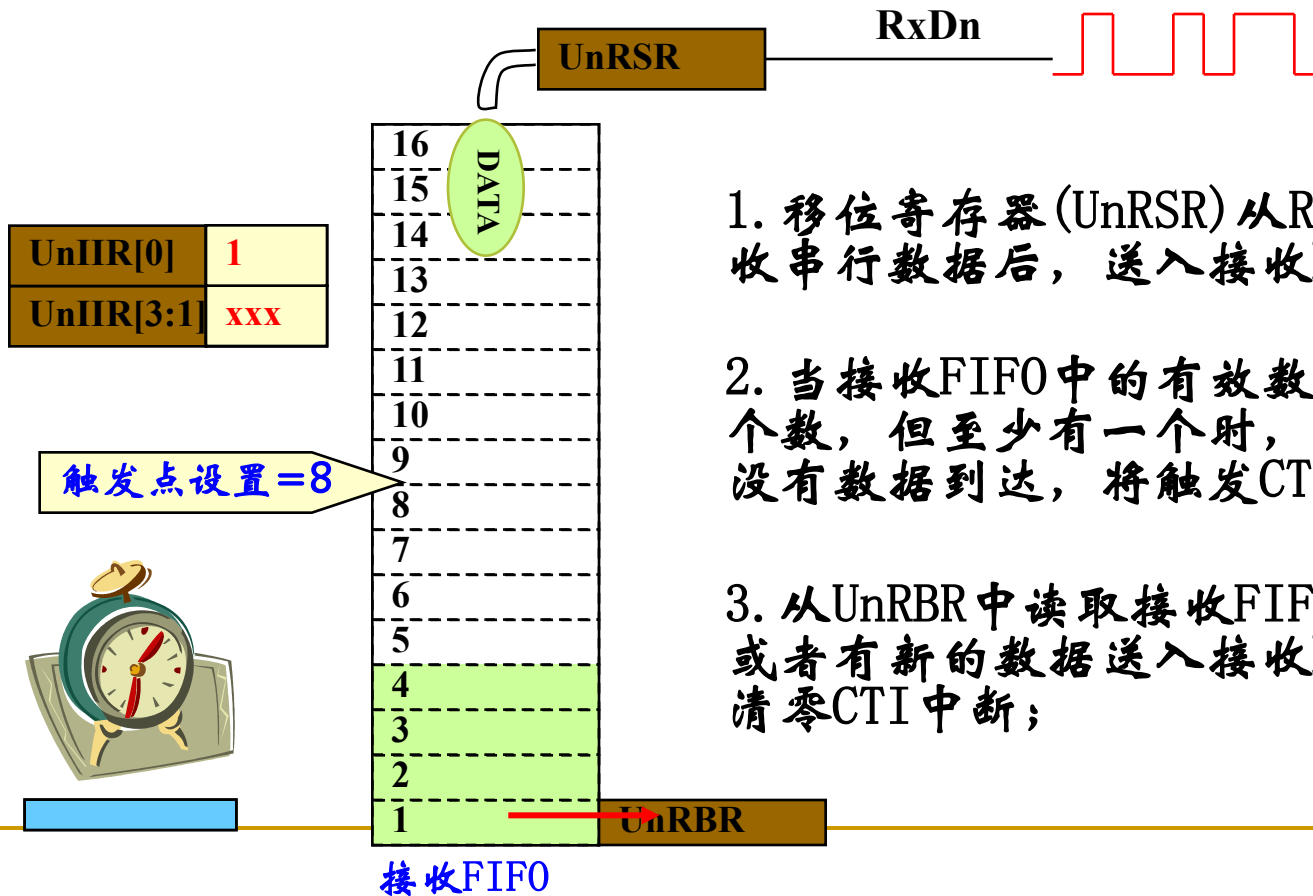
1. 移位寄存器(UnRSR)从RxDn引脚接收串行数据后, 送入接收FIFO中;

2. 当接收FIFO中的有效数据数量到达预定的触发点时, 置位RDA中断;

3. 从UnRBR寄存器中读取FIFO中最早到达的数据, 当FIFO中的有效数据小于预定触发点时, 清零RDA中断;

中断源说明

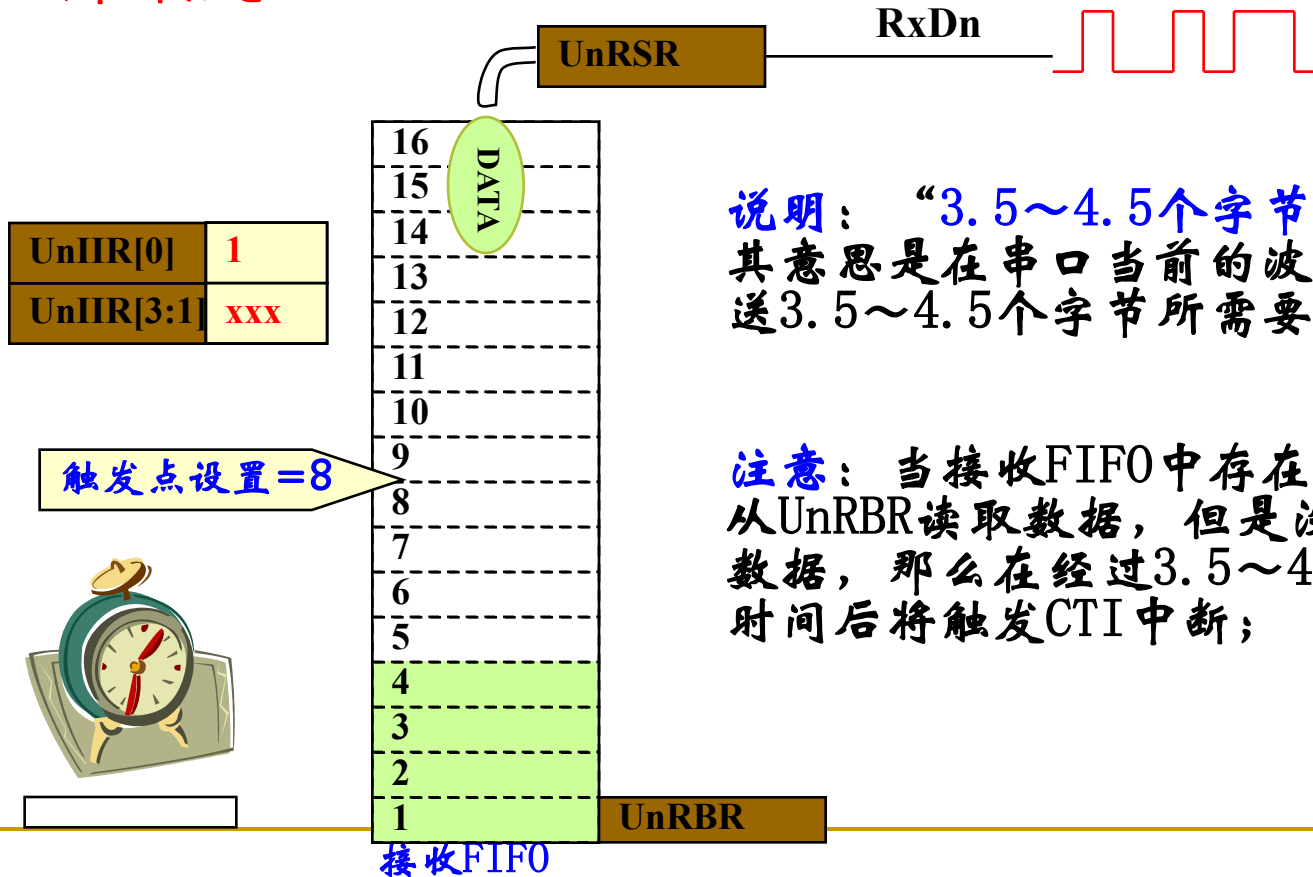
- **CTI中断**：当接收FIFO中的有效数据少于预定的触发点数量（至少有一个字节）时，如果在一定时间内仍然没有接收到新的数据，那将触发该中断。这个时间为：3.5~4.5个字节所需要的时间。**注：对接收FIFO的任何操作都会清零该中断标志。**



1. 移位寄存器(UnRSR)从RxDn引脚接收串行数据后，送入接收FIFO中；
2. 当接收FIFO中的有效数据少于触发个数，但至少有一个时，如果长时间没有数据到达，将触发CTI中断；
3. 从UnRBR中读取接收FIFO中的数据，或者有新的数据送入接收FIFO，都将清零CTI中断；

中断源说明

- **CTI中断**：当接收FIFO中的有效数据少于预定的触发点数量（至少有一个字节）时，如果在一定时间内仍然没有接收到新的数据，那将触发该中断。这个时间为：3.5~4.5个字节所需要的时间。**注：对接收FIFO的任何操作都会清零该中断标志。**



说明：“3.5~4.5个字节的时间”，其意思是在串口当前的波特率下，发送3.5~4.5个字节所需要的时间；

注意：当接收FIFO中存在多个数据，从UnRBR读取数据，但是没有读完所有数据，那么在经过3.5~4.5个字节的时间后将触发CTI中断；

中断源说明

- THRE中断**: 该中断为第三优先级。当发送FIFO为空并且满足一定的条件时, 该中断将被触发。这些条件是:

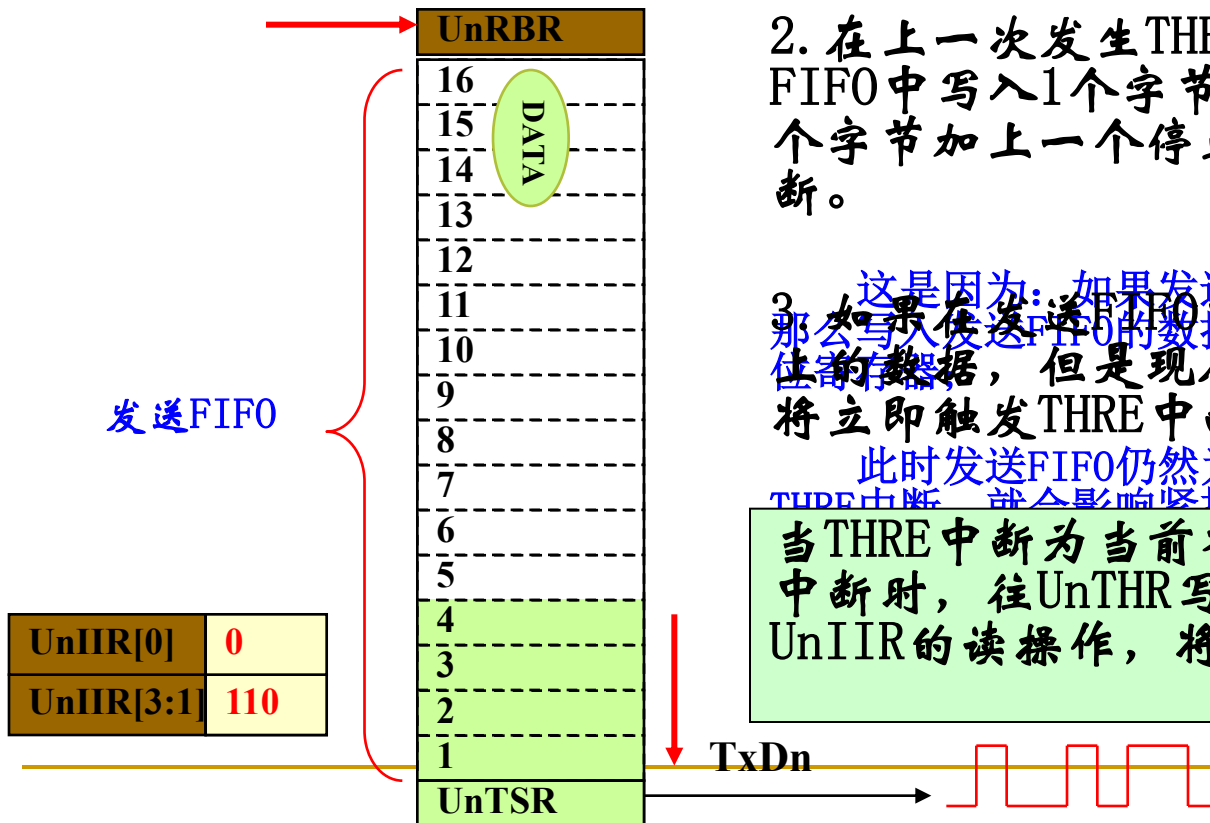
1. 系统启动时, 虽然发送FIFO为空, 但不会产生THRE中断。

2. 在上一次发生THRE中断后, 向发送FIFO中写入1个字节数据, 将在延时一个字节加上一个停止位后发生THRE中断。

3. 这是因为: 如果发送移位寄存器为空, 那么在发送FIFO中写入两个字节以上的数据, 但是现在发送FIFO为空时, 将立即触发THRE中断。

此时发送FIFO仍然为空, 如果立即产生THRE中断, 就会影响紧接着写入发送FIFO。

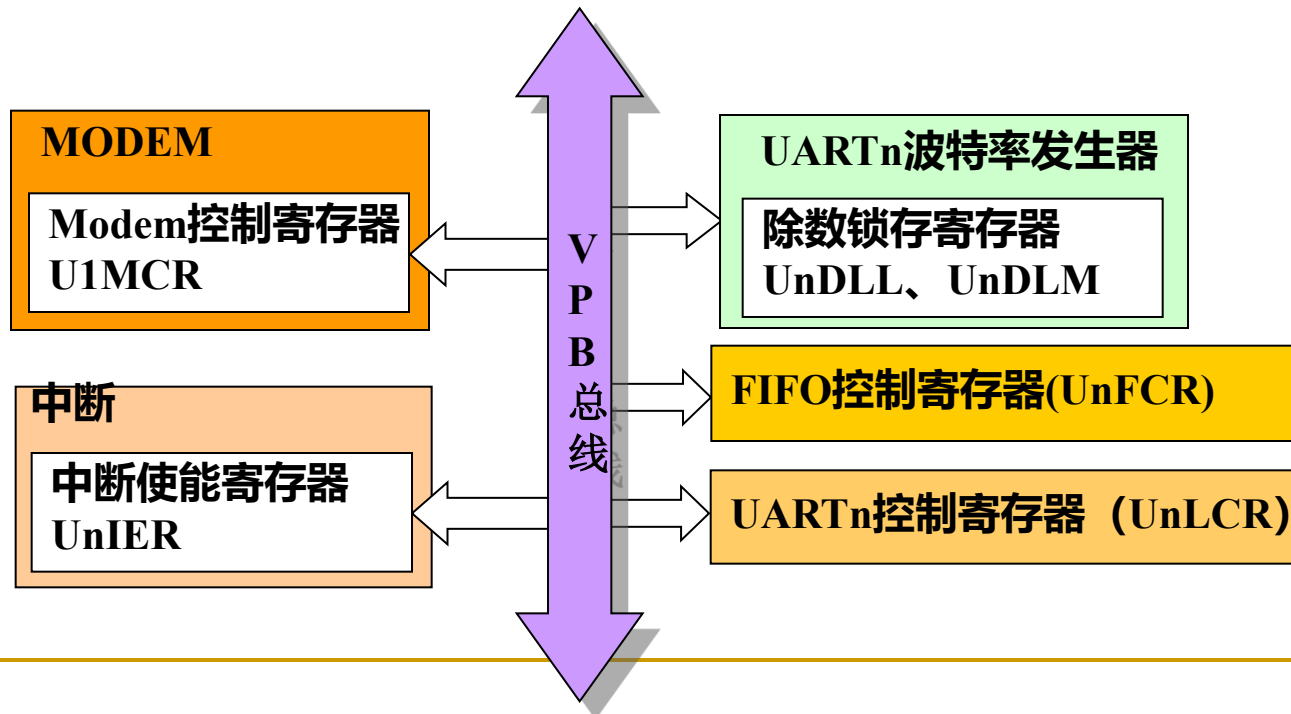
当THRE中断为当前有效的最高优先级中断时, 往UnTHR写数据, 或者对UnIIR的读操作, 将使THRE中断复位。



4.14 UART(0、1)

- UART设置

使用UART前需要设置的寄存器



4.14 UART(0、1)

- UART设置

使用UART前需要设置的寄存器

寄存器名称	功能
UnLCR	设置UARTn的通信格式。
UnFCR	设置UARTn的接收FIFO缓冲区。
UnDLL、 UnDLM	设置UARTn的通信波特率。
UnIER	设置UARTn的中断。
U1MCR	Modem接口设置（只有UART1具有）。

4.14 UART(0、1)

- UART设置

使用UART前需要设置的寄存器——UnLCR，
设置UART通信字符长度、停止位个数、奇偶校验
位等参数。

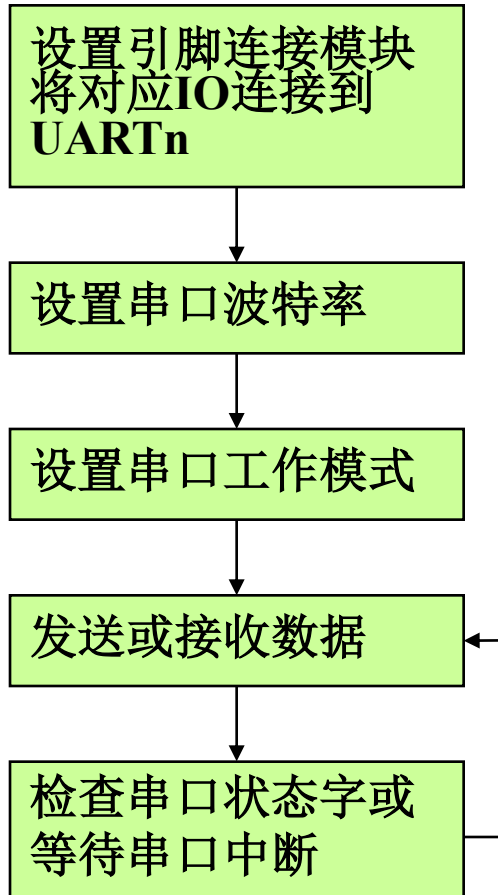
UART控制寄存器						
位	7	6	[5 : 4]	6	2	[1 :0]
功能	除数锁存	间隔	奇偶选择	奇偶设置	停止位	字长

• UART——特殊寄存器

位置相同的寄存器

UART	寄存器	地址	访问方式
UART0	UORBR	0xE000 C000	DLAB=0, 对地址: 0xE000 C000进行 读 访问
	UOTHR		DLAB=0, 对地址: 0xE000 C000进行 写 访问
	UODLL		DLAB=1, 对地址: 0xE000 C000进行访问
	UOIER	0xE000 C004	DLAB=0, 对地址: 0xE000 C004进行访问
	UODLM		DLAB=1, 对地址: 0xE000 C004进行访问
UART1	U1RBR	0xE001 0000	DLAB=0, 对地址: 0xE001 0000进行 读 访问
	U1THR		DLAB=0, 对地址: 0xE001 0000进行 写 访问
	U1DLL		DLAB=1, 对地址: 0xE001 0000进行访问
	U1IER	0xE001 0004	DLAB=0, 对地址: 0xE001 0004进行访问
	U1DLM		DLAB=1, 对地址: 0xE001 0004进行访问

• UART应用示例——操作流程



• UART应用示例——初始化代码

UART0初始化代码:

```
#define UART_BPS 115200
```

定义表示波特率的宏，方便修改

```
Void UART0_Ini(void)  
{
```

```
    uint16 Fdiv;
```

```
    PINSEL0 = 0x00000005;
```

设置引脚连接模块

```
    UOLCR = 0x83;
```

置位除数锁存位

```
    Fdiv = (Fpclk / 16) / UART_BPS;
```

根据波特率计算分频值

```
    UODLM = Fdiv / 256;
```

```
    UODLL = Fdiv % 256;
```

设置除数寄存器
(除数锁存访问位必须置位)

```
    UOLCR = 0x03;
```

清除除数锁存位，并设置工作模式

```
}
```


UART应用示例——初始化代码

```
#define UART_BPS 115200
```

```
Void UART0_Ini(void)
```

```
{
```

```
    uint16 Fdiv;
```

```
    PINSEL0 = 0x00000005;
```

```
    UOLCR = 0x83;
```

```
    Fdiv = (Fpclk / 16) / UART_BPS;
```

```
    UODLM = Fdiv / 256;
```

```
    UODLL = Fdiv % 256;
```

```
    UOLCR = 0x03;
```

```
}
```

设置引脚连接模块
将对应IO连接到
UART0

设置串口波特率

设置串口工作模式

发送或接收数据

检查串口状态字或
等待串口中断

UART应用示例——收发数据

查询方式发送一字节数据：

```
void UART0_SendByte(uint8 data)
{
    U0THR = data;
    while((U0LSR & 0x40) == 0);
}
```

将要发送的一字节数据写入U0THR

等待数据发送完毕

查询方式接收一字节数据：

```
uint8 UART0_RcvByte(void)
{
    uint8 rcv_data;
    while((U0LSR & 0x01) == 0);
    rcv_data = U0RBR;
    return(rcv_data);
}
```

等待数据到达

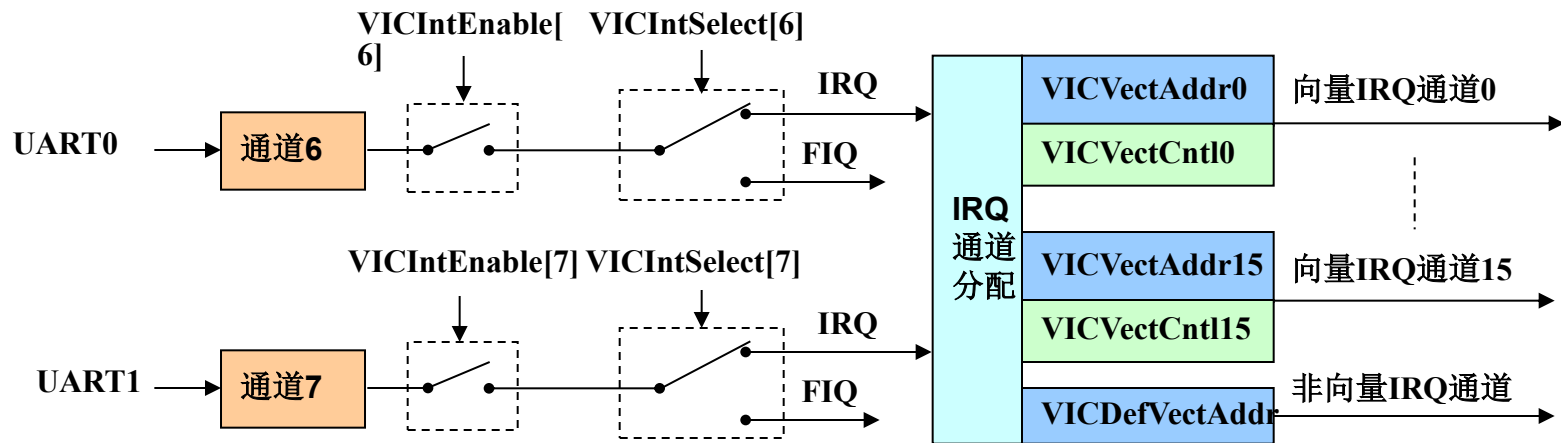
从U0RBR中读出接收的数据

返回接收的数据

UART中断

- UART与VIC的关系

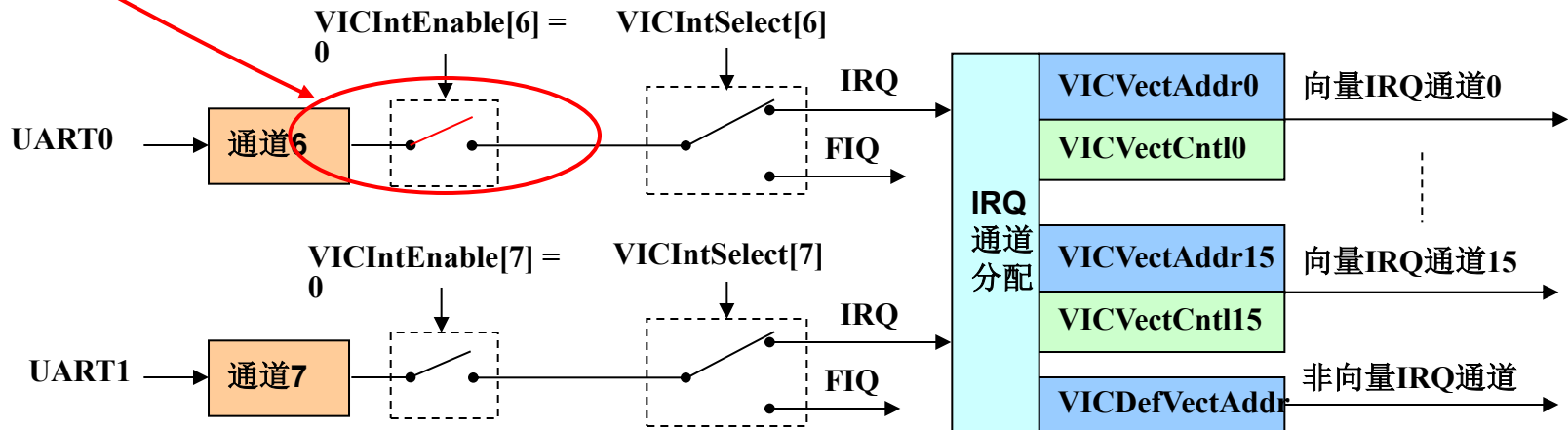
UART0、UART1分别位于VIC的通道6和通道7。中断使能寄存器VICIntEnable的Bit6和Bit7分别用来控制通道6和通道7的使能。



UART中断

• UART0与VIC的关系

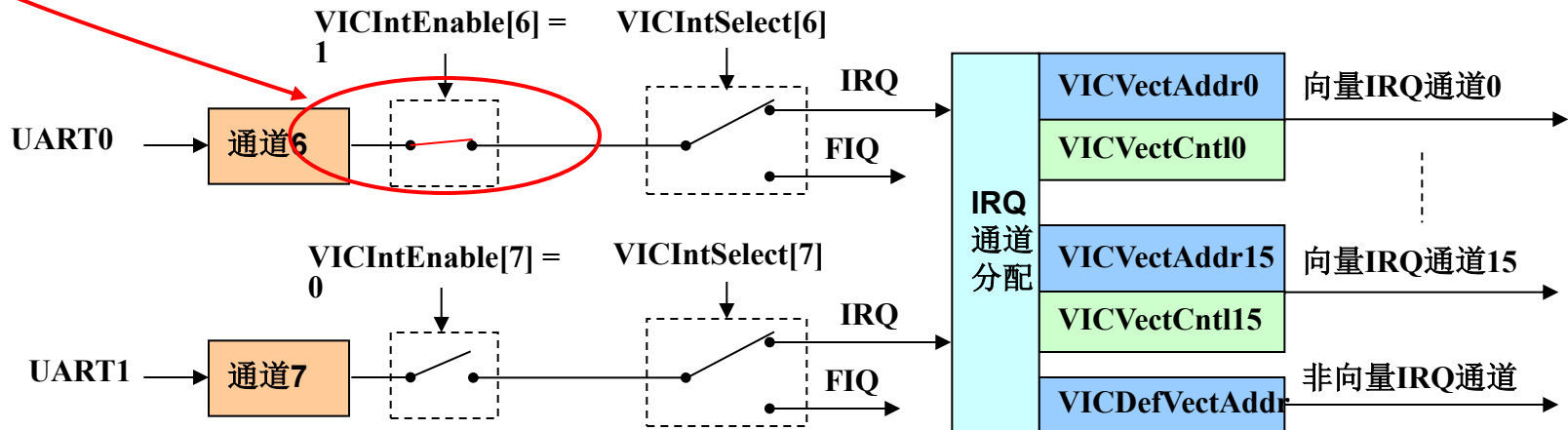
➤ 当 $VICIntEnable[6] = 0$ 时，通道6中断禁止；



UART中断

• UART0与VIC的关系

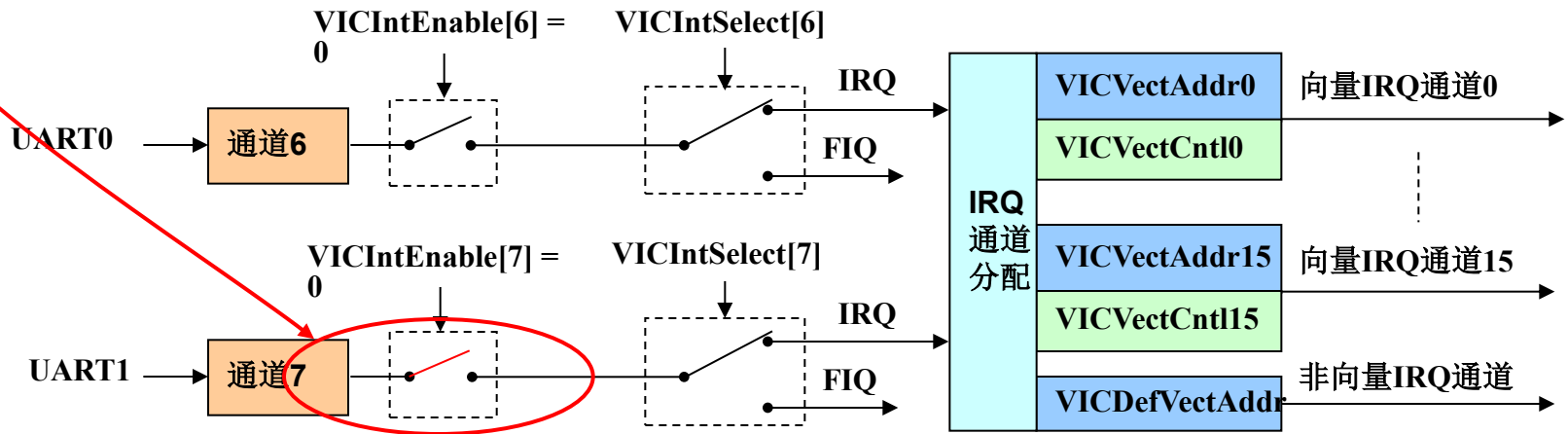
- 当 $VICIntEnable[6] = 0$ 时，通道6中断禁止；
- 当 $VICIntEnable[6] = 1$ 时，通道6中断使能。



UART中断

• UART1与VIC的关系

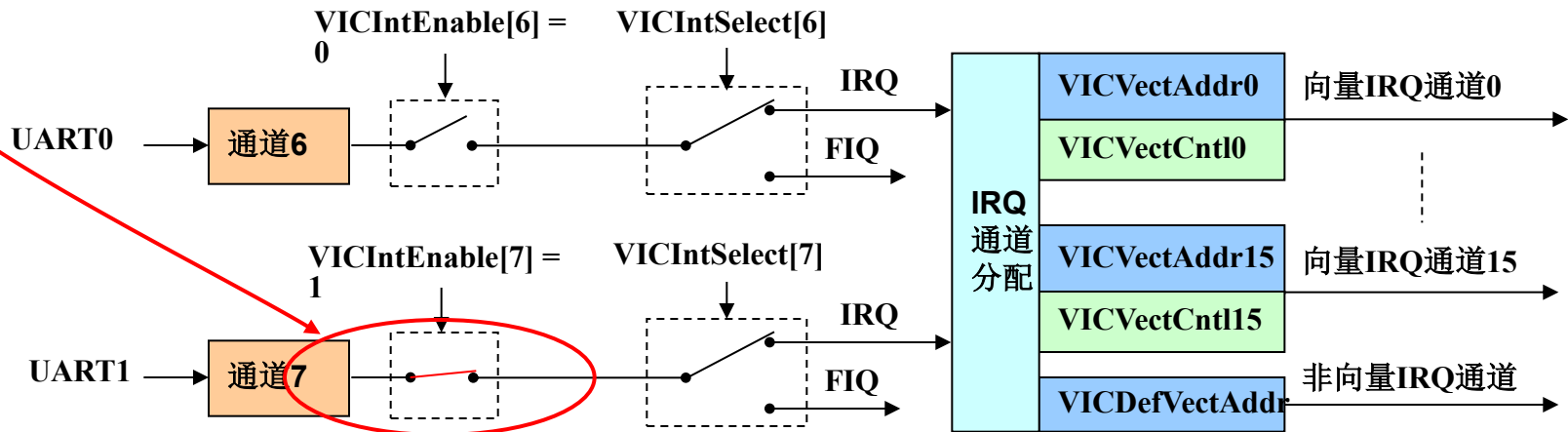
➤ 当 $VICIntEnable[7] = 0$ 时，通道7中断禁止；



UART中断

• UART1与VIC的关系

- 当 $VICIntEnable[7] = 0$ 时，通道7中断禁止；
- 当 $VICIntEnable[7] = 1$ 时，通道7中断使能。





UART中断

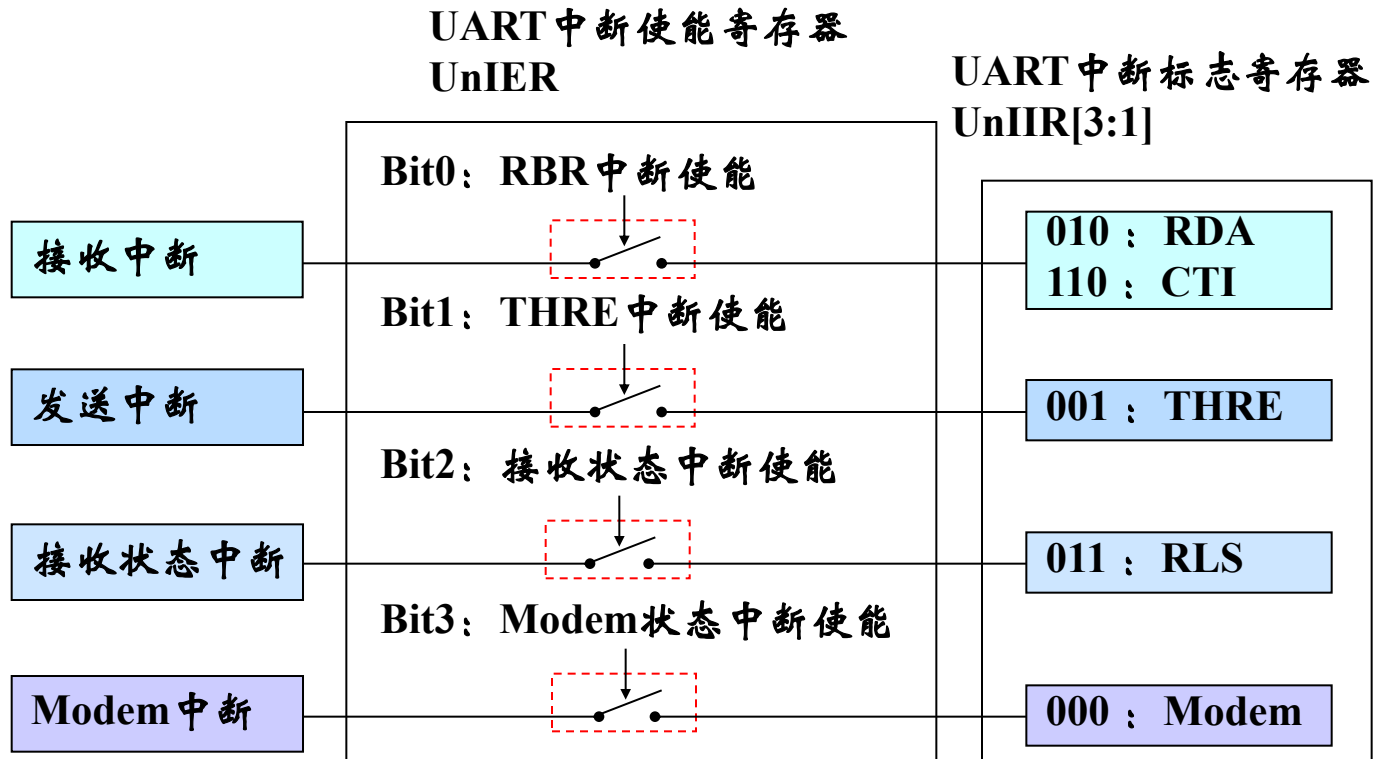
- UART 中断

LPC2000 系列 ARM UART 中断分为 四类：

- 接收中断
 - ✓ 接收超时中断 (CTI)
 - ✓ 接收数据可用中断 (RDA)
- 发送中断 (THRE)
- 接收状态中断 (RLS)
- Modem 中断 (Modem)

UART中断

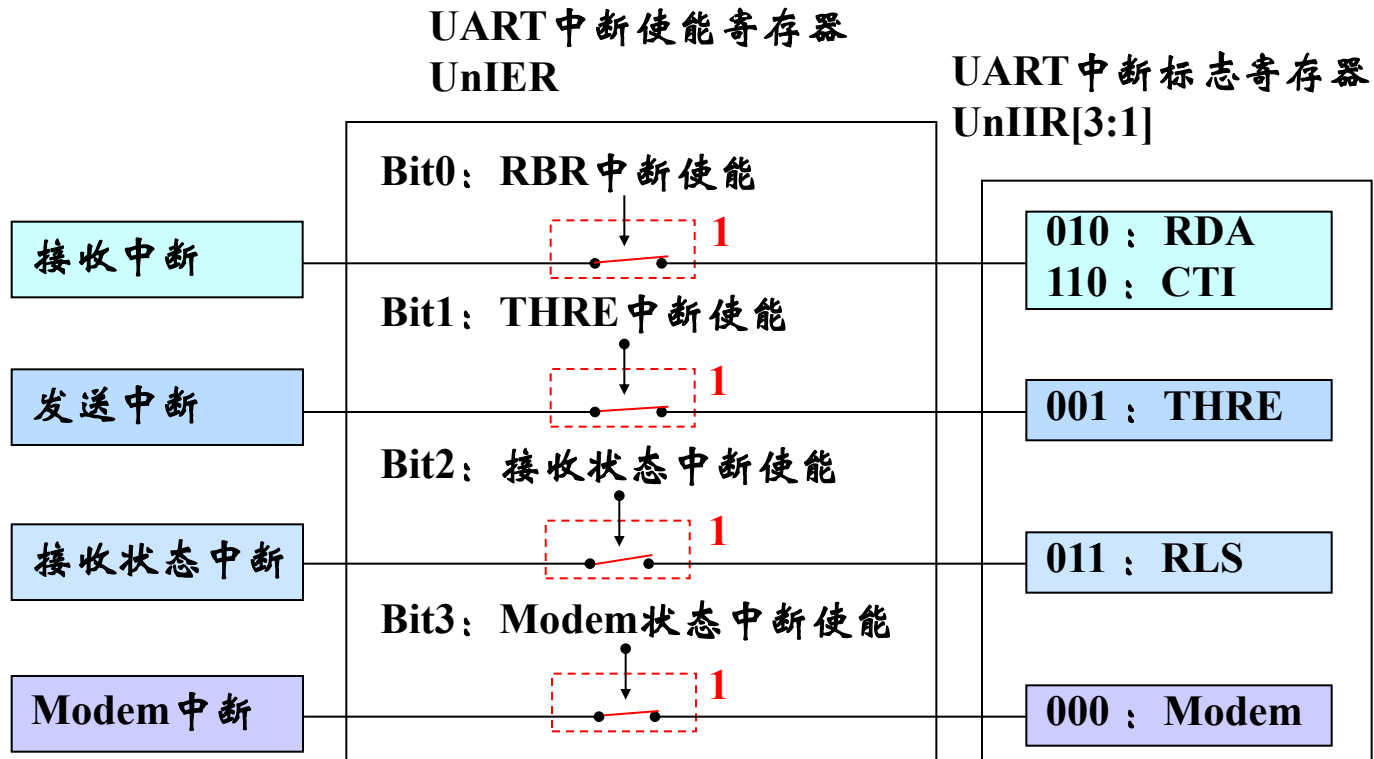
• UART中断示意图



注意：只有UART1具有Modem中断。

UART中断

UART中断示意图



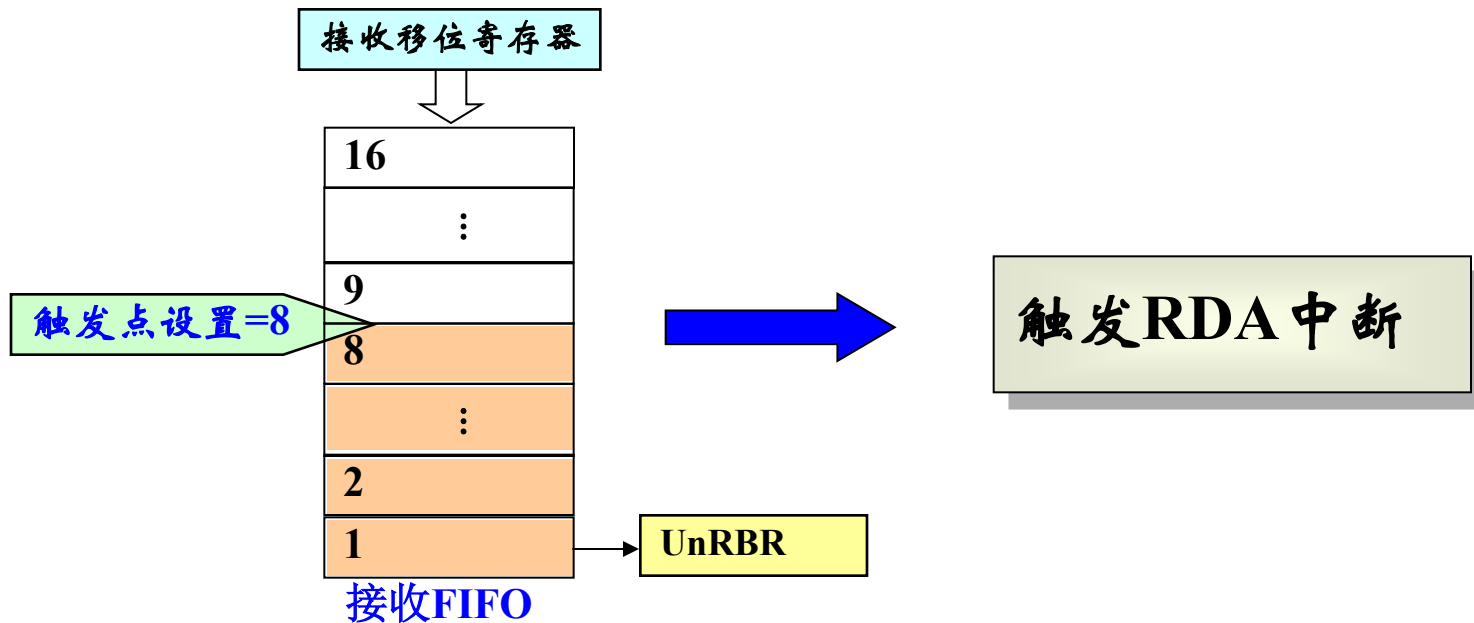
中断使能位为1时，对应的中断使能

注意：只有UART1具有Modem中断。

UART中断总结

- 接收中断——数据可用中断 (RDA)

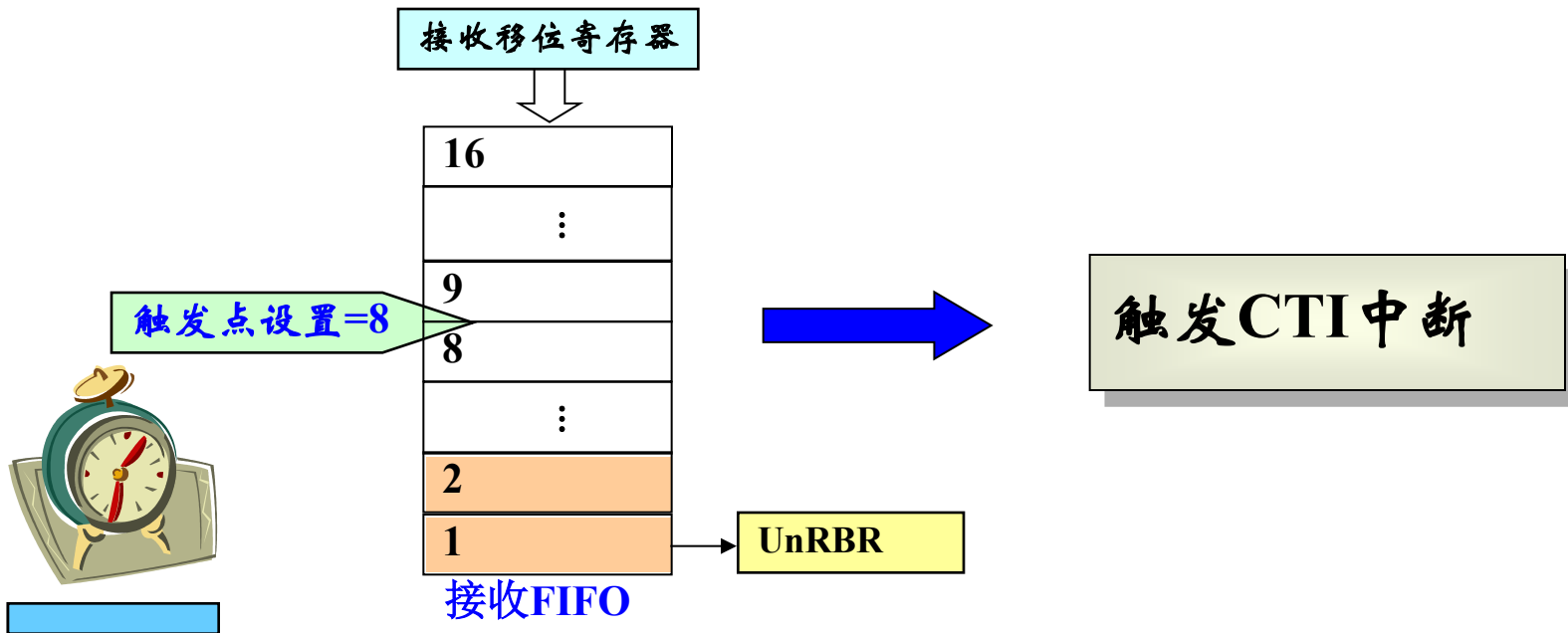
LPC2000系列ARM UART接口具有16字节的接收FIFO，接收触发点可以设置为1、4、8、14字节，当接收到的字节数达到接收触发点时，便会触发中断。



UART中断总结

- 接收中断——接收超时中断 (CTI)

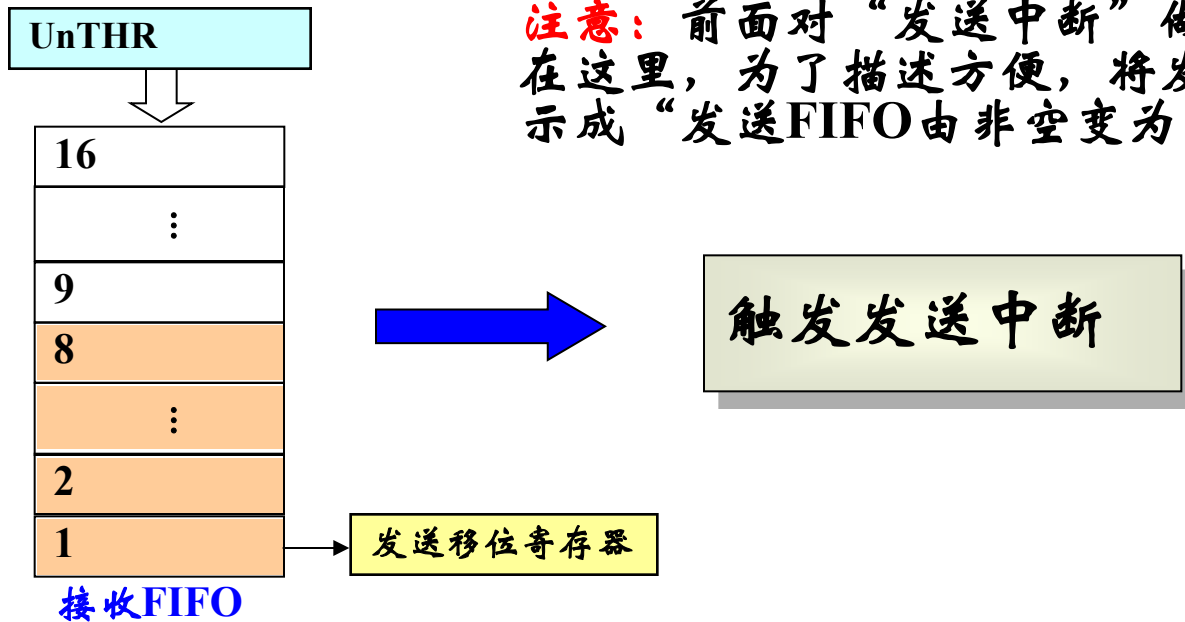
当接收FIFO中的有效数据个数少于触发个数时（注：接收FIFO中至少有一个字节），如果在3.5到4.5个字符的时间内，没有收到其它数据，将触发CTI中断。



UART中断总结

- 发送中断

LPC2000系列ARM UART接口具有16字节的发送FIFO，当发送FIFO由非空变为空时，便会触发“发送中断”。

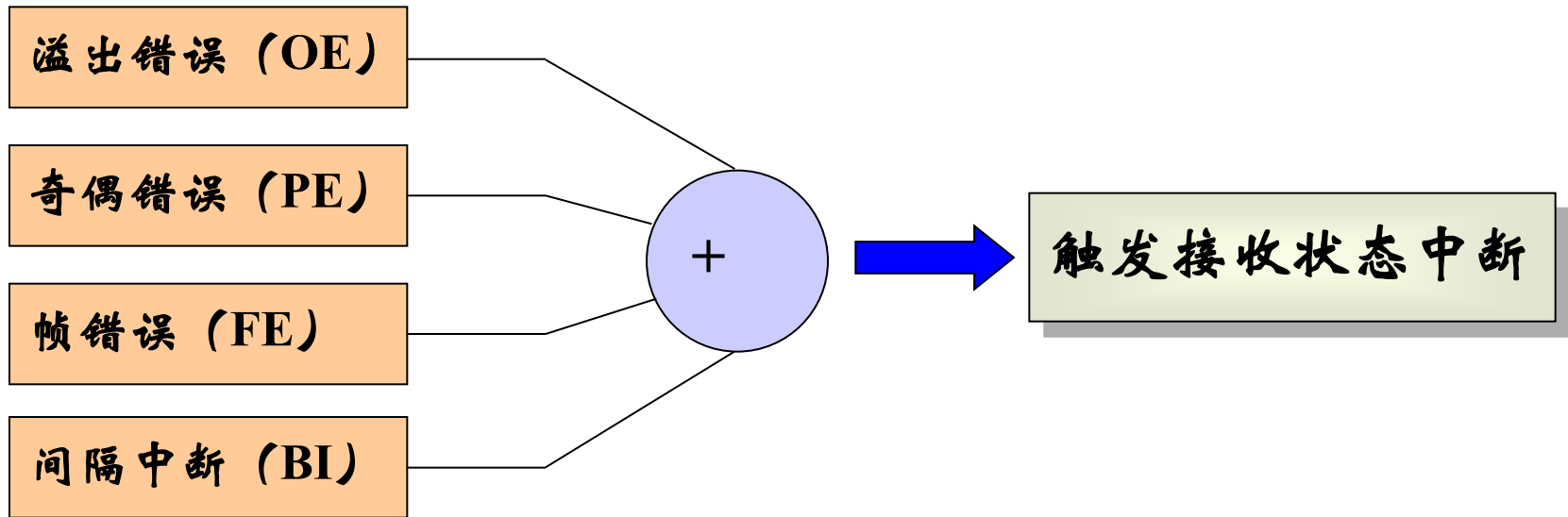


注意：前面对“发送中断”做了详细的描述，在这里，为了描述方便，将发送中断简单表示成“发送FIFO由非空变为空”。

UART中断总结

- 接收状态中断

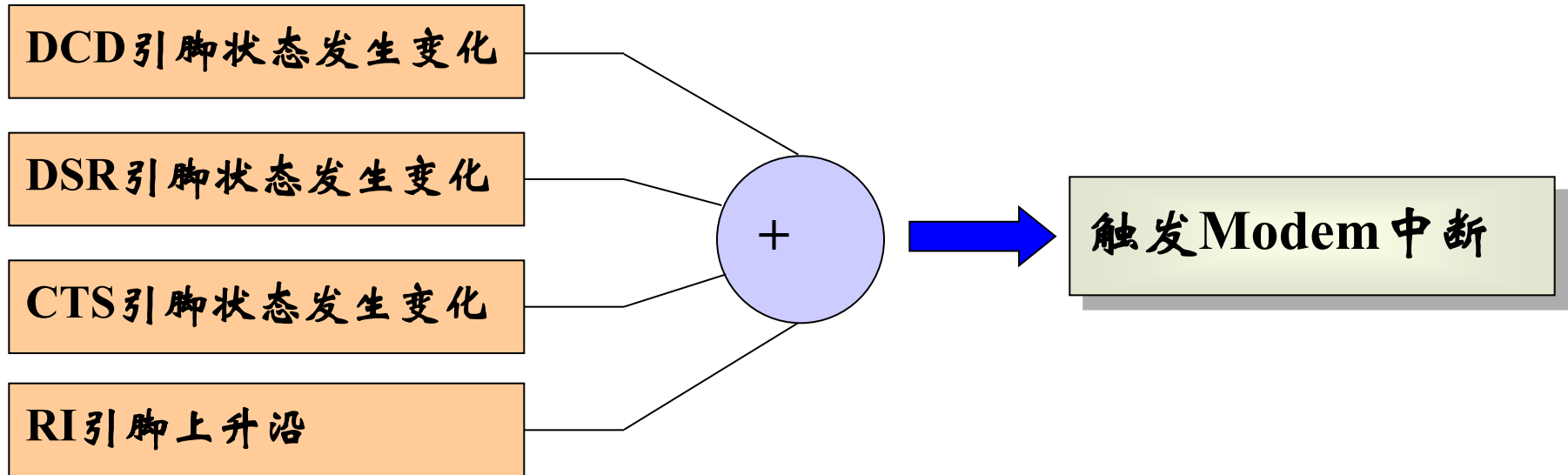
在UART接收数据时，如果出现：溢出错误（OE）、奇偶错误（PE）、帧错误（FE）和间隔中断（BI）中的任意一个错误时，都会触发接收状态中断。



UART中断总结

- Modem中断

UART1接口具有Modem中断，当引脚DCD、DSR或CTS上发生状态变化时，都会触发Modem中断。此外，Modem输入引脚RI上低到高电平的跳变也会产生一个Modem中断。



■ 第一个区别当然是名字：

SPI(Serial Peripheral Interface: 串行外设接口);

I2C(INTER IC BUS: 意为IC之间总线)

UART(Universal Asynchronous Receiver Transmitter: 通用异步收发器)

第二，区别在电气信号线上：

SPI总线由三条信号线组成：串行时钟(**SCLK**)、串行数据输出(**SDO**)、串行数据输入(**SDI**)。**SPI**总线可以实现 多个**SPI**设备互相连接。提供**SPI**串行时钟的**SPI**设备为**SPI**主机或主设备(**Master**)，其他设备为**SPI**从机或从设备(**Slave**)。主从设备间可以实现全双工通信，当有多个从设备时，还可以增加一条从设备选择线。

如果用通用**IO**口模拟**SPI**总线，必须要有一个输出口(**SDO**)，一个输入口(**SDI**)，另一个口则视实现的设备类型而定，如果要实现主从设备，则需输入输出口，若只实现主设备，则需输出口即可，若只实现从设备，则只需输入口即可。

I2C总线是双向、两线(**SCL**、**SDA**)、串行、多主控 (**multi-master**) 接口标准，具有总线仲裁机制，非常适合在器件之间进行近距离、非经常性的数据通信。在它的协议体系中，传输数据时都会带上目的设备的设备地址，因此可以实现设备组网。

如果用通用**IO**口模拟**I2C**总线，并实现双向传输，则需一个输入输出口(**SDA**)，另外还需一个输出口(**SCL**)。（注：**I2C**资料了解得比较少，这里的描述可能很不完备）

UART总线是异步串口，因此一般比前两种同步串口的结构要复杂很多，一般由波特率产生器(产生的波特率等于传输波特率的16倍)、**UART**接收器、**UART**发送器组成，硬件上由两根线，一根用于发送，一根用于接收。

显然，如果用通用**IO**口模拟**UART**总线，则需一个输入口，一个输出口。

第三，从第二点明显可以看出，**SPI**和**UART**可以实现全双工，但**I2C**不行；



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.15 A/D转换器

■ 特性

LPC2114/2124具有一个AD转换器，LPC2200系列具有两个AD转换器，它们具有如下特性：

- 10位逐次逼近式模式转换器；
- 测量范围：0~3.3V；
- 10位转换事件 $\geq 2.44\mu\text{s}$ ；
- 可设置AD转换触发方式；
- 具有掉电模式。

4.15 A/D转换器

■ A/D转换器描述

A/D转换器的基本时钟由VPB时钟提供。可编程分频器可将时钟调整至逐步逼近转换所需的4.5MHz（最大）。如要得到10位精度的结果，需要11个A/D转换时钟。

A/D转换器的参考电压来自 V_{3A} 和 V_{SSA} 引脚。



4.15 A/D转换器

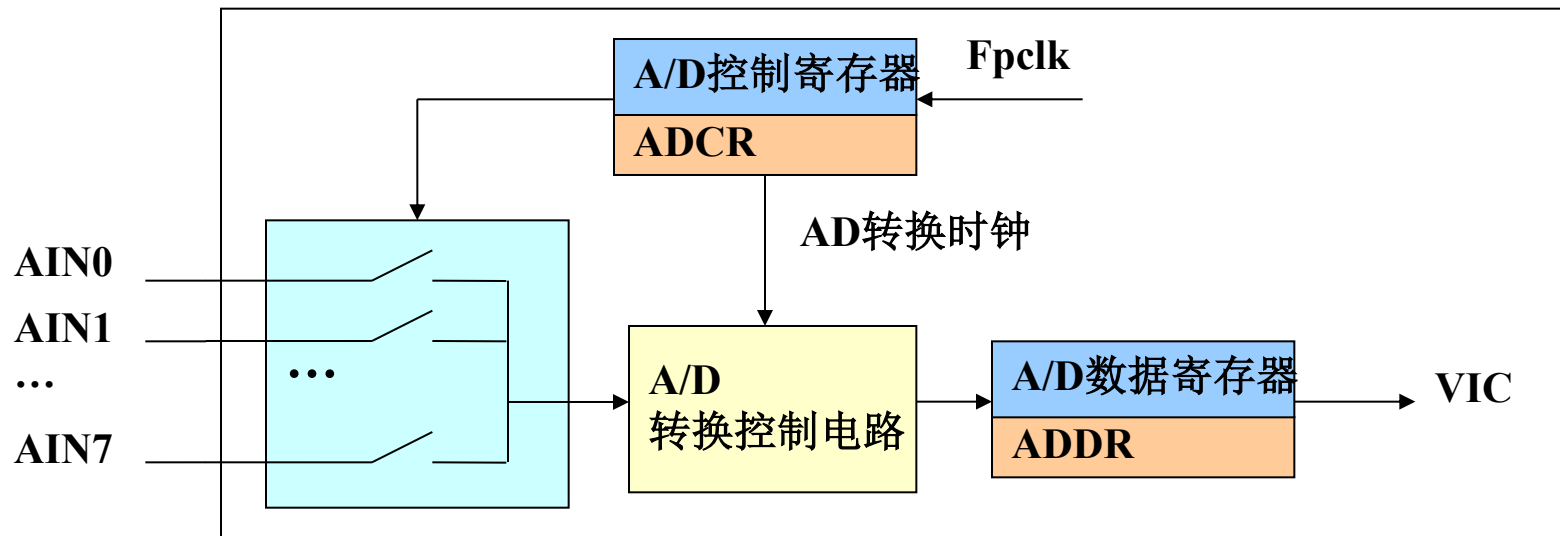
■ A/D转换器描述

A/D引脚描述

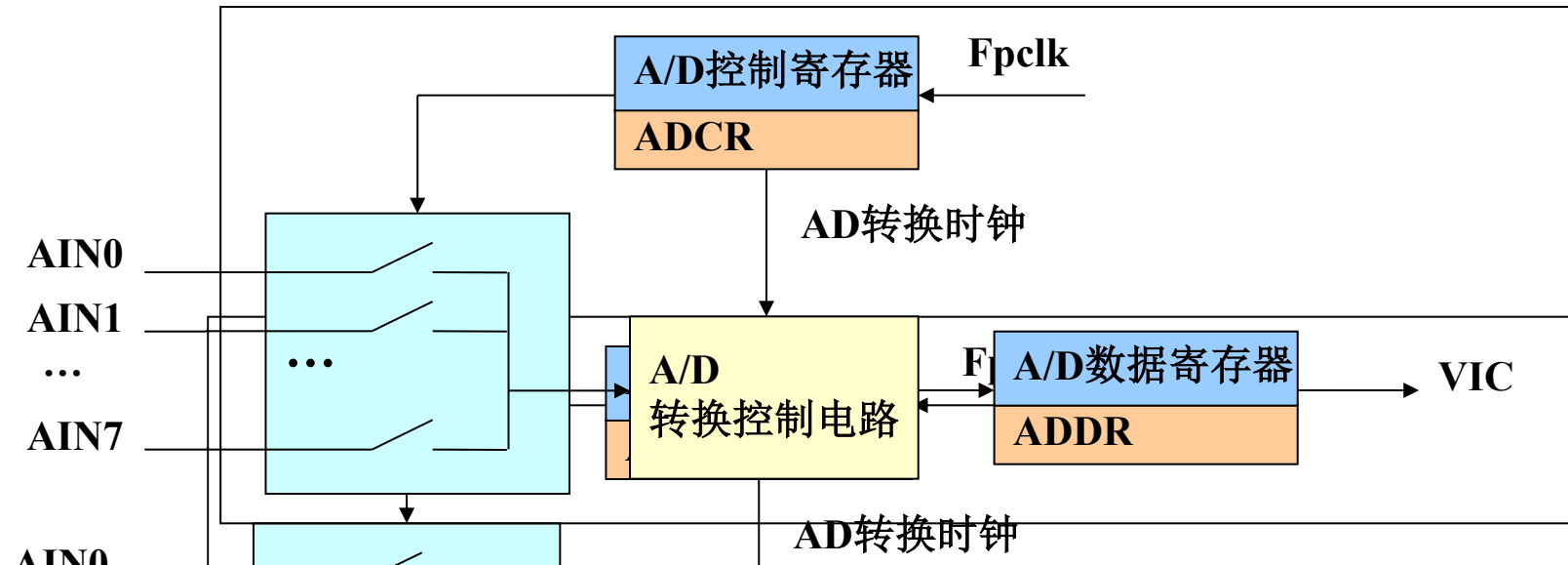
引脚名称	类型	引脚描述
AIN7~AIN0	输入	模拟输入 。A/D转换器单元可分时测量这8个引脚上的输入信号电压。即使这些引脚设置为GPIO功能，仍可以使用A/D转换器部件。
V_{3A}, V_{SSA}	电源	模拟电源和地 。它们分别与标称的 V_3 和 V_{SSD} 的电压相同，但为了降低噪声和出错几率，两者应当隔离。转换器的 V_{refP} 和 V_{refN} 信号在内部与这两个电源信号相连。

4.15 A/D转换器

■ A/D转换器内部结构

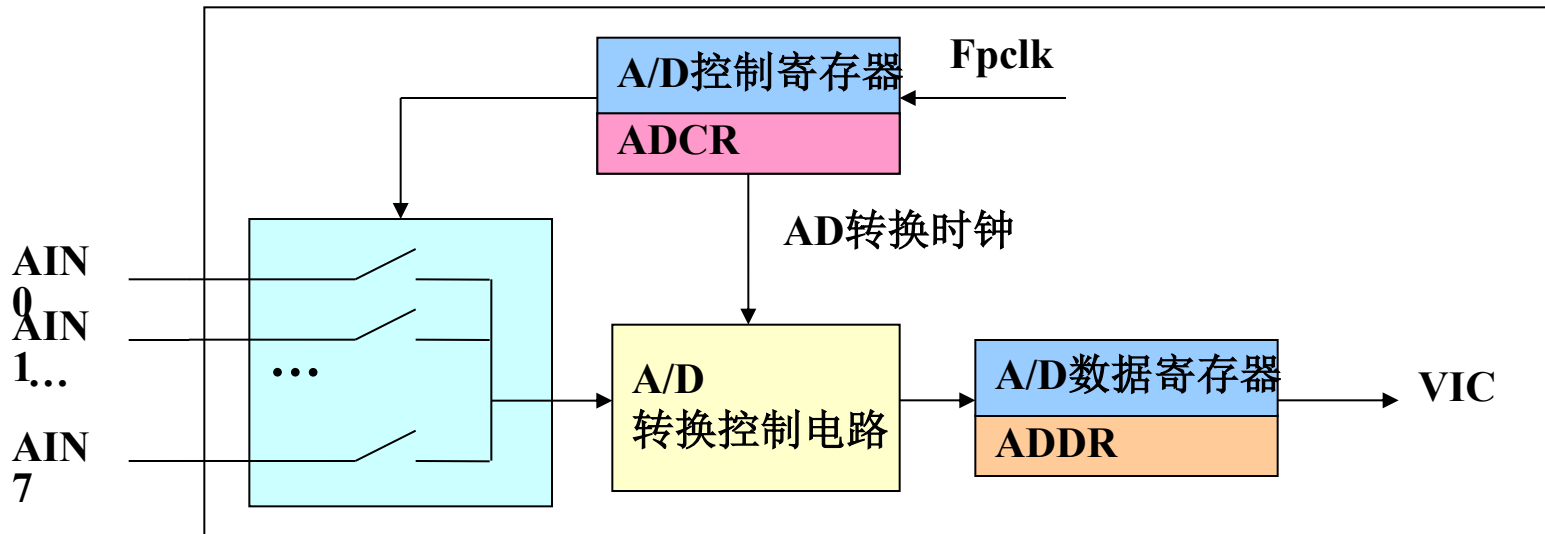


A/D转换器寄存器描述



名称	描述	访问	复位值
ADCR	A/D控制寄存器。A/D转换开始前，必须设置ADCR寄存器来选择工作模式。	读写	0x0000001
ADDR	A/D数据寄存器。该寄存器包含ADC的结束标志位和10位的转换结果（当结束标志位为1时，转换结果才是有效的）。	读写	NA

A/D转换器寄存器描述—控制寄存器



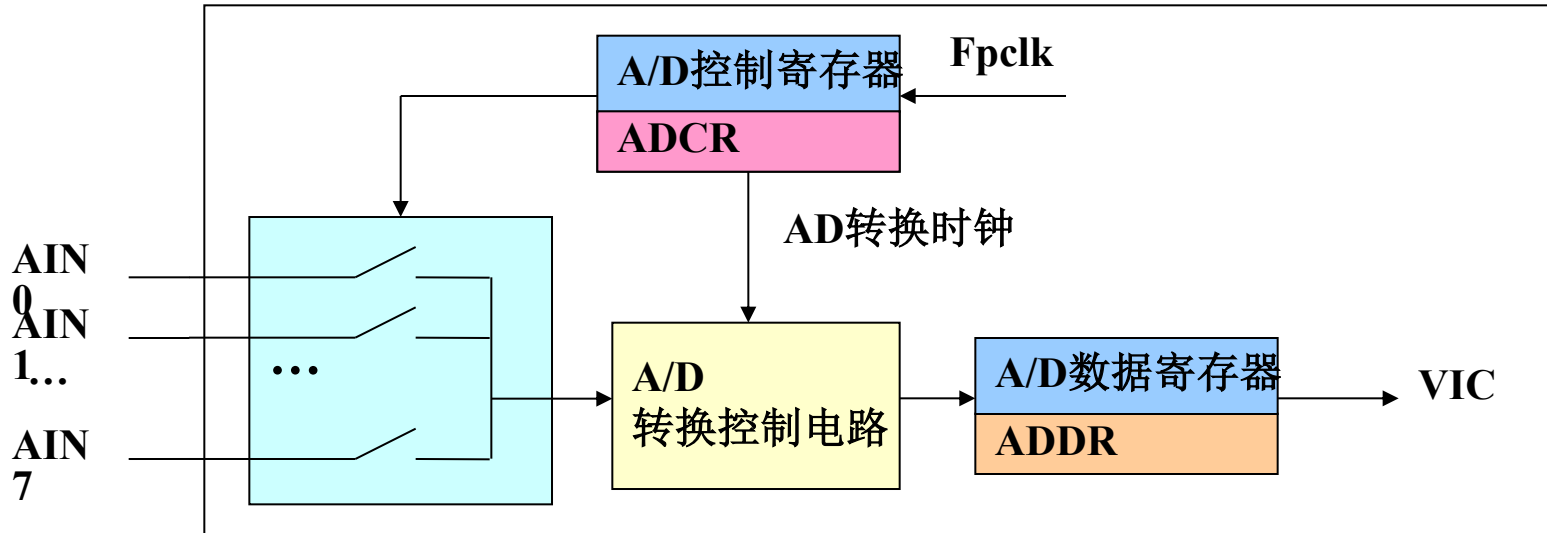
位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

SEL: 从AIN3~AIN0(LPC2114/2124)或AIN7~AIN0(LPC2212/2214)中选择采样和转换输入引脚。Bit0控制AIN0, bit1控制AIN1, 依此类推。

1:对应输入端被选中; **0:**对应输入端未选中;

注意: 软件模式下只能置位其中一位, 硬件模式下可以是任意组合。

A/D转换器寄存器描述—控制寄存器

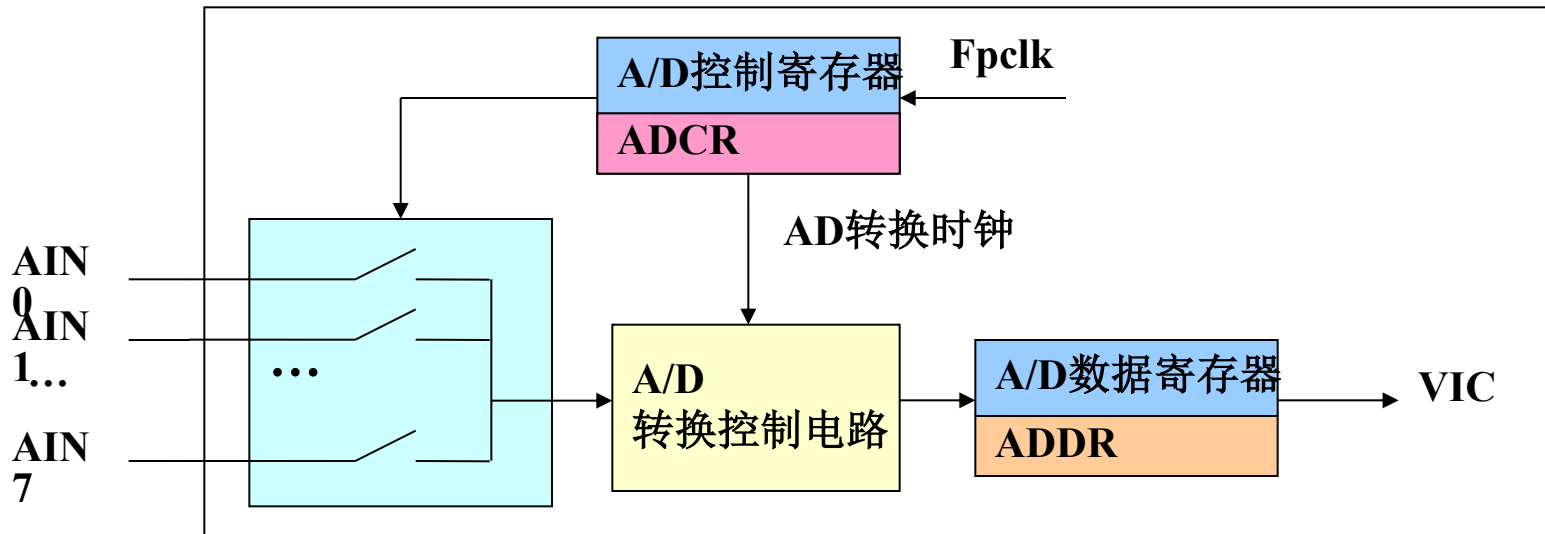


位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

位	7	6	5	4	3	2	1	0
值	0	0	0	0	0	0	0	0

AIN7 AIN6 AIN5 AIN4 AIN3 AIN2 AIN1 AIN0

A/D转换器寄存器描述—控制寄存器



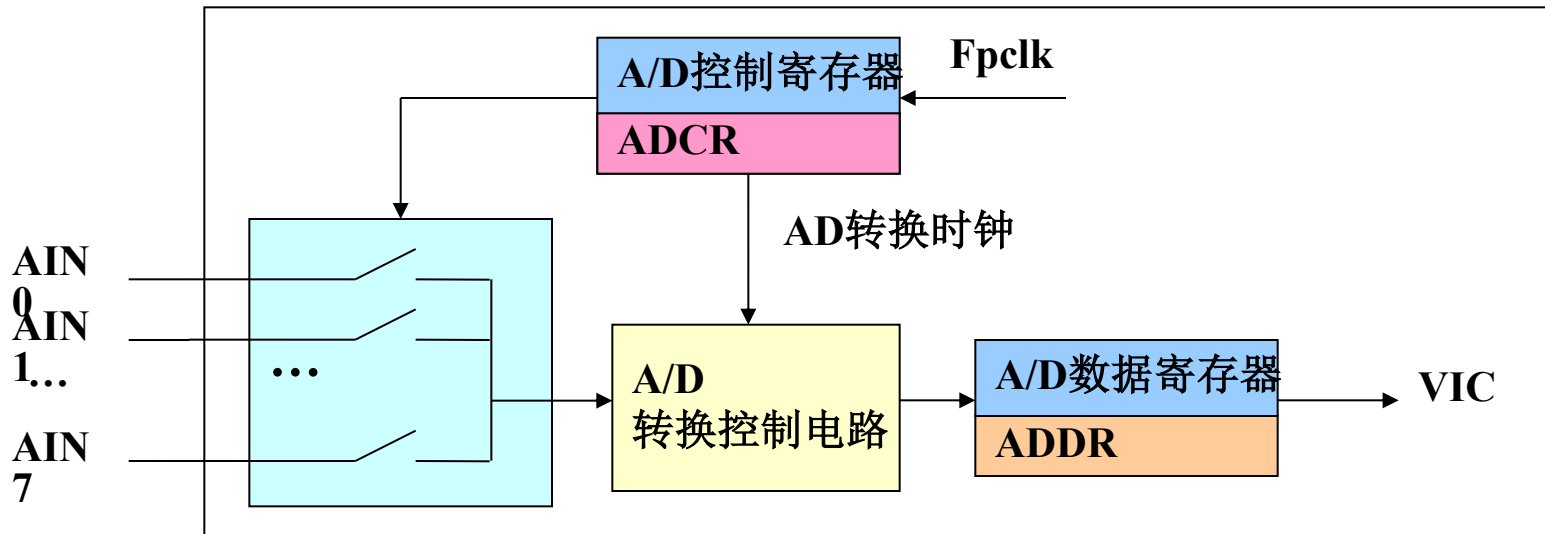
位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

CLKDIV: 将VPB时钟 (PCLK) 进行分频, 得到AD转换时钟。分频后的时钟必须小于或等于4.5MHz。通常将CLKDIV编程为允许的最小值, 以获得4.5MHz或稍低于4.5MHz的时钟。

A/D转换器时钟 = PCLK / (CLKDIV + 1)

CLKDIV = (PCLK / AD转换时钟) - 1

A/D转换器寄存器描述—控制寄存器

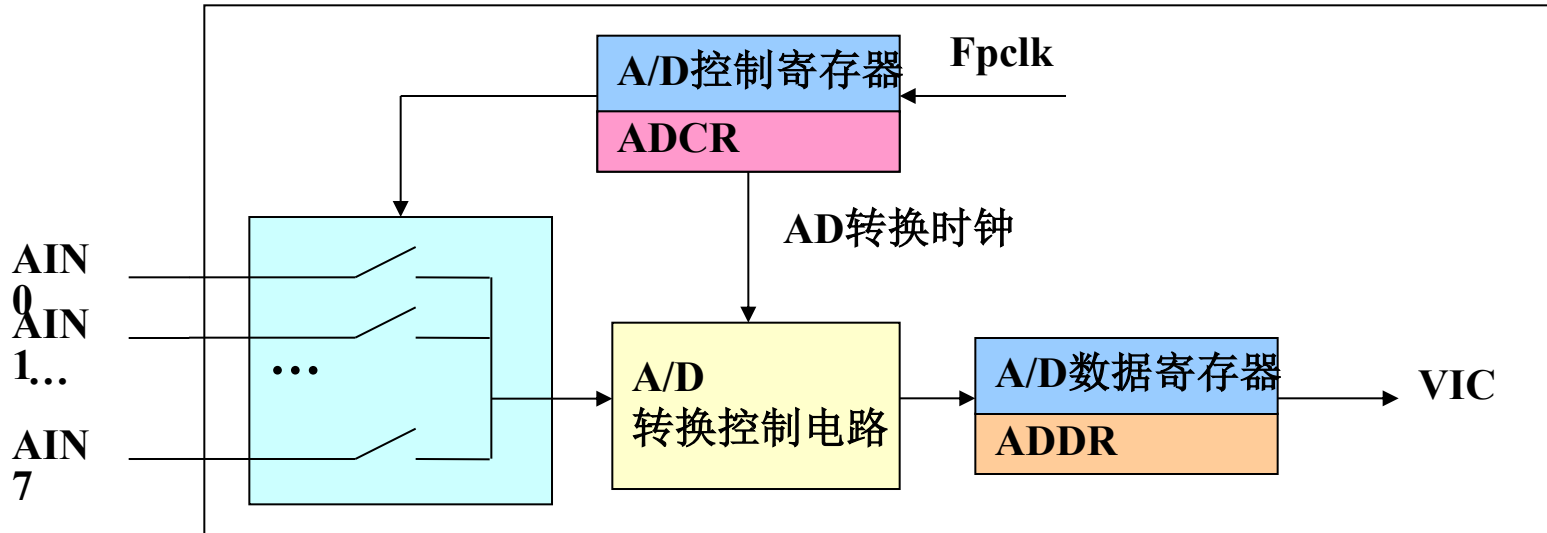


位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

BURST: BURST/软件方式控制。当该位为0时, 选择软件方式启动AD转换, 需要11个时钟才能完成。当该位为1时, 选择BURST(突发)模式启动AD转换, 所需时钟数由CLK字段控制。

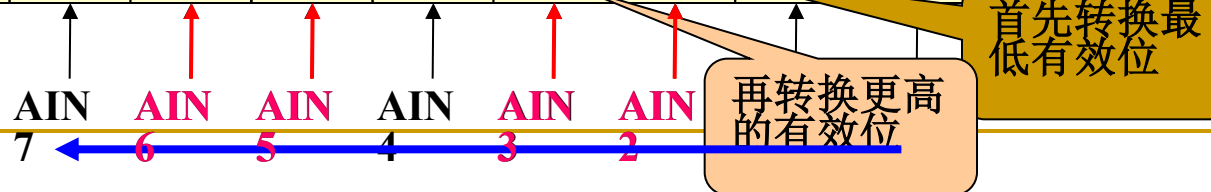
BURST模式下, 对所有在SEL字段中置1的位对应的输入端进行转换, 首先转换的是最低有效位。然后是更高的位。如此周而复始, 直至该位清零。

A/D转换器寄存器描述—控制寄存器

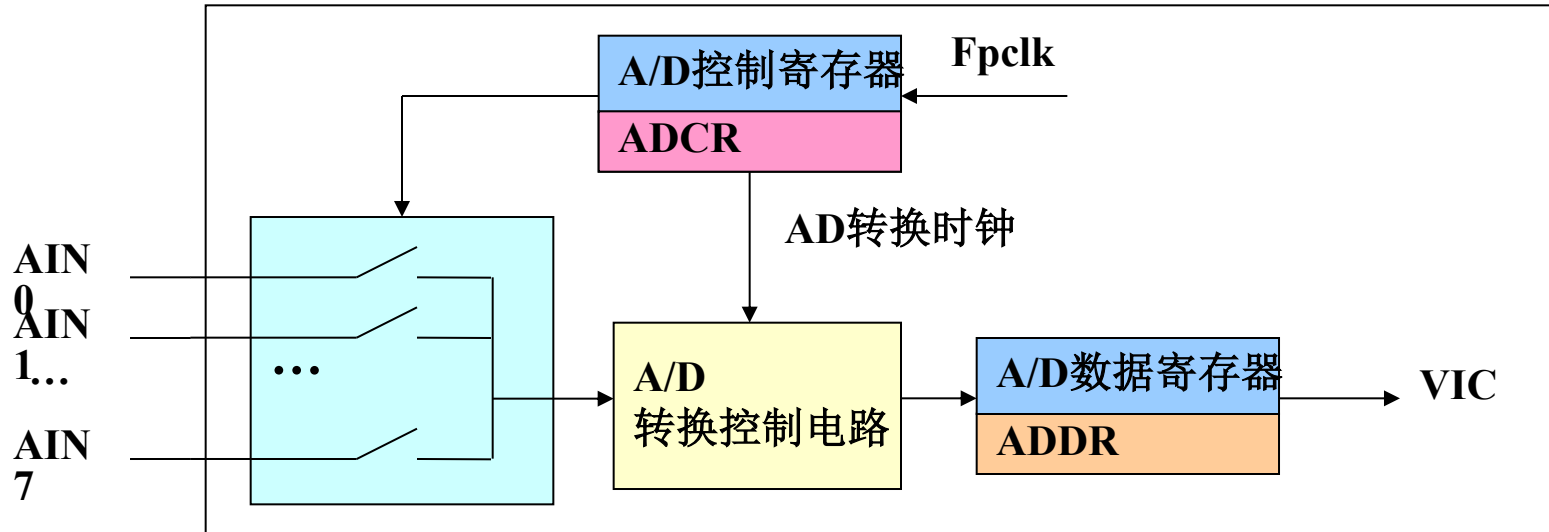


位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

位	7	6	5	4	3	2	1	0
值	0	1	1	0	1	1	0	0



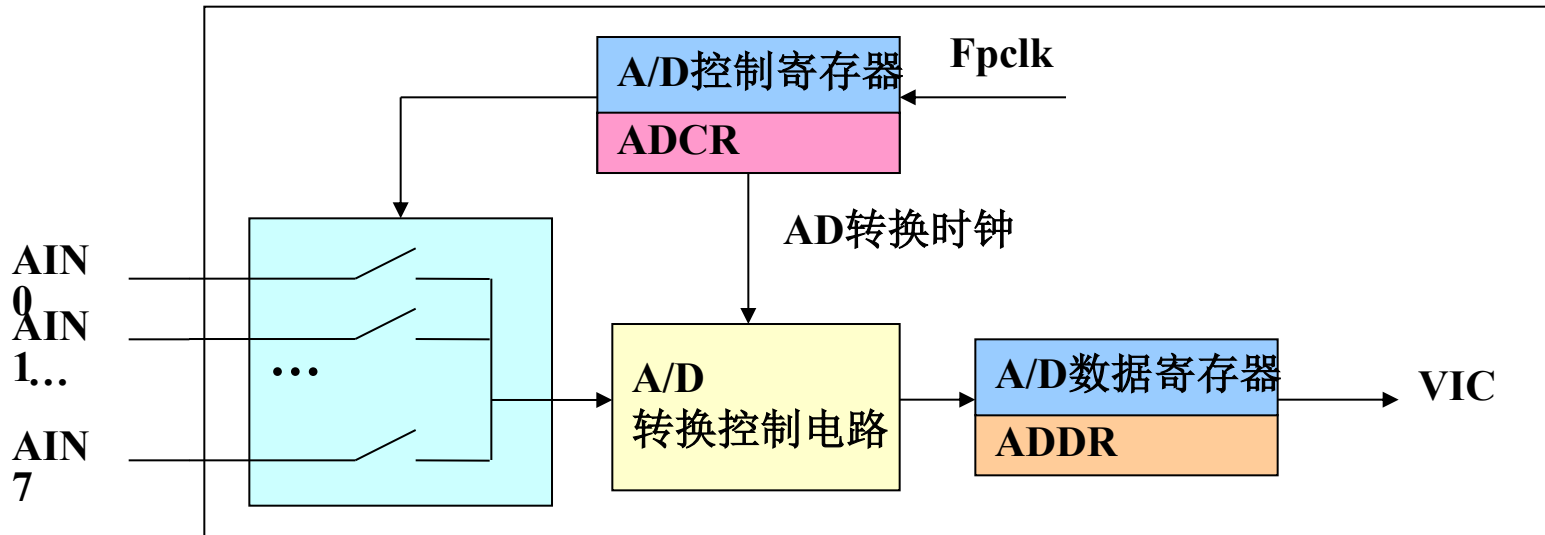
A/D转换器寄存器描述—控制寄存器



位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

CLKS: 控制BURST模式下每次转换需要使用的时钟数和所得ADDR转换结果的LS位中可确保精度的位的数目, CLKS可在11个时钟 (10位) ~4个时钟 (3位) 之间选择: 000=11个时钟/10位, 001=10个时钟/9位, ...111=4个时钟/3位。

A/D转换器寄存器描述—控制寄存器

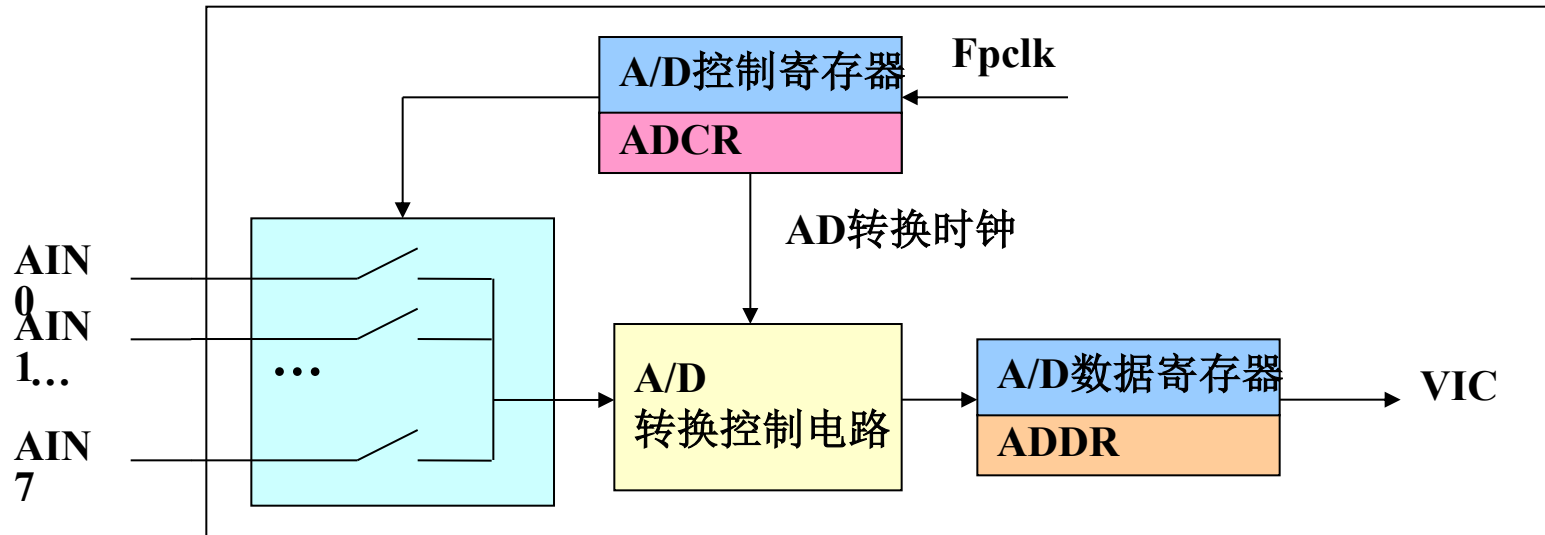


位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

PDN: 控制AD部件是否工作。

1: A/D转换器处于正常工作模式; **0**: A/D转换器处于掉电模式;

A/D转换器寄存器描述—控制寄存器



位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

TEST1:0: 器件测试控制位。

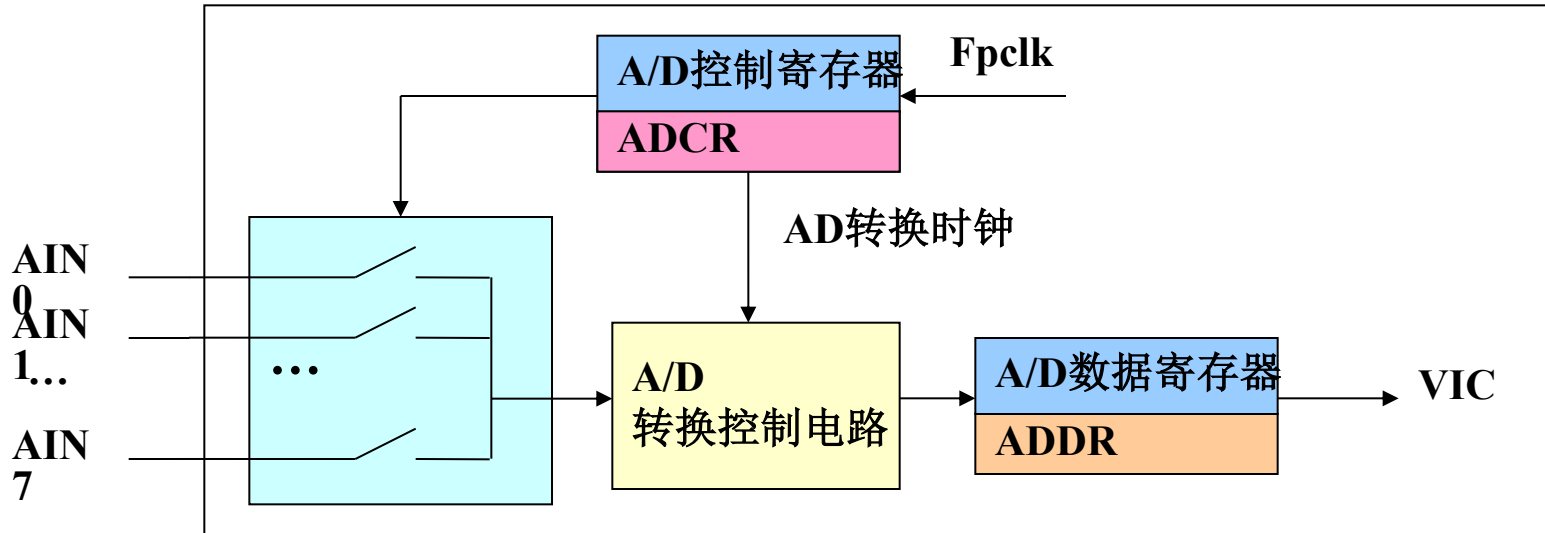
00: 正常模式;

10: DAC测试模式;

01: 数字测试模式;

11: 一次转换测试模式;

A/D转换器寄存器描述—控制寄存器



位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

START: 该字段用于控制AD转换的启动方式，该字段只有在BURST为0时有效。

000: 不启动；

001: 立即启动转换；

010: P0. 16引脚出现预置的电平时，启动AD转换。以下值也具有相同特性；

011: P0. 22引脚；

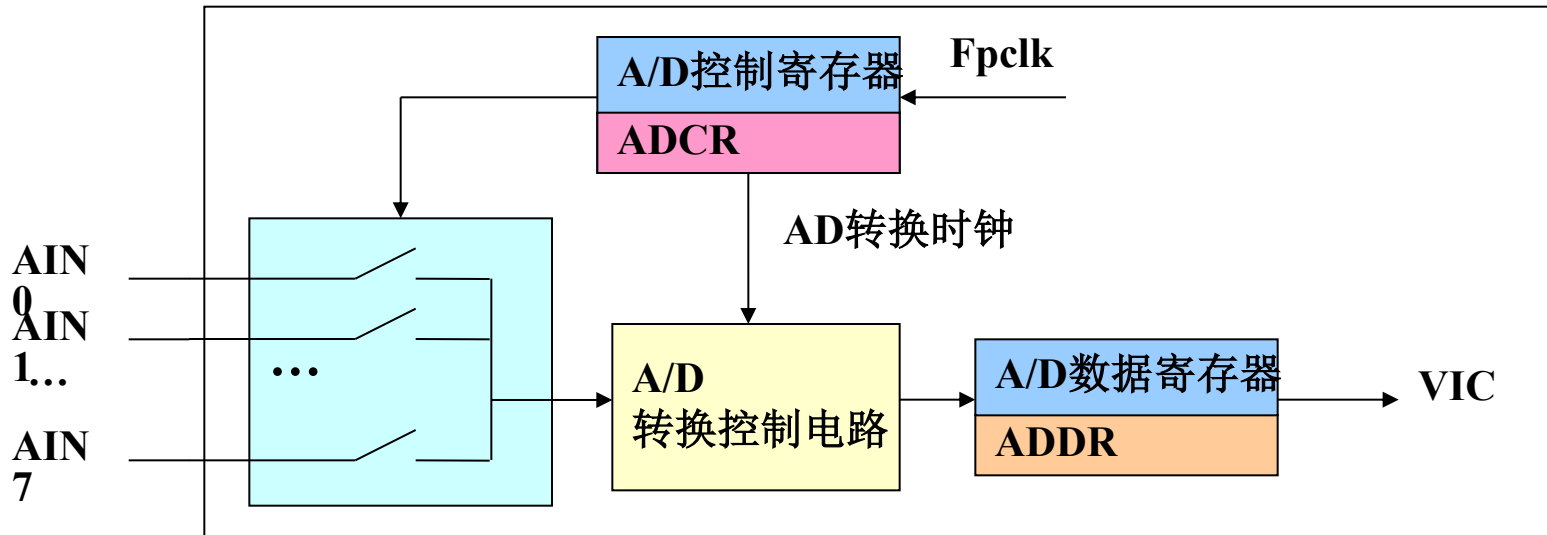
100: MAT0. 1引脚；

101: MAT0. 3引脚；

110: MAT1. 0引脚；

111: MAT1. 1引脚；

A/D转换器寄存器描述—控制寄存器



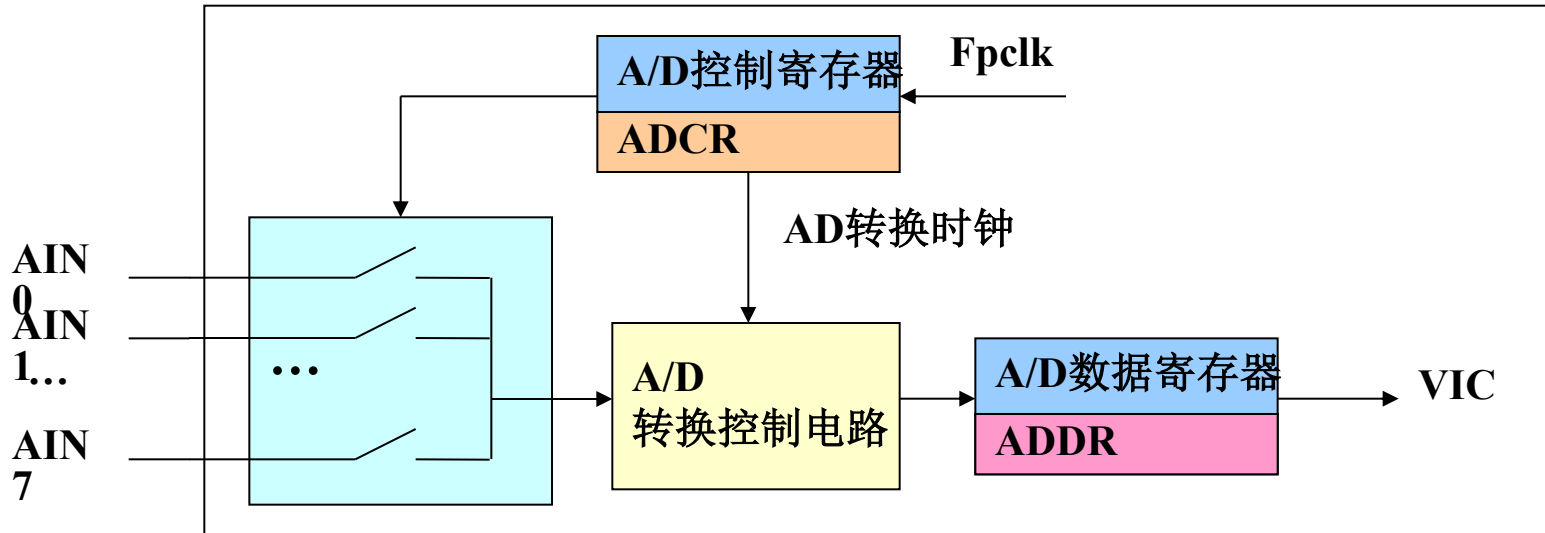
位	27	26:24	23:22	21	19:17	16	15:8	7:0
功能	EDGE	START	TEST1:0	PDN	CLKS	BURST	CLKDIV	SEL

EDGE: 当START字段的值为010~111时, 该位的设置有效。

0: 在所选CAP/MAT信号的下降沿启动转换

1: 在所选CAP/MAT信号的上升沿启动转换

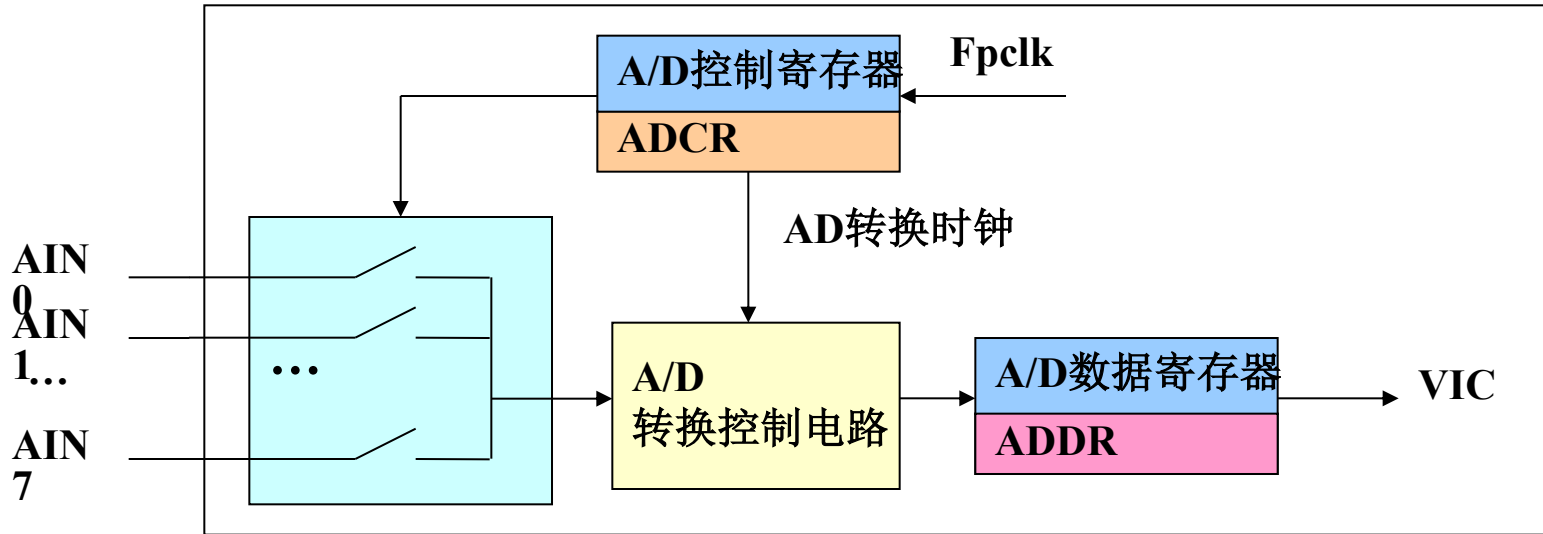
A/D转换器寄存器描述—数据寄存器



位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/VddA	0

0: 这些位读出时为0。用于未来扩展功能更强大的AD转换器。

A/D转换器寄存器描述—数据寄存器



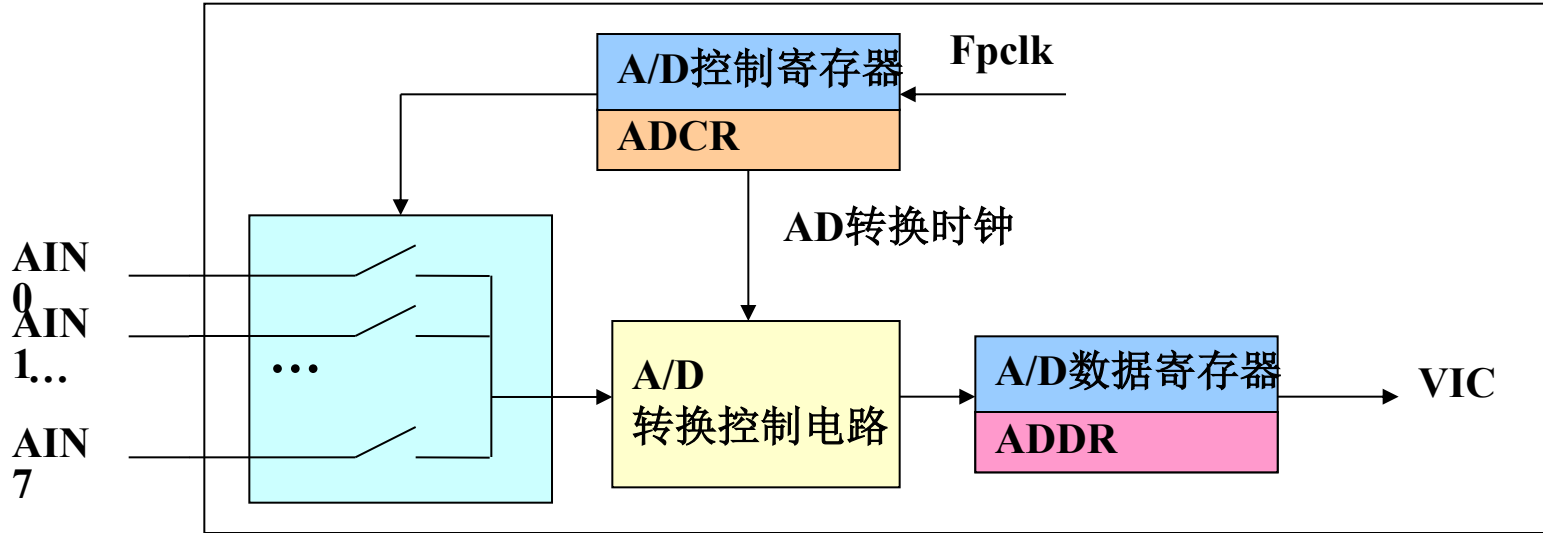
位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/V _{ddA}	0

V/V_{ddA}: 当DONE位为1时, 该字段包含对SEL字段选中的Ain脚的转换结果, 为一个二进制数。

转换结果为0时, 表示Ain引脚电平小于、等于或接近于V_{SSA}。为0x3FF时, 表示Ain引脚电平等于、大于或接近于V_{ddA}。输入电压计算公式为:

$$V_{in} = \text{结果} \times (V_{SSA} / 0x3FF)$$

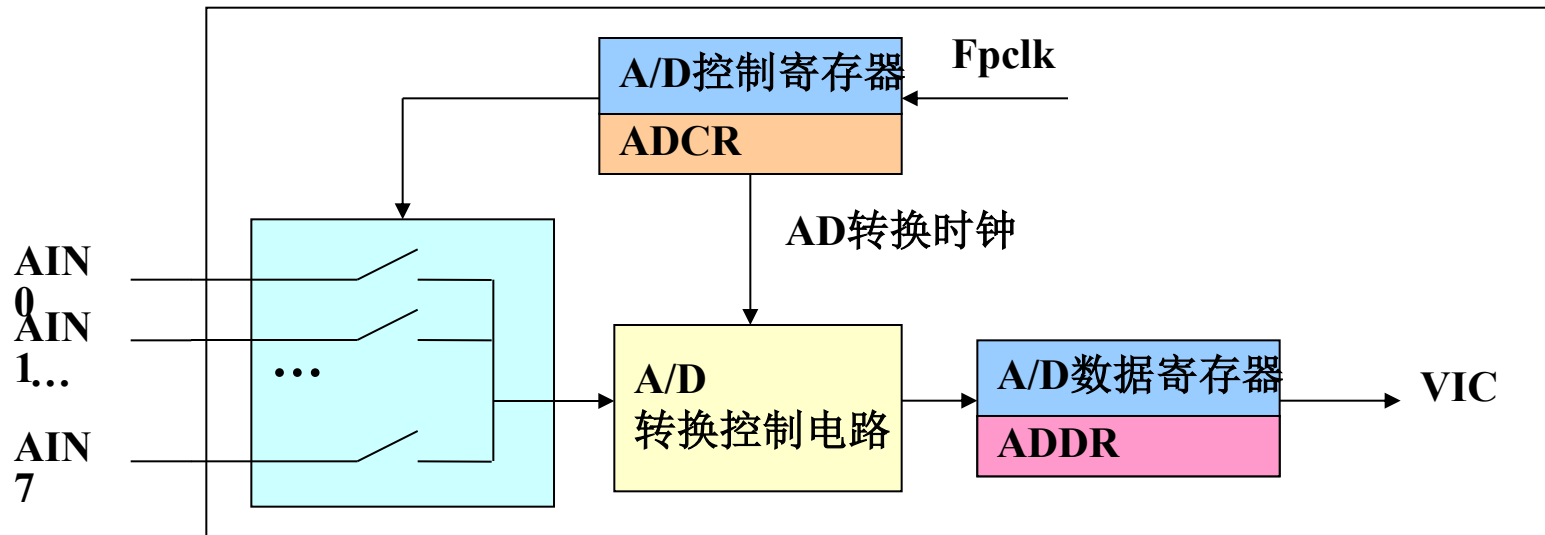
A/D转换器寄存器描述—数据寄存器



位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/V _{ddA}	0

0: 这些位读出时为0。它们允许连续A/D值的累加，而不需要屏蔽处理，使得至少有256个值不会溢出到CHN字段。

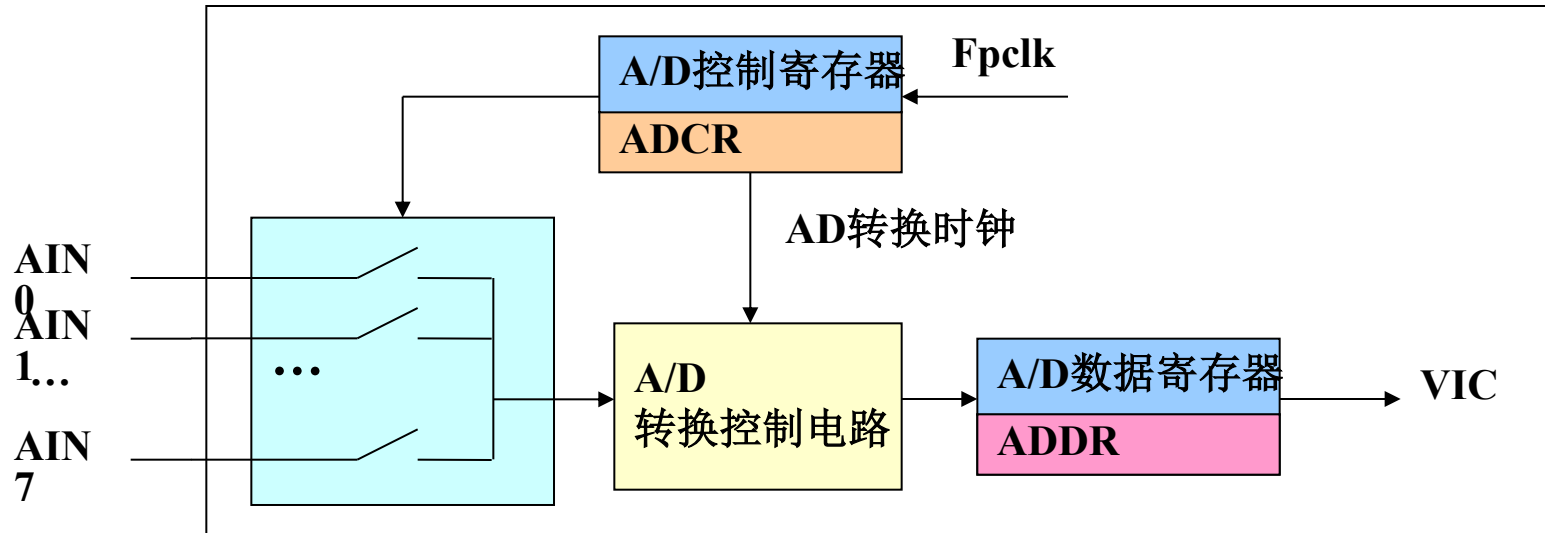
A/D转换器寄存器描述—数据寄存器



位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/V _{ddA}	0

CHN: 该字段包含的是LS位的转换通道。

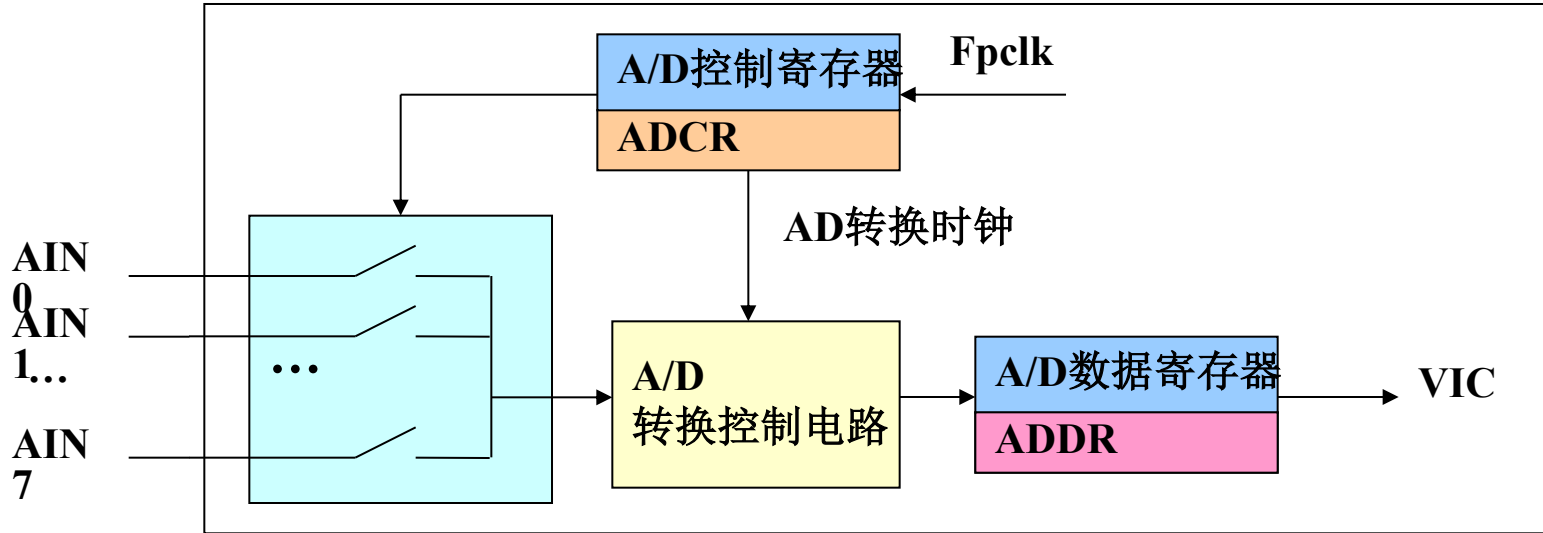
A/D转换器寄存器描述—数据寄存器



位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/V _{ddA}	0

0: 这些位读出为0。用于未来CHN字段的扩展，使之兼容更多通道的转换值。

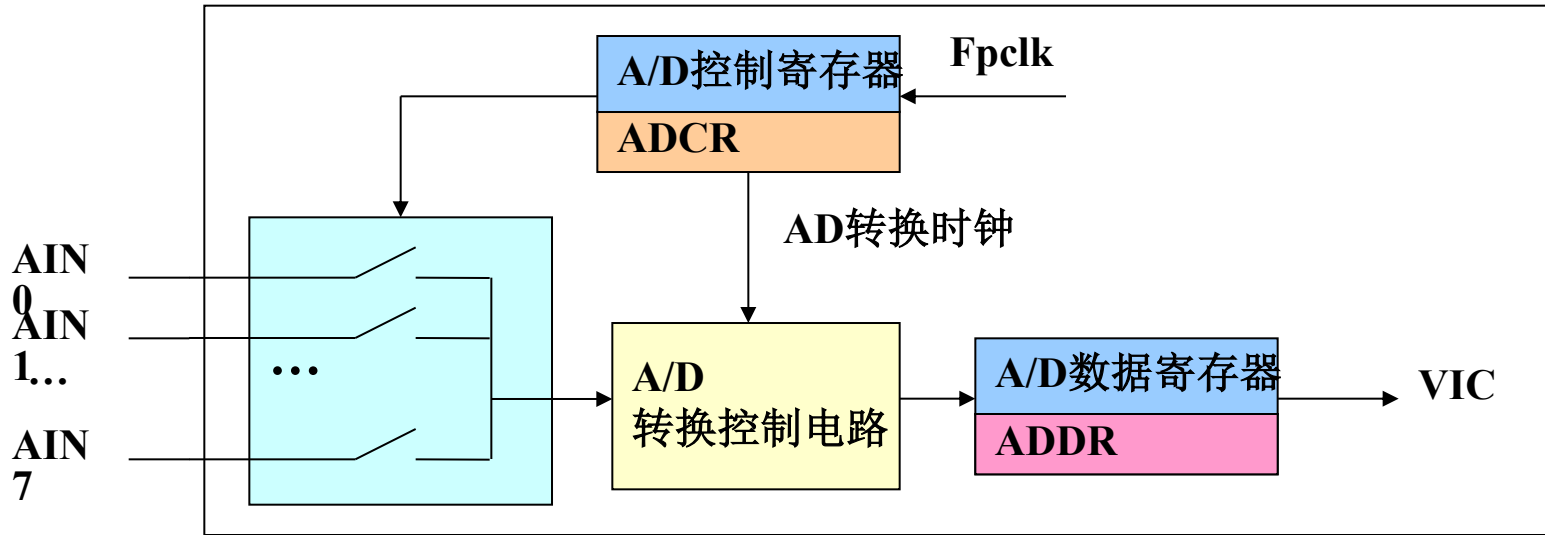
A/D转换器寄存器描述—数据寄存器



位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/VddA	0

OVERUN: 在BURST模式下, 如果在转换产生最低位之前, 以前转换的结果丢失或被覆盖, 该位将置位。读ADDR寄存器时, 该位清零。

A/D转换器寄存器描述—数据寄存器



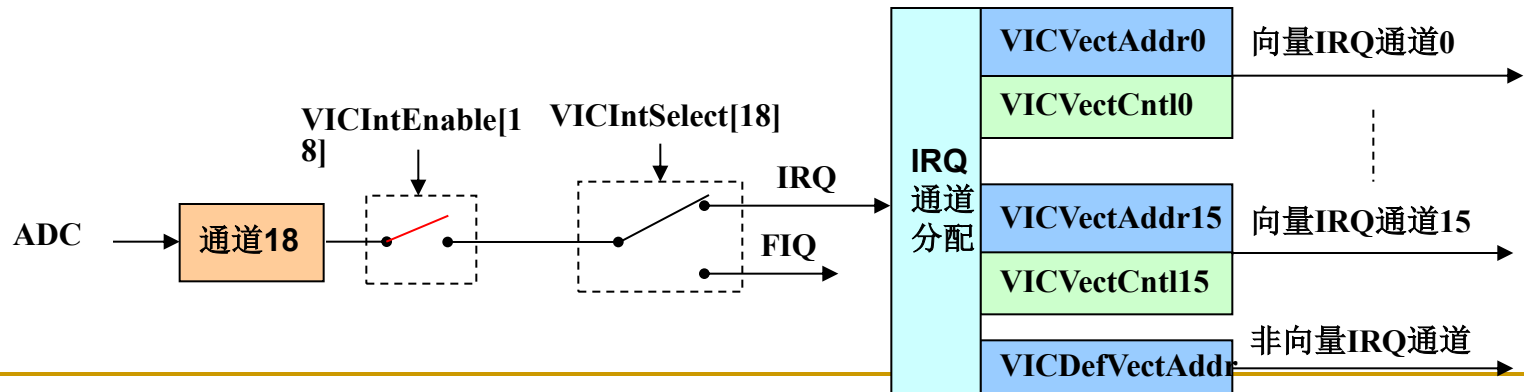
位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/VddA	0

DONE: AD转换完成标志位。当AD转换结束时该位置位。在读取ADDR或ADCR被写入时, 该位清零。如果在转换过程中, 设置了ADCR, 那么该位将置位, 并启动一次新的转换。

ADC中断

■ ADC中断与VIC的关系

ADC处于VIC的通道18，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

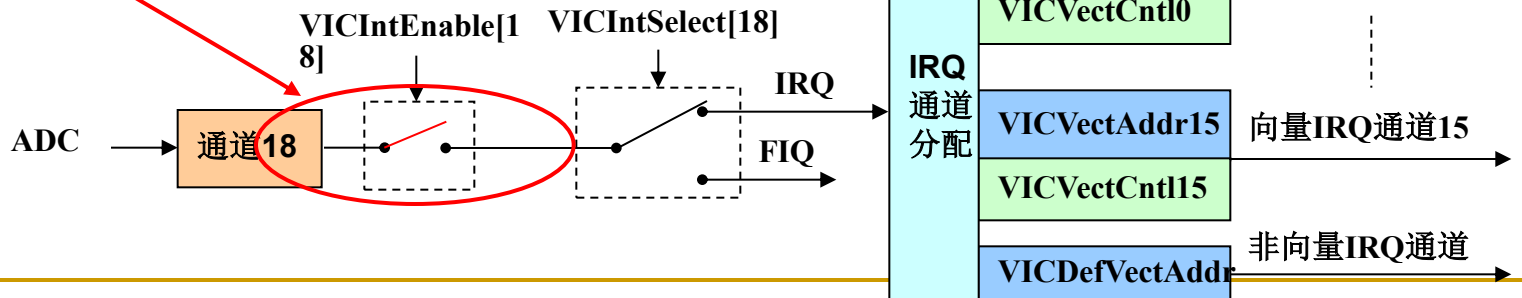


ADC中断

■ ADC中断与VIC的关系

ADC处于VIC的通道18，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

➤ 当VICIntEnable[18] = 0时，通道18中断禁止；

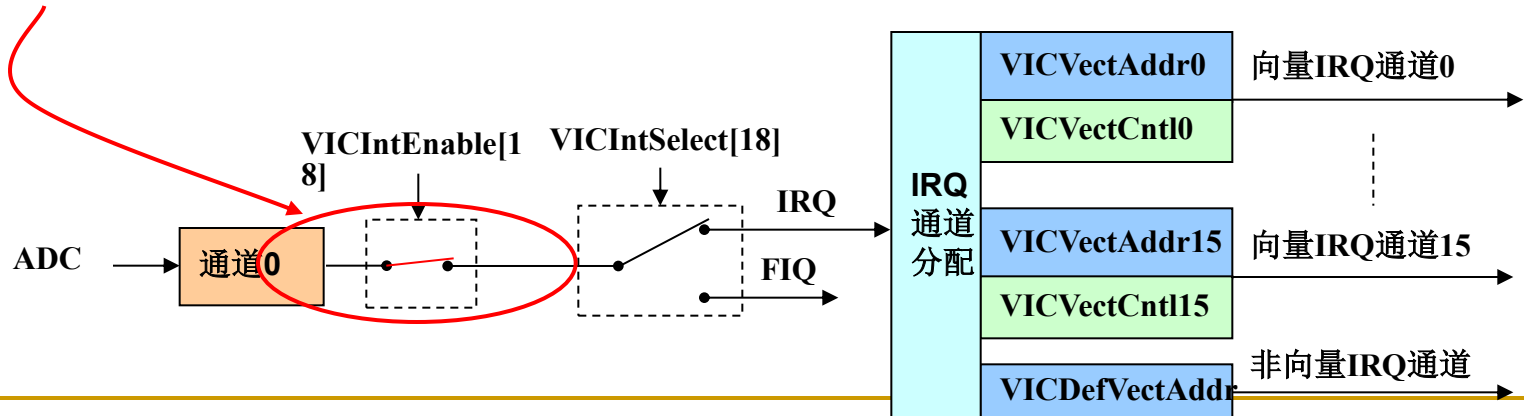


ADC中断

- ADC中断与VIC的关系

ADC处于VIC的通道18，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

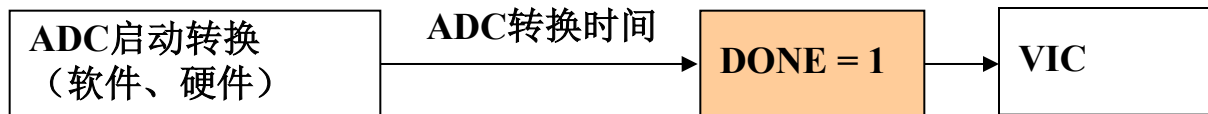
- 当VICIntEnable[18] = 0时，通道18中断禁止；
- 当VICIntEnable[18] = 1时，通道18中断使能；



ADC中断

- ADC中断说明

A/D转换器没有专门的中断使能位，这一点是和其它功能部件所不同的。



启动A/D转换，A/D转换后，DONE置位，触发中断。

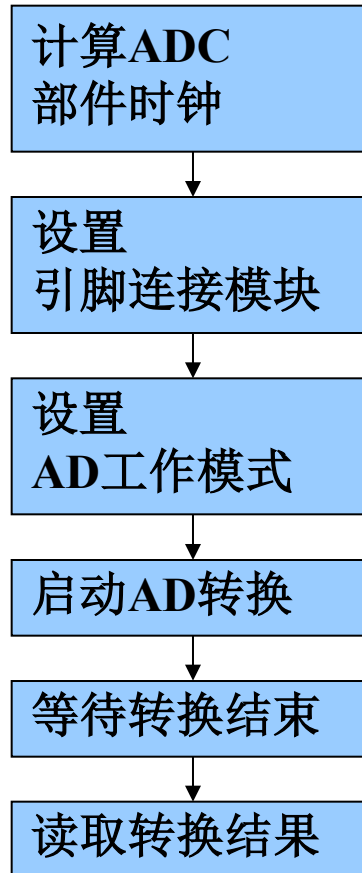
4.15 A/D转换器

■ 使用A/D转换器的注意要点

- AD转换器的时钟不能大于4.5MHz;
- 使用MAT引脚触发AD转换启动时, 相应的MAT信号不必输出到引脚。使用MAT引脚触发的方法, 可以实现AD转换定时启动;
- BURST模式下, 每次转换结束后立即开始下一路的转换, 所以BURST模式具有最高的效率;
- 软件模式下, SEL字段中只能有一位置位, 如果多位置位, 将使用最低有效位。

• A/D转换器操作示例

操作流程



A/D转换器操作示例

使用软件方式对Ain0转换:

```
#define ADCLK 4500000 // 定义AD部件时钟频率, 单位: Hz
#define ADBIT 10 // 定义BURST模式下的转换精度
#define ADBIT2 (10 - ADBIT)
```

...

```
PINSEL1 = (PINSEL1 & 0xFC3FFFFF) | 0x00400000;
ADCR = (0x01 << 27) | // EDGE
        (0x05 << 24) | // START
        (0x00 << 22) | // TEST1:0
        (0x01 << 21) | // PDN
        (ADBIT2 << 17) | // CLKS
        (0x00 << 16) | // BUREST
        ((Fpclk/ADCLK + 1) << 8) | // CLKDIV
        (0x01 << 0); // SEL
```

```
ADCR |= (1 << 24);
```

```
While((ADCR & 0x80000000) != 0),
```

```
ADCData = (ADCR >> 6) & 0x3FF;
```

...

设置引脚连接模块

硬件触发边沿设置

AD启动设置

测试模式设置

AD部件上电设置

BURST模式精度

BURST模式禁止

ADC部件时钟

转换通道选择

启动AD转换

等待转换结束

读取转换结果



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.16 看门狗

■ 看门狗简介

在嵌入式应用中，CPU必须可靠工作，即使因为某种原因进入了一个错误状态，系统也应该可以自动恢复。看门狗的用途就是使微控制器在进入错误状态后的一定时间内复位。

其原理是在系统正常工作时，用户程序每隔一段时间执行喂狗动作（一些寄存器的特定操作），如果系统出错，喂狗间隔超过看门狗溢出时间，那么看门狗将会产生复位信号，使微控制器复位。

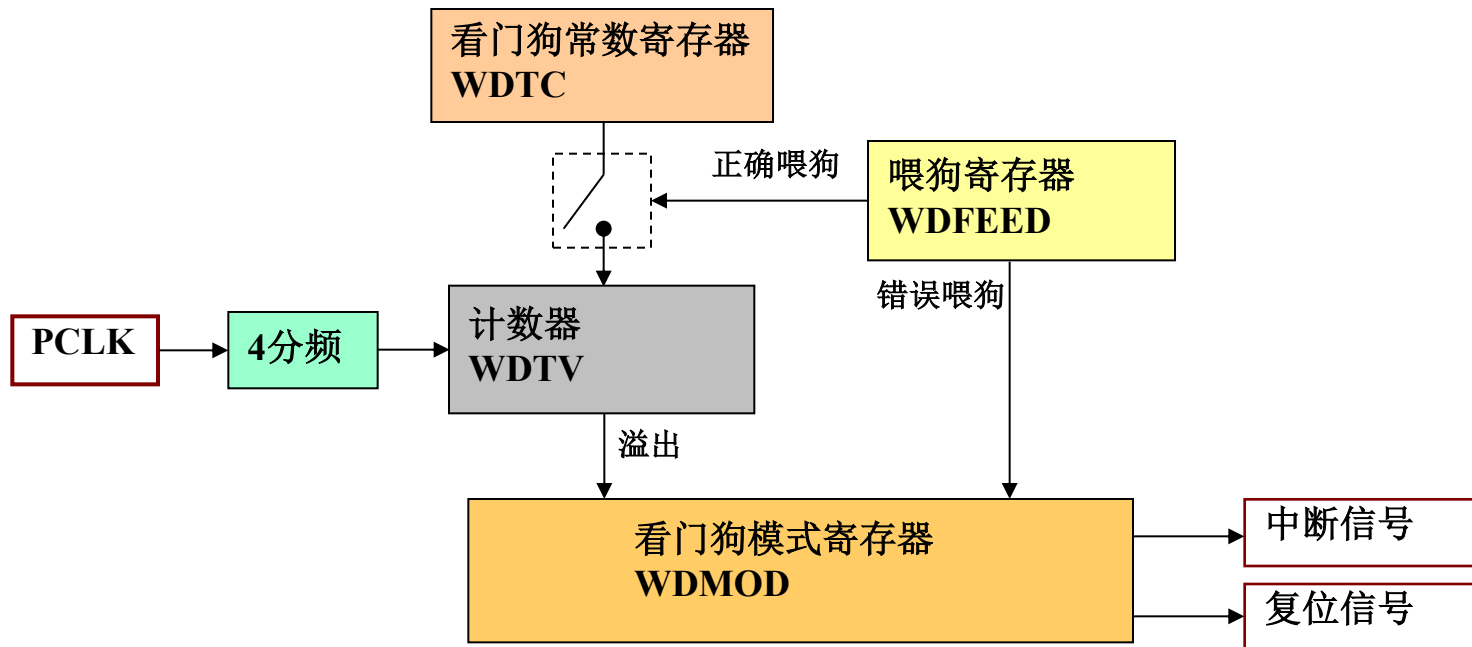
4.16 看门狗

■ 特性

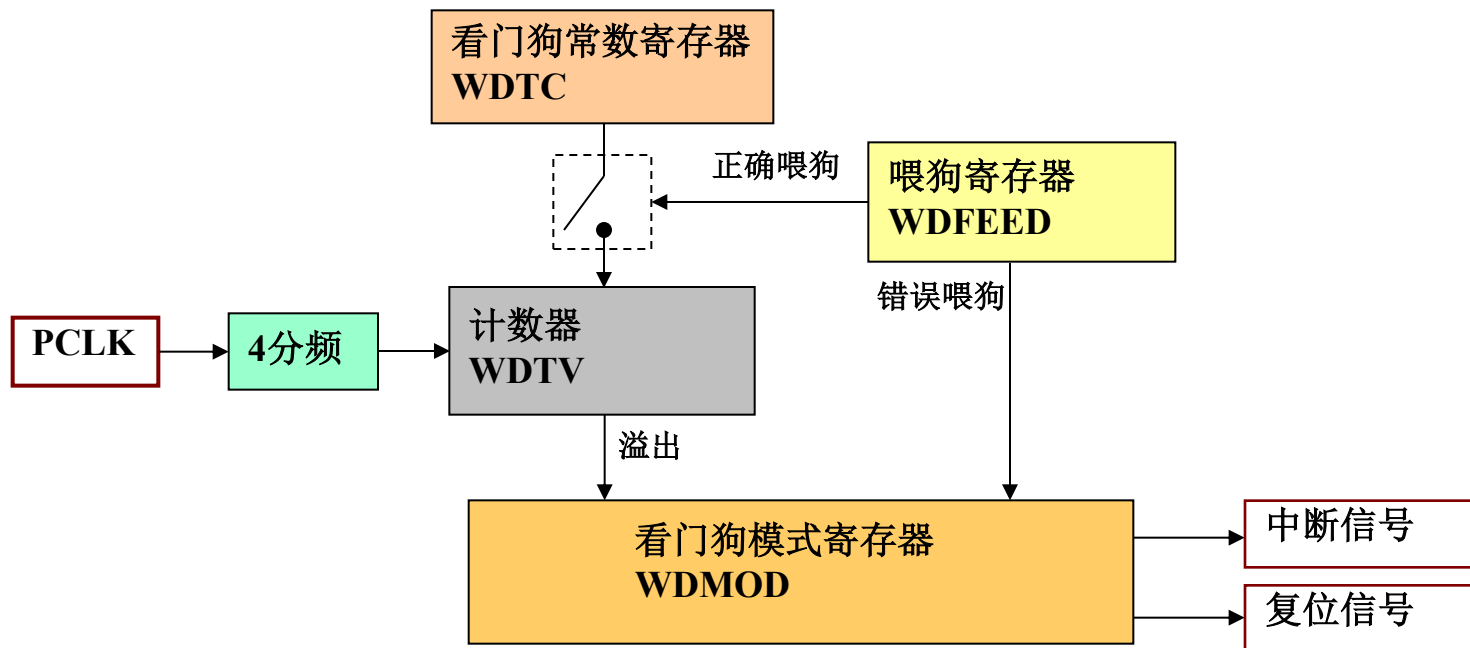
LPC2000系列微控制器都集成有看门狗部件，其特性为：

- 带内部预分频器的可编程32位定时器；
- 如果没有周期性重装（喂狗）动作，则产生片内复位；
- 具有调试模式；
- 看门狗软件使能后，必须由复位来禁止；
- 错误的喂狗动作，将立即引起复位。

■ 看门狗一内部结构图

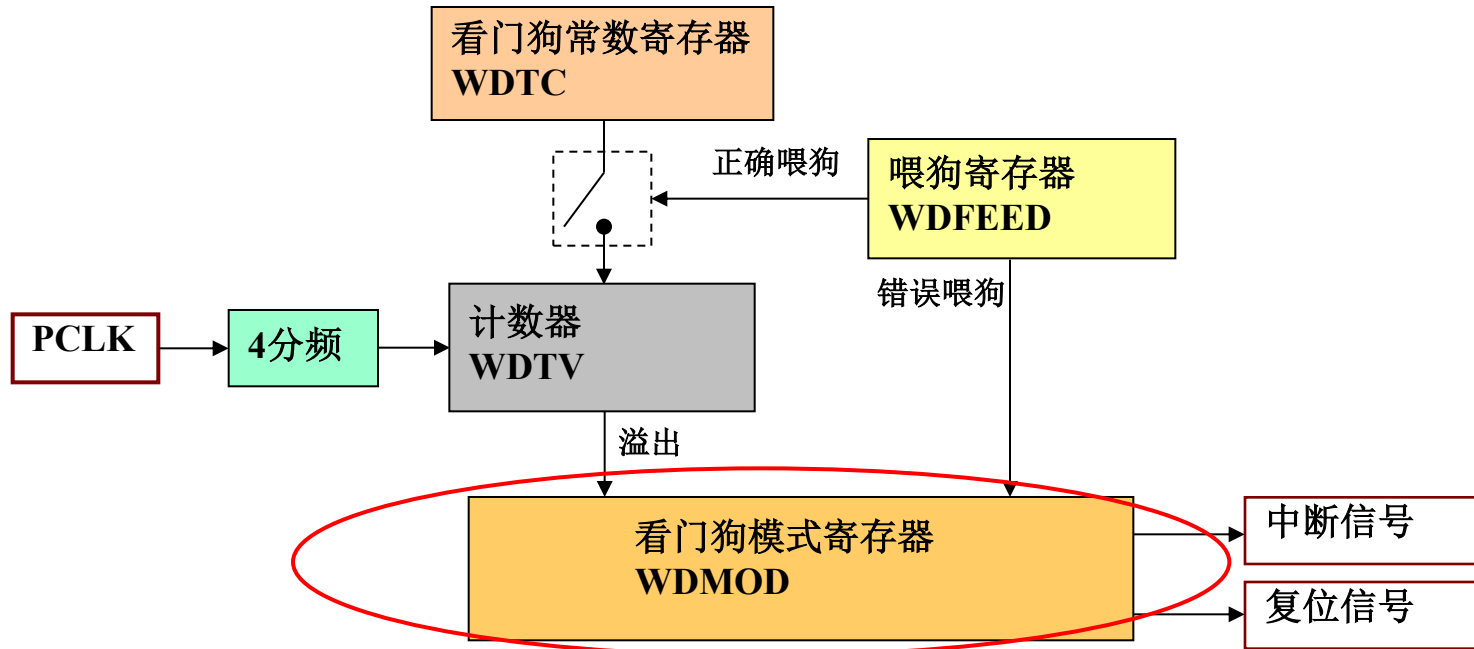


■ 看门狗—寄存器描述



名称	描述	访问
WDMOD	看门狗模式寄存器。该寄存器包含看门狗定时器的基本模式和状态。	读/设置
WDTC	看门狗定时器常数寄存器。该寄存器决定超时值。	读写
WDFEED	看门狗喂狗寄存器。通过它执行特定喂狗时序。	只写
WDTV	看门狗定时器值寄存器。反映倒计数器当前值。	只读

寄存器描述—看门狗模式寄存器



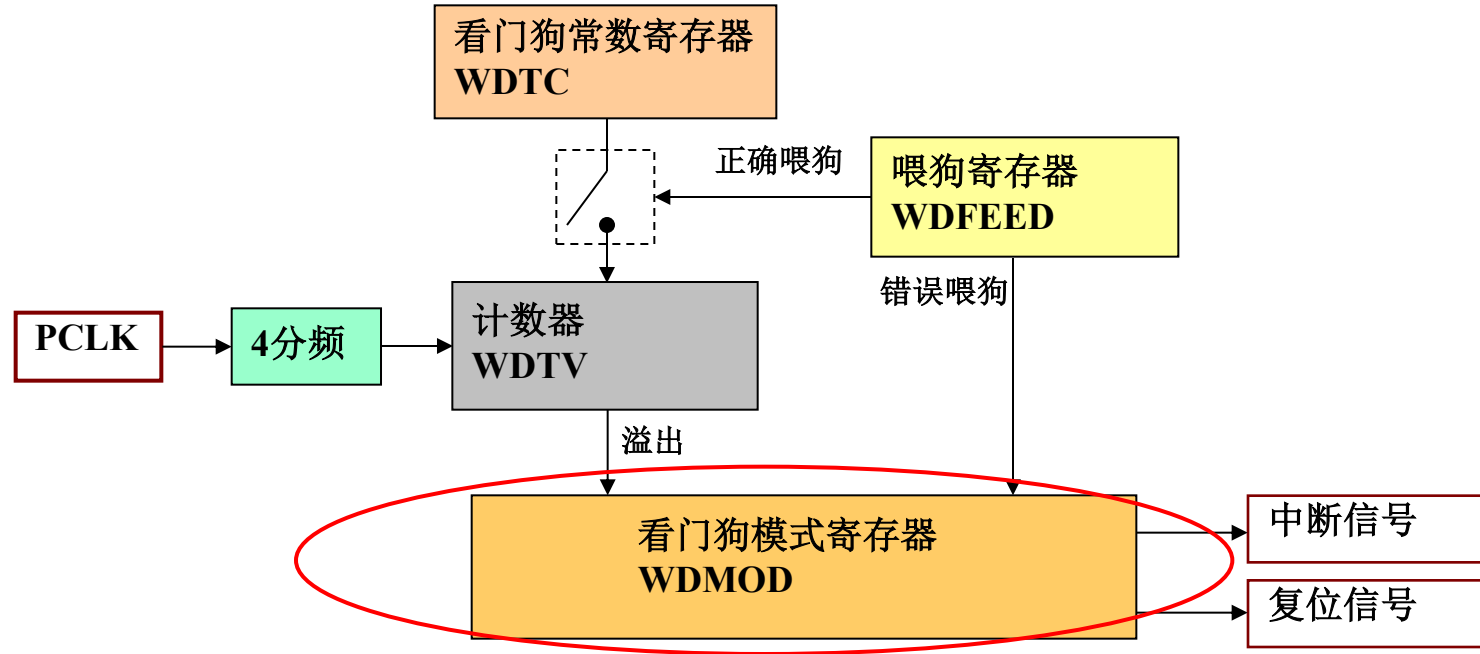
位	7 : 4	3	2	1	0
功能	保留	WDINT	WDTOF	WDRESET	WDEN

WDEN：看门狗中断使能位，该位只能置位。

该位置位后，将使能看门狗。一旦该位置位，软件无法将其清零。只有外部复位或看门狗定时器溢出能将其清零。

注意：将该位置位后只是使能WDT，但没有启动WDT，当第一次喂狗操作时才启动WDT。

寄存器描述—看门狗模式寄存器

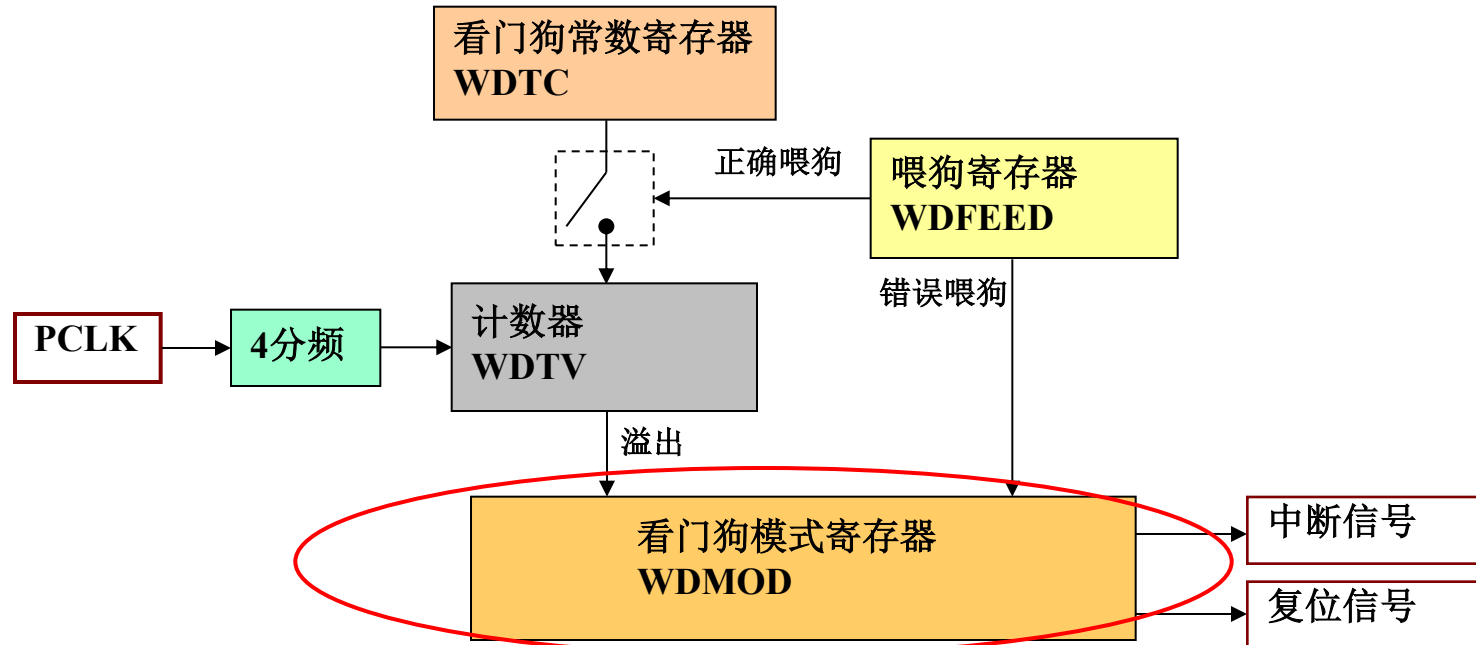


位	7:4	3	2	1	0
功能	保留	WDINT	WDTOF	WDRESET	WDEN

WDRESET: 看门狗复位使能位，该位只能软件清除。

该位置位后，看门狗溢出将引起复位。一旦该位置位，软件无法将其清零。只有外部复位或看门狗定时器溢出能将其清零。

寄存器描述—看门狗模式寄存器

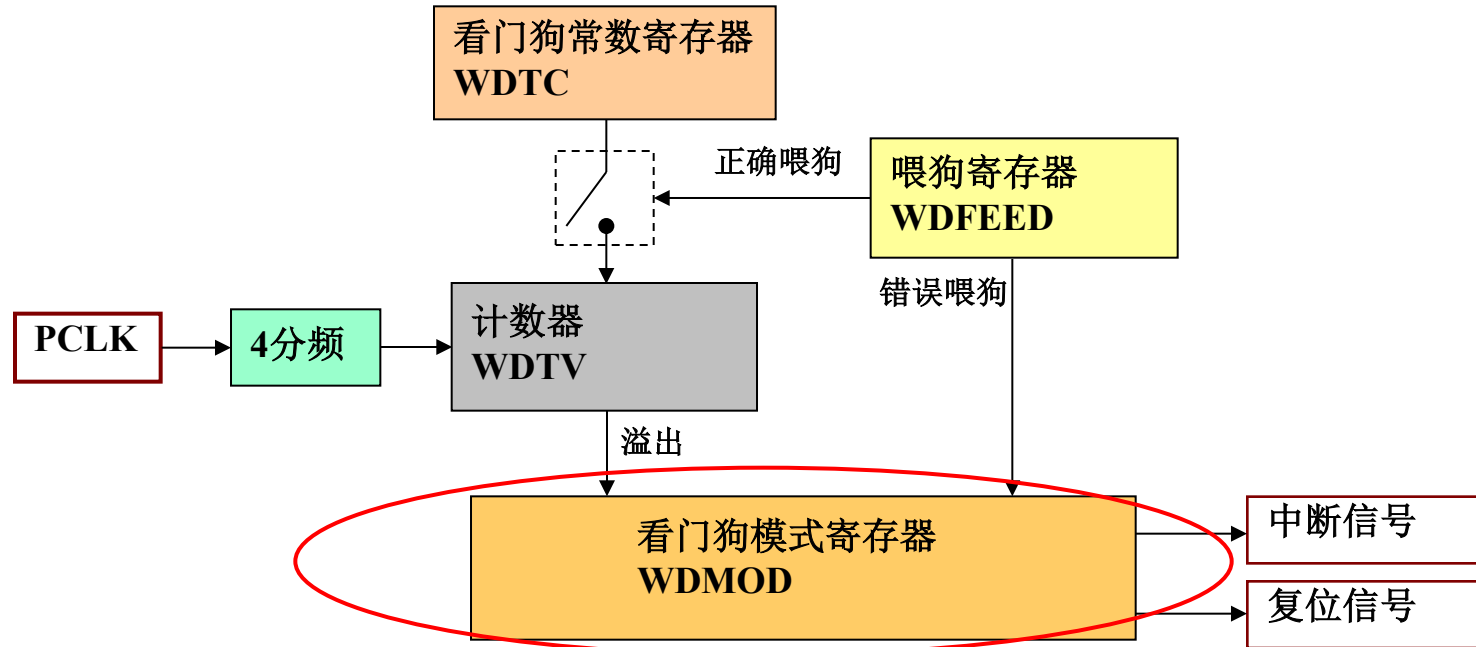


位	7 : 4	3	2	1	0
功能	保留	WDINT	WDTOF	WDRESET	WDEN

WDTOF: 看门狗超时标志。

当看门狗发生超时，看门狗超时标志置位。该标志由软件清零。

寄存器描述—看门狗模式寄存器

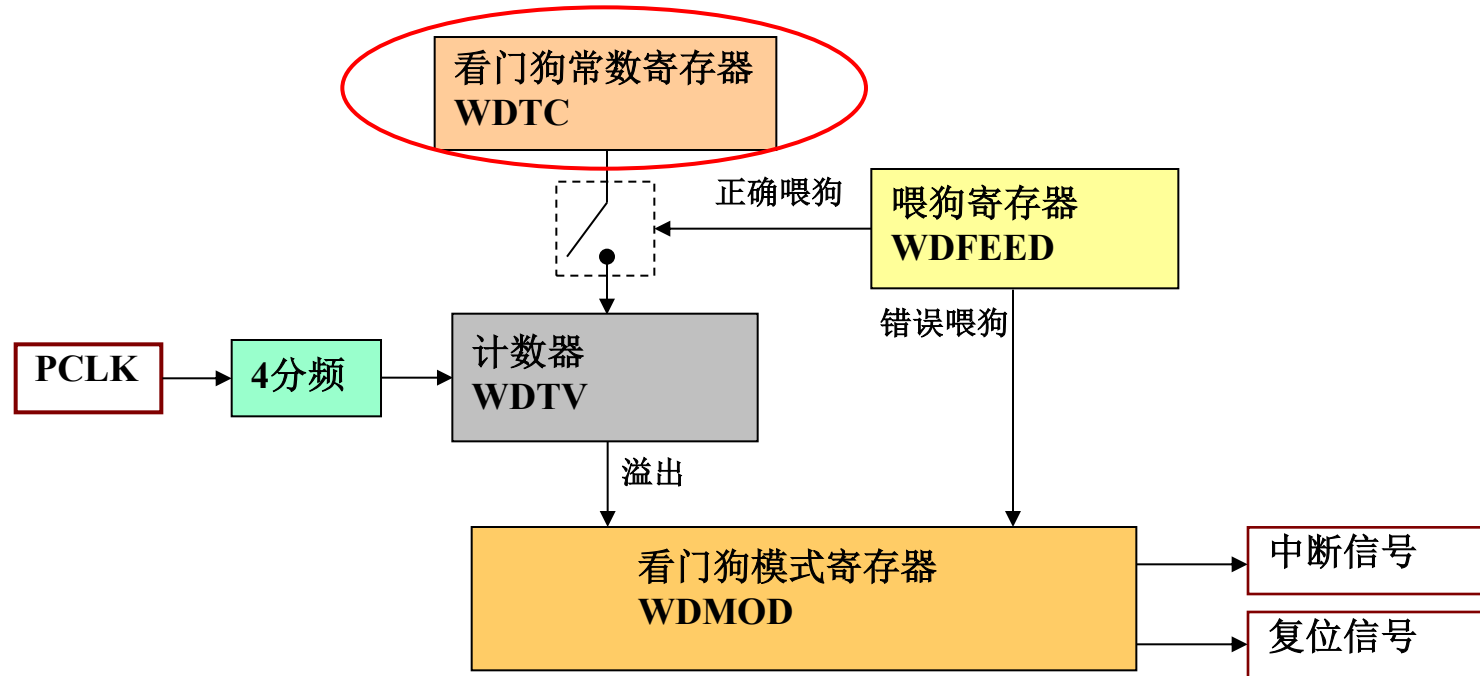


位	7 : 4	3	2	1	0
功能	保留	WDINT	WDTOF	WDRESET	WDEN

WDINT: 看门狗中断标志。

当看门狗发生超时，该位置位。该标志不能由软件清零，只能通过外部复位或者看门狗复位清零。

寄存器描述—看门狗常数寄存器

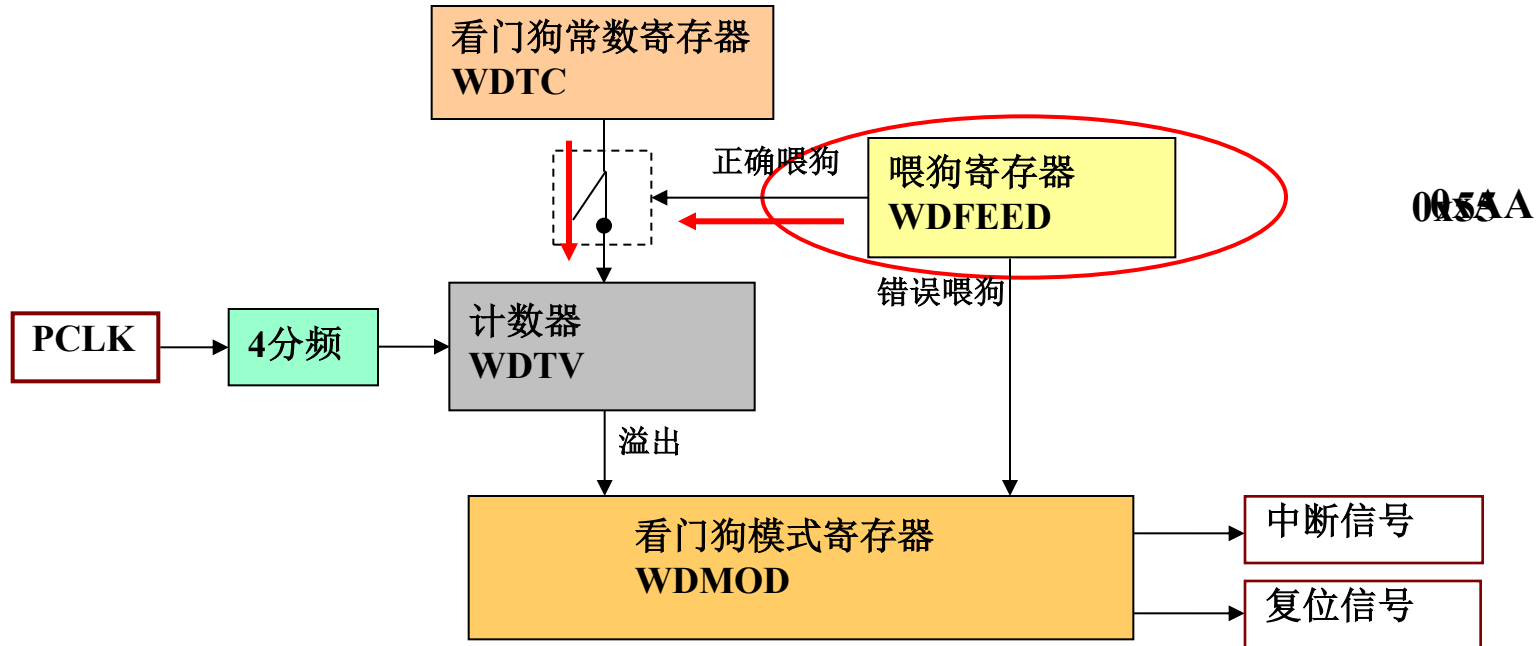


该寄存器决定看门狗超时值，当喂狗时序产生时，该寄存器的内容重新装入看门狗定时器。该寄存器的复位值为0xFF，即使写入更小的值，也会装入0xFF。

溢出最小时间： $t_{pclk} \times 0xFF \times 4$

溢出最大时间： $t_{pclk} \times 0xFFFFFFFF \times 4$

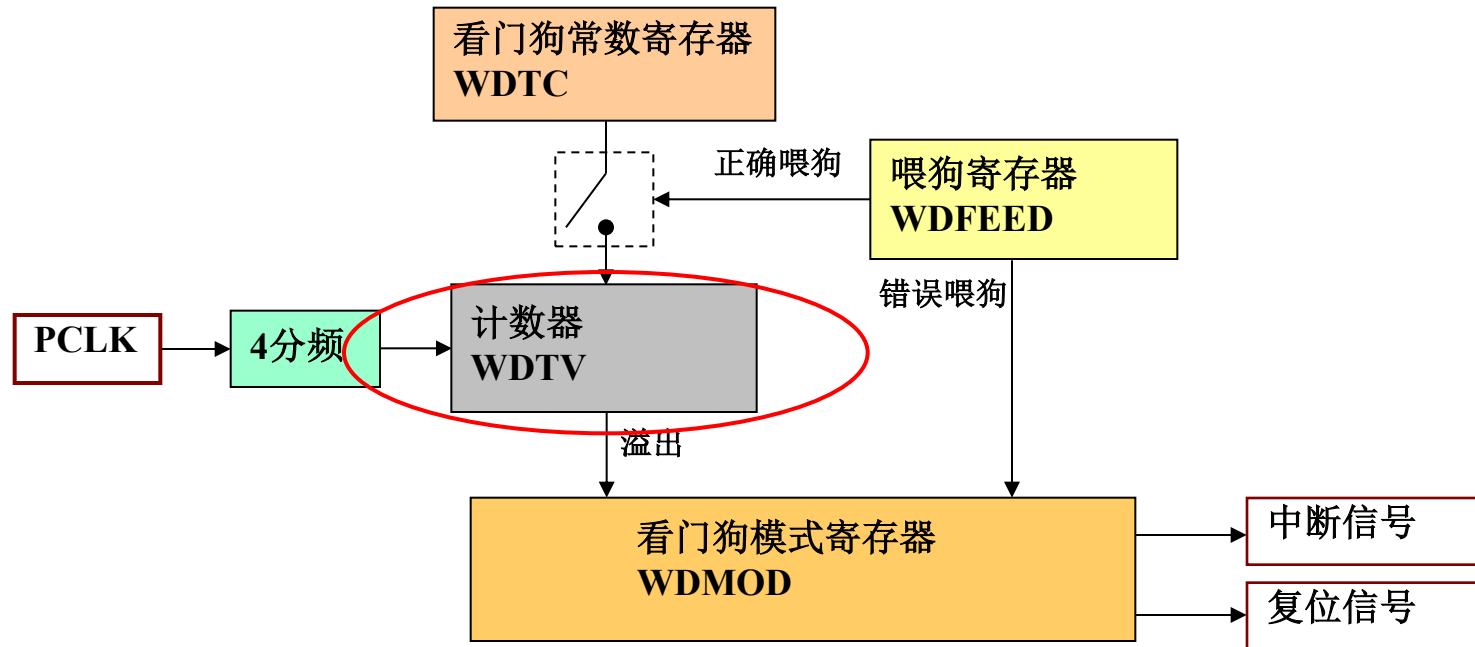
寄存器描述—看门狗喂狗寄存器



向该寄存器写入0xAA，然后写入0x55会使WDTC的值重新装入看门狗定时器。如果看门狗通过WDMOD寄存器使能，那么第一次喂狗操作还将启动看门狗运行。在看门狗能够产生中断/复位之前，即看门狗溢出之前，必须完成一次有效的喂狗时序。

注意：如果喂狗时序不正确，将在喂狗之后的第二个PCLK周期产生看门狗复位。

寄存器描述—看门狗定时器值寄存器

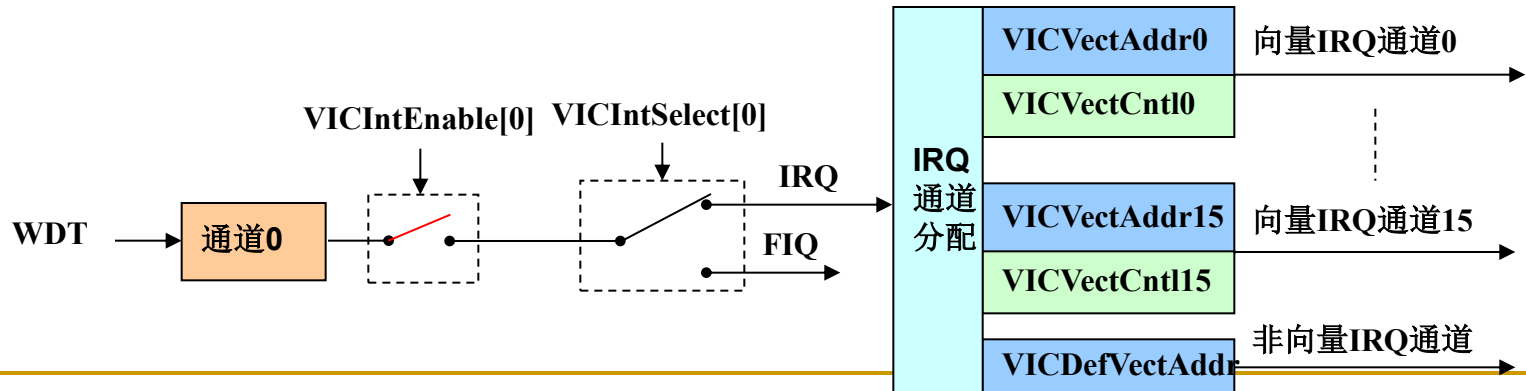


该寄存器用于读取看门狗定时器的当前值，该寄存器为只读。

WDT中断

■ WDT中断与VIC的关系

WDT处于VIC的通道0，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

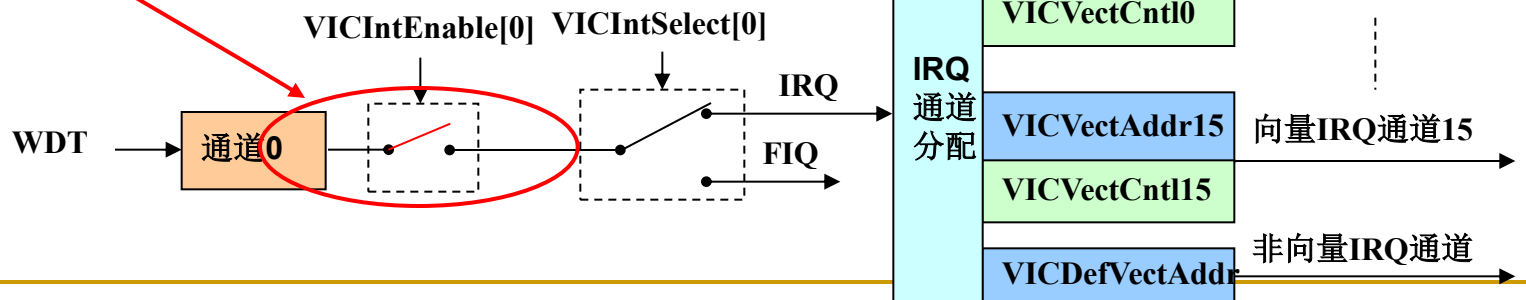


WDT中断

■ WDT中断与VIC的关系

WDT处于VIC的通道0，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

➤ 当VICIntEnable[0] = 0时，通道0中断禁止；

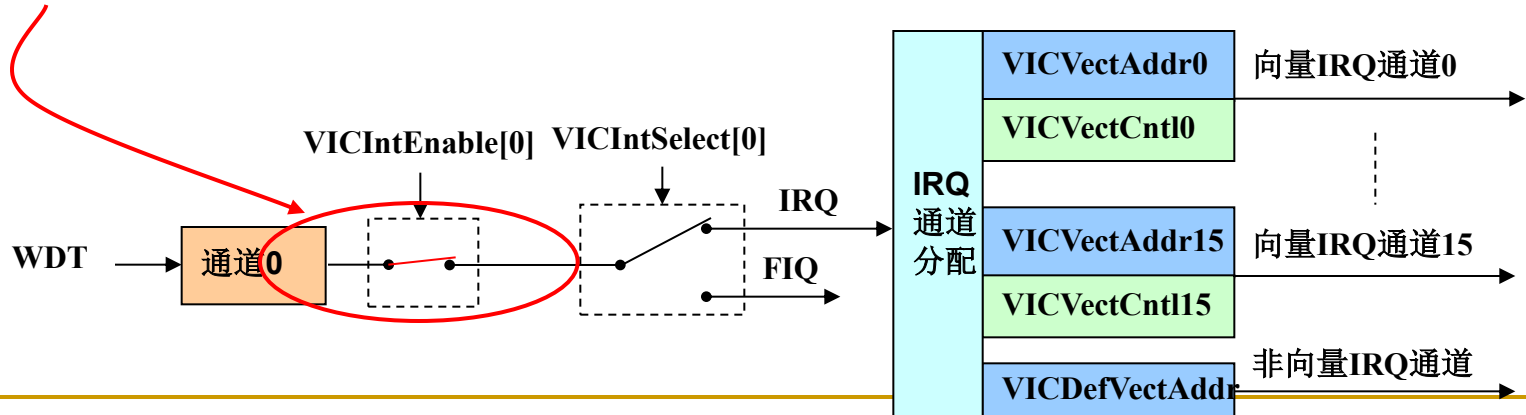


WDT中断

• WDT中断与VIC的关系

WDT处于VIC的通道0，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

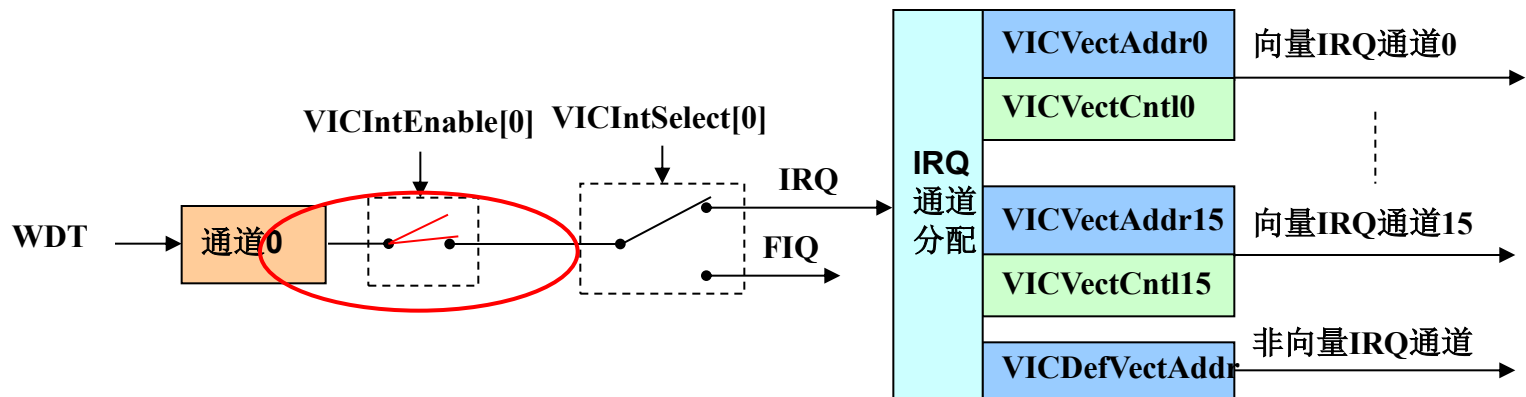
- 当VICIntEnable[0] = 0时，通道0中断禁止；
- 当VICIntEnable[0] = 1时，通道0中断使能；



WDT中断

- WDT中断注意事项:

WDT的中断标志位无法通过软件清零，只能通过硬件复位清零。因此，当发生WDT中断时，只能通过禁止WDT中断的方式返回，即， $VICIntEnable[0] = 0$ 。



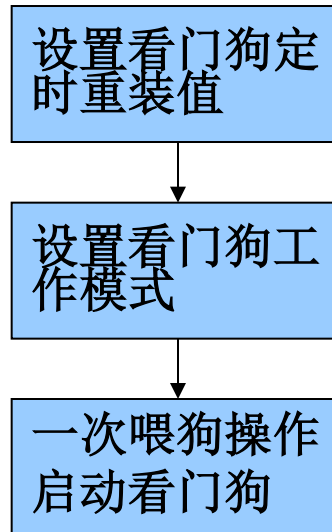
4.16 看门狗

■ 使用看门狗的注意要点

- WDT定时器为递减计数，向下溢出时产生中断和(或)复位；
- 使能看门狗后，必需要要执行一次正确的喂狗操作才能启动看门狗；
- 看门狗没有独立的振荡器，其使用PCLK作为时钟。所以CPU不能进入掉电模式，否则看门狗将停止工作；
- 看门狗溢出时间 = $N \times t_{\text{pclk}} \times 4$

- 使用示例

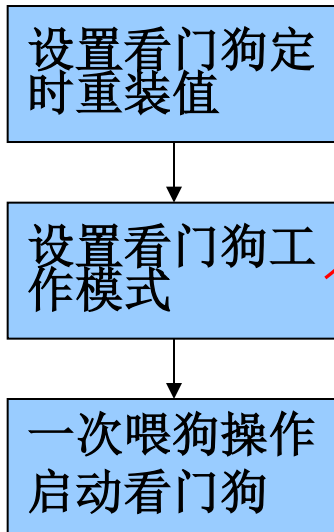
操作流程



• 使用示例

C代码:

操作流程



```
Void WDTInit(void)           // 看门狗初始化
{
    WDTTC = 0x10000;
    WDMOD = 0x03;
    WdtFeed();
}

Void WdtFeed(void)           // 喂狗程序
{
    WDFEED = 0xAA;
    WDFEED = 0x55;
}
```



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.17 脉宽调制器

■ 特性

- 独立的32位定时器/计数器；
- 7个匹配寄存器，可实现6路单边沿PWM输出，或3路双边沿PWM输出，或两者的混合输出；
- 所有PWM输出的频率都是相同的；
- 脉冲周期和宽度可以是任意的定时器计数值；
- 匹配寄存器更新与脉冲输出同步，防止产生错误脉冲。

- 脉冲宽度调制(PWM), 是英文 “Pulse Width Modulation”的缩写, 简称脉宽调制, 是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术, 广泛应用于从测量、通信到功率控制与变换的许多领域中。

- 对噪声抵抗能力的增强是**PWM**相对于模拟控制的另外一个优点，而且这也是在某些时候将**PWM**用于通信的主要原因。
- 从模拟信号转向**PWM**可以极大地延长通信距离。在接收端，通过适当的**RC**或**LC**网络可以滤除调制高频方波并将信号还原为模拟形式。
- **PWM**既经济、节约空间、抗噪性能强，是一种值得广大工程师在许多设计应用中使用的有效技术。

■ PWM软件法控制充电电流

- 利用单片机具有的PWM端口，在不改变PWM方波周期的前提下，通过软件的方法调整单片机的PWM控制寄存器来调整PWM的占空比，从而控制充电电流。本方法所要求的单片机必须具有ADC端口和PWM端口这两个必须条件，另外ADC的位数尽量高，单片机的工作速度尽量快。在调整充电电流前，单片机先快速读取充电电流的大小，然后把设定的充电电流与实际读取到的充电电流进行比较，若实际电流偏小则向增加充电电流的方向调整PWM的占空比；若实际电流偏大则向减小充电电流的方向调整PWM的占空比。在软件PWM的调整过程中要注意ADC的读数偏差和电源工作电压等引入的纹波干扰，合理采用算术平均法等数字滤波技术。

■ PWM在推力调制中的应用

- 脉宽调制发动机控制方式是在每一个脉动周期内，通过改变阀门在开或关位置上停留的时间来改变流经阀门的气体流量，从而改变总的推力效果，对于质量流率不变的系统，可以通过脉宽调制技术来获得变推力的效果。

■ PWM控制的制动器。

- 制动器是紧夹住某种东西的一种装置。许多制动器使用模拟输入信号来控制夹紧压力(或制动功率)的大小。加在制动器上的电压或电流越大，制动器产生的压力就越大。

4.17 脉宽调制器

■ PWM简介

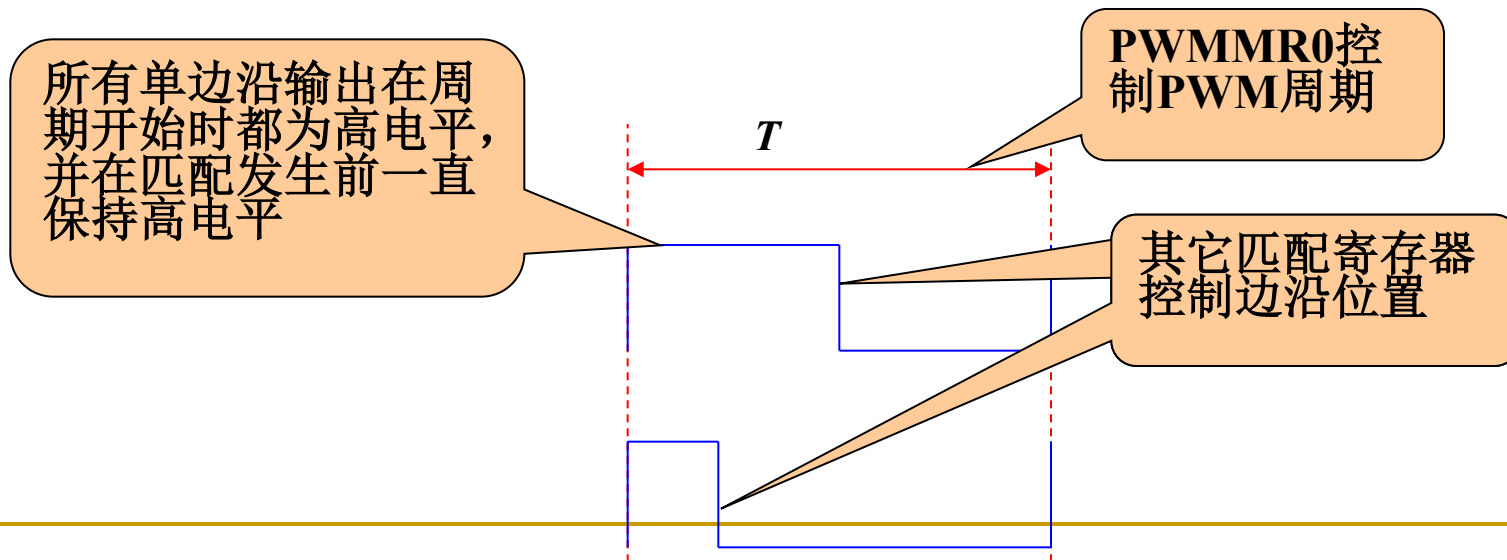
LPC2000的PWM基于标准的定时器模块，具有定时器的所有特性，它是定时器匹配功能的扩展。使用PWM功能，可以在指定引脚输出需要的波形。输出波形可分为两类：

- 单边沿输出；
- 双边沿输出。

■ PWM简介—单边沿输出

使用两个匹配寄存器就可以实现单边沿PWM输出。
这两个匹配寄存器的功能：

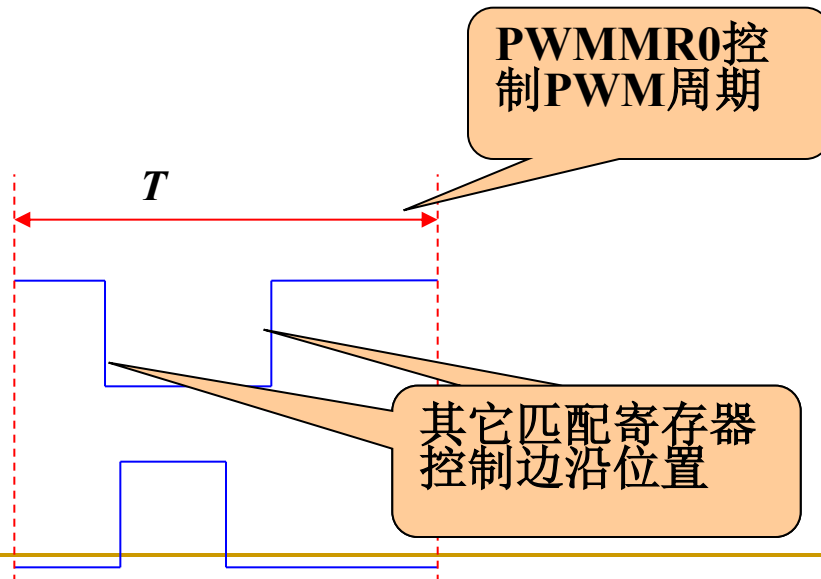
- 控制PWM周期——PWMMR0；
- 控制PWM的占空比——PWMMRn。



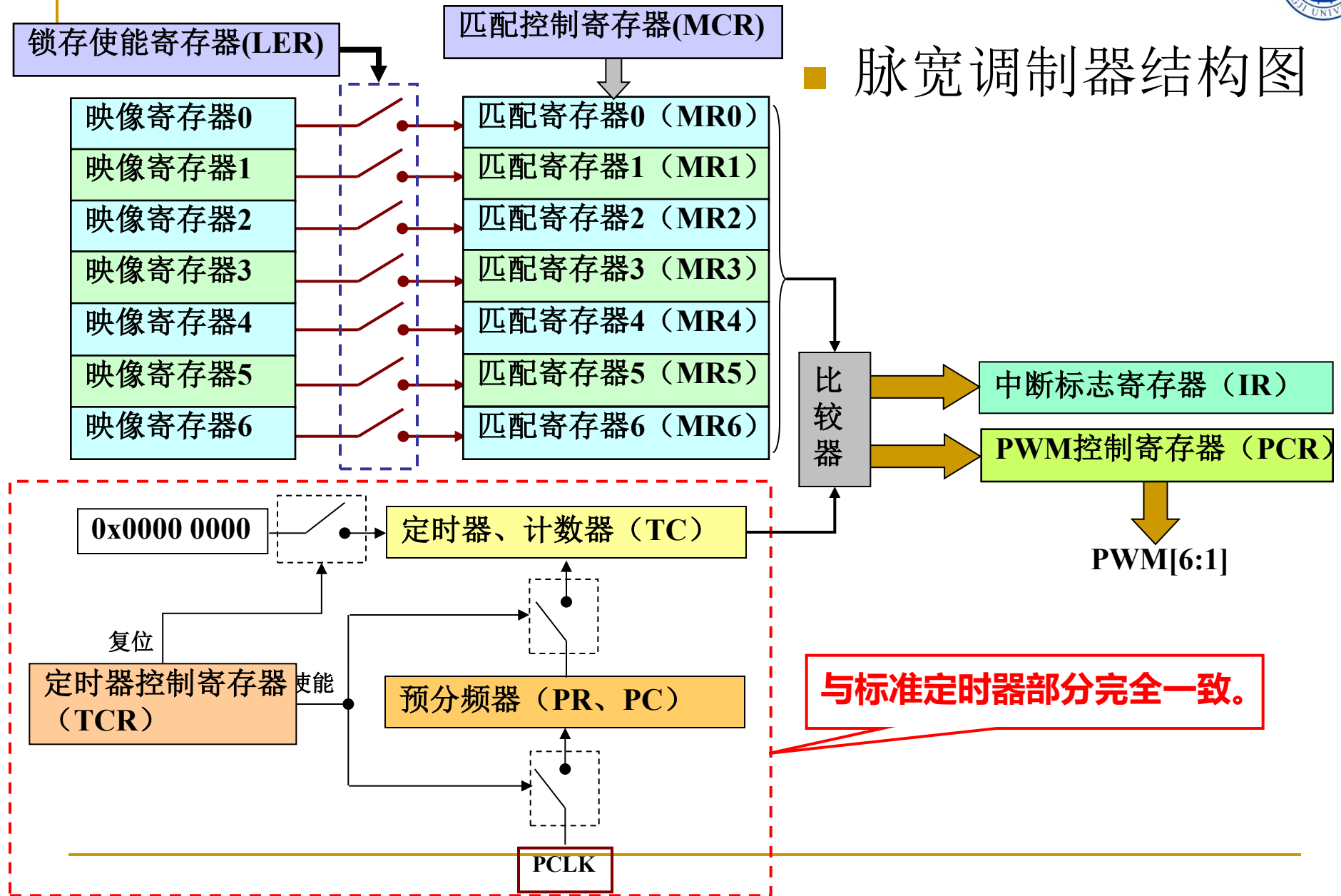
■ PWM简介—双边沿输出

实现双边沿输出需要3个匹配寄存器进行控制，其功能分别为：

- 控制PWM周期——PWMMR0；
- 控制PWM输出上升沿的位置；
- 控制PWM输出下降沿的位置。



■ 脉宽调制器结构图



4.17 脉宽调制器

■ PWM引脚描述

管脚名称	管脚方向	管脚描述
PWM1	输出	PWM通道 1 输出
PWM2	输出	PWM通道 2 输出
PWM3	输出	PWM通道 3 输出
PWM4	输出	PWM通道 4 输出
PWM5	输出	PWM通道 5 输出
PWM6	输出	PWM通道 6 输出

注意：虽然有7个匹配寄存器，但是并没有PWM0输出通道。因此，通常都使用匹配寄存器0来控制PWM通道的周期。

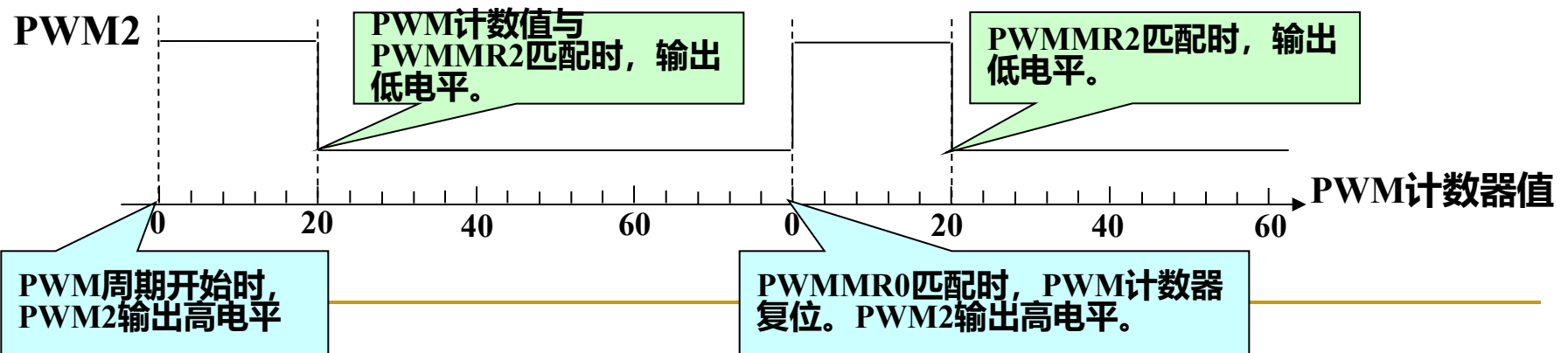
4.17 脉宽调制器

■ 单边沿PWM输出

- 单边沿PWM输出在PWM周期开始时为高电平；
- 单边沿PWM输出在到达其匹配值时会变为低电平。

举例：

单边沿输出通道：PWM2，PWMMR0 = 80，PWMMR2 = 20。



4.17 脉宽调制器

■ 双边沿PWM输出

- 双边沿PWMn输出的**上升沿**位置在PWMMRn-1处;
- 双边沿PWMn输出的**下降沿**位置在PWMMRn处。

举例:

双边沿输出通道: PWM2, PWMMR0 = 80, PWMMR1 = 20, PWMMR2 = 40。

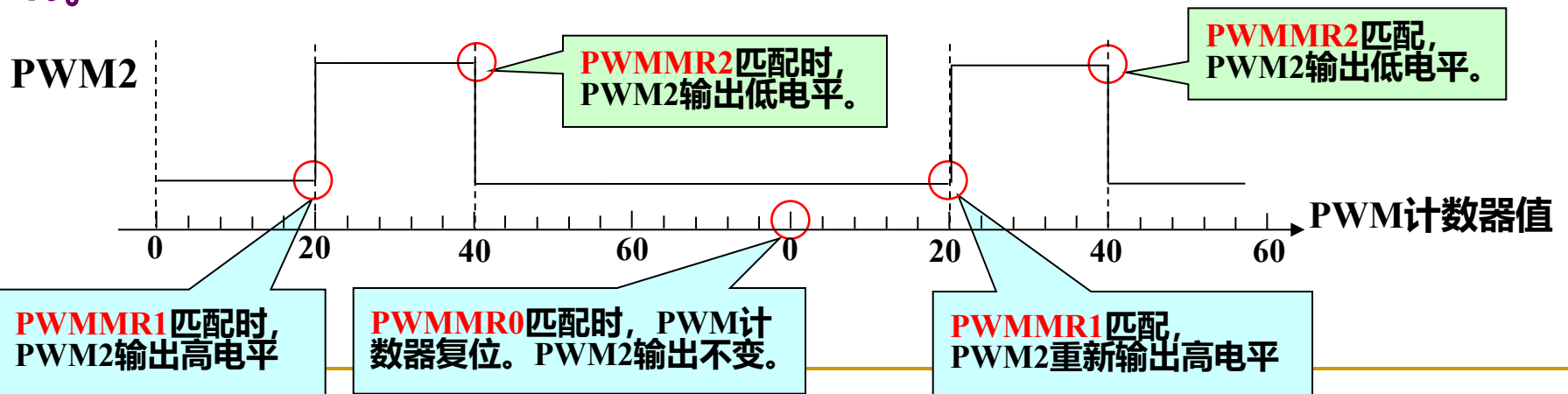
4.17 脉宽调制器

■ 双边沿PWM输出

- 双边沿PWMn输出的**上升沿**位置在PWMMRn-1处；
- 双边沿PWMn输出的**下降沿**位置在PWMMRn处。

举例：

双边沿输出通道：PWM2，PWMMR0 = 80，PWMMR1 = 20，PWMMR2 = 40。



4.17 脉宽调制器

■ PWM输出触发关系

PWM通道	单边沿PWM (PWMSELn = 0)		双边沿PWM (PWMSELn = 1)	
	上升沿	下降沿	上升沿	下降沿
1	匹配0	匹配1	匹配0	匹配1
2	匹配0	匹配2	匹配1	匹配2
3	匹配0	匹配3	匹配2	匹配3
4	匹配0	匹配4	匹配3	匹配4
5	匹配0	匹配5	匹配4	匹配5
6	匹配0	匹配6	匹配5	匹配6

举例说明：使用PWM通道5输出时。

如果为单边沿输出，那么匹配0事件置位输出引脚，匹配5事件清零输出引脚。
如果为双边沿输出，那么匹配4事件置位输出引脚，匹配5事件清零输出引脚。

4.17 脉宽调制器

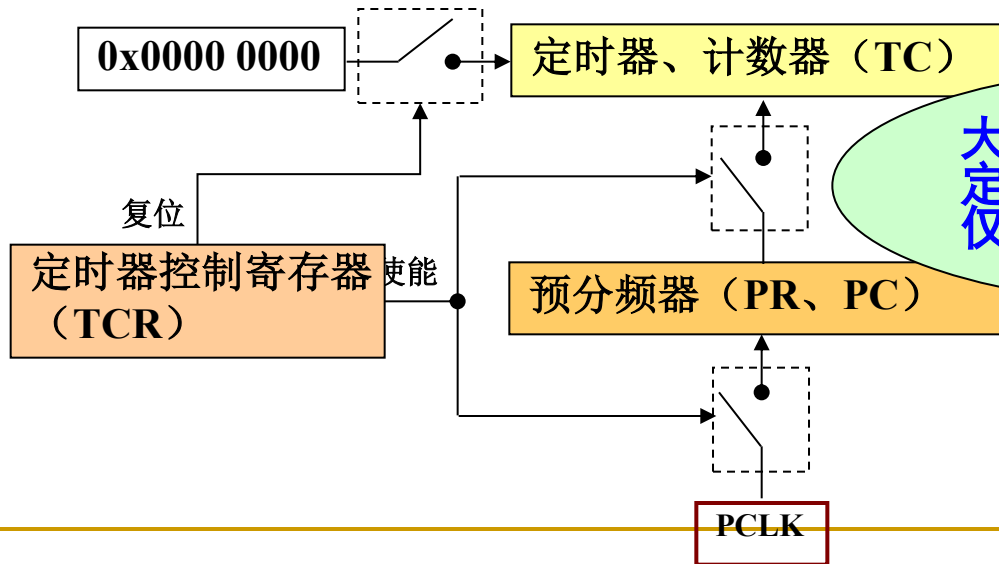
■ PWM寄存器描述

PWM模块建立在定时器的基础上，所以一部分寄存器功能与定时器功能类似。所有寄存器大致可以分为两类：

- 基本功能寄存器
- 匹配控制寄存器

PWM寄存器描述—基本功能寄存器

名称	描述	访问
PWMTCR	PWM定时器控制寄存器 。控制PWM定时/计数器功能（禁止或复位）。	读写
PWMTC	PWM定时器计数器 。为32位计数器，计数频率为PCLK经过预分频计数器后的频率值。	读写
PWMPR	PWM预分频控制寄存器 。用于设定预分频值，为32位寄存器。	读写
PWMPC	PWM预分频计数器 。为32位计数器，计数频率为PCLK，当计数值等于预分频计数器的值时，TC计数器加一。	读写



大部分寄存器的功能与定时器部件相同，所以仅介绍与之不同的地方

• PWM寄存器描述—定时器控制寄存器

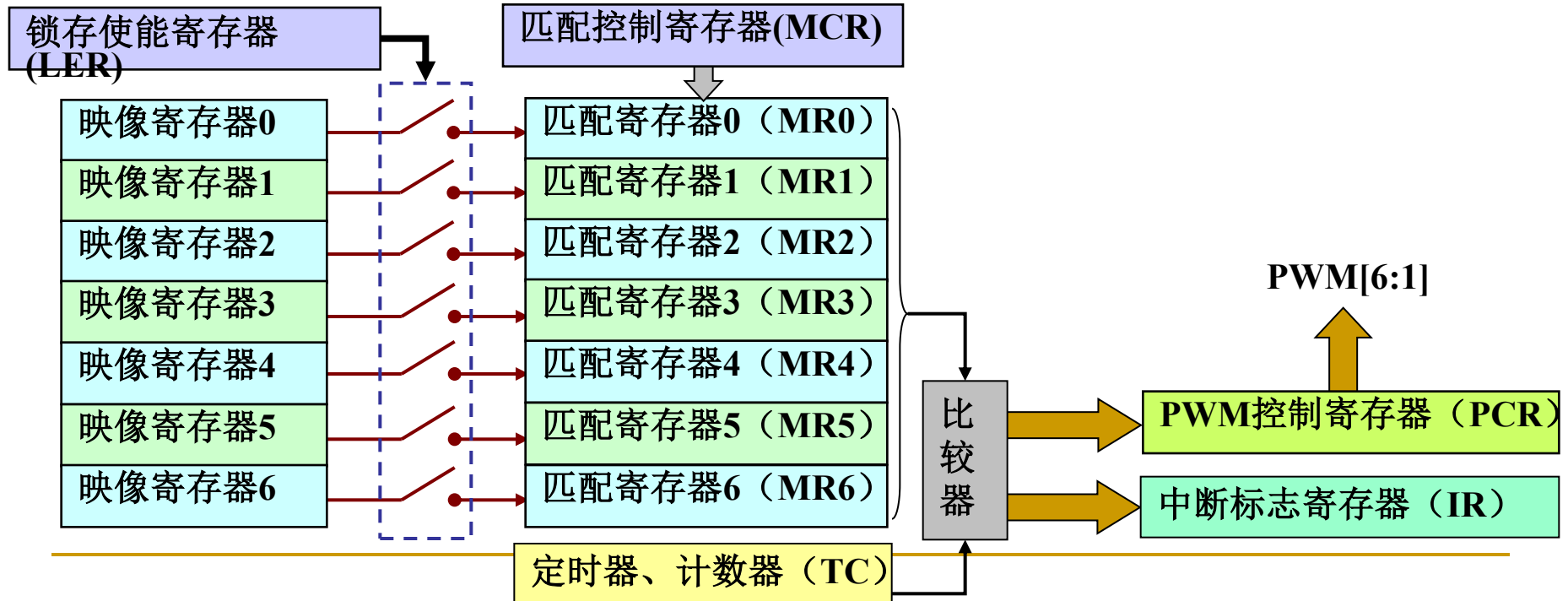
PWM定时器控制寄存器TCR含有两个功能：

- 控制定时器计数器的操作；
- 控制PWM的使能。

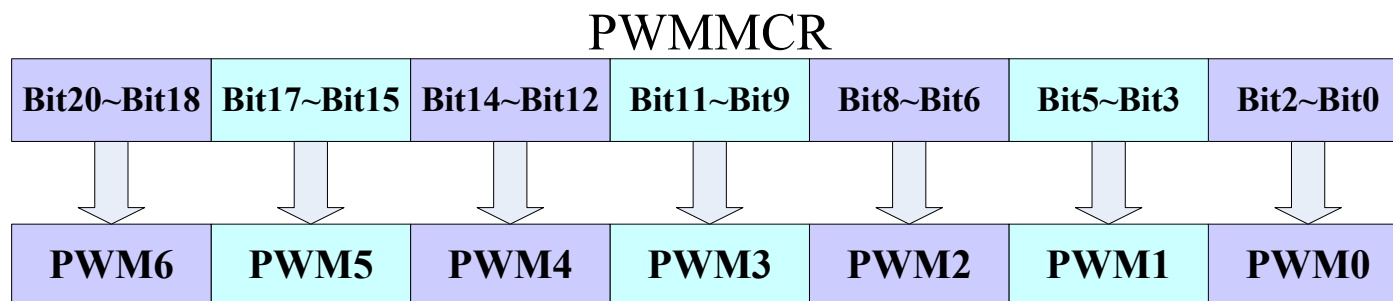
TCR	功能	描述	复位值
0	计数器使能	1:定时器计数器和预分频计数器使能计数； 0:定时器计数器和预分频计数器停止计数。	0
1	计数器复位	为1时，定时器计数器和预分频计数器在PCLK的下一个上升沿同步复位。计数器在TCR的bit1恢复为0之前保持复位状态。	0
2	保留	用户程序不要向该位写入1	NA
3	PWM使能	为1时，PWM模式使能。为0时，PWM禁止	0

PWM寄存器描述—匹配功能寄存器

名称	描述	访问
PWMCR	PWM匹配控制寄存器。用于控制匹配时是否产生中断或复位TC。	读写
PWMR0 ~ 6	PWM匹配寄存器。每个寄存器对应于一个匹配值。	读写
PWMPCR	PWM控制寄存器。使能PWM输出并选择PWM通道类型为单边沿或双边沿控制。	读写
PWMLER	PWM锁存使能寄存器。使能新的PWM匹配值有效。	读写
PWMIR	PWM中断标志寄存器。读该寄存器识别中断源。	读写



PWM寄存器描述—匹配控制寄存器



位	功能	描述	复位值
0	中断 (PWMMR0)	1: PWMMR0与PWMTTC值匹配时, 将产生中断。 0: 该中断被禁止。	0
1	复位 (PWMMR0)	1: PWMMR0与PWMTTC值匹配时, 将使PWM计数器复位。 0: 该特性被禁止。	0
2	停止 (PWMMR0)	1: PWMMR0与PWMTTC值匹配时, 将使PWM计数器停止。 0: 该特性被禁止。	0
3:20	PWMMR1~6	功能与PWMMR0相同 (略)	0

• PWM寄存器描述—匹配寄存器

PWM匹配寄存器值连续与PWM定时器计数值相比较当这两个值匹配时，根据匹配控制寄存器的设置，自动执行相应动作：产生中断、复位PWM计数器、停止PWM计数器。

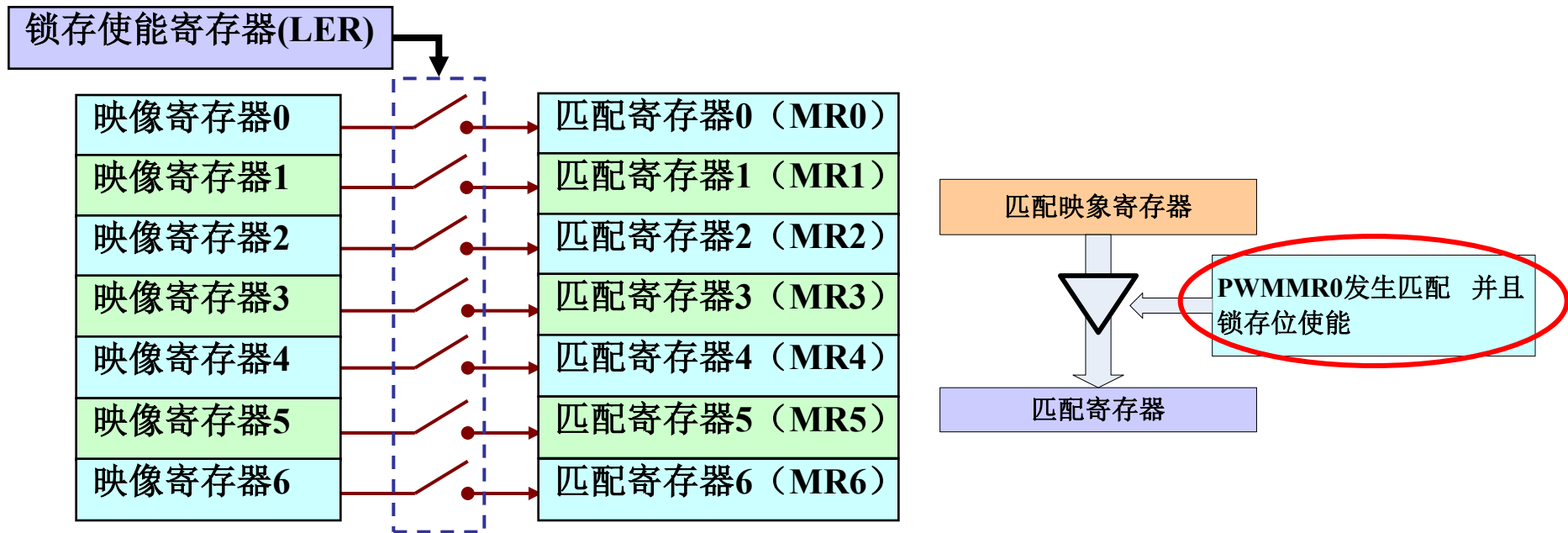
位	31 : 0	复位值
功能	设定的匹配值	0

通常使用匹配寄存器0（PWMMR0）来控制PWM的周期、频率。

$$\text{PWM 的输出频率为: } f = \frac{F_{\text{pclk}}}{\text{PWMMR0}(\text{PWMPR} + 1)}$$

$$\text{PWM 的输出周期为: } T = \frac{\text{PWMMR0}(\text{PWMPR} + 1)}{F_{\text{pclk}}}$$

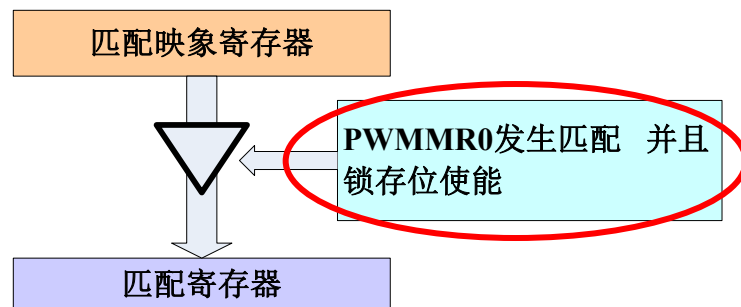
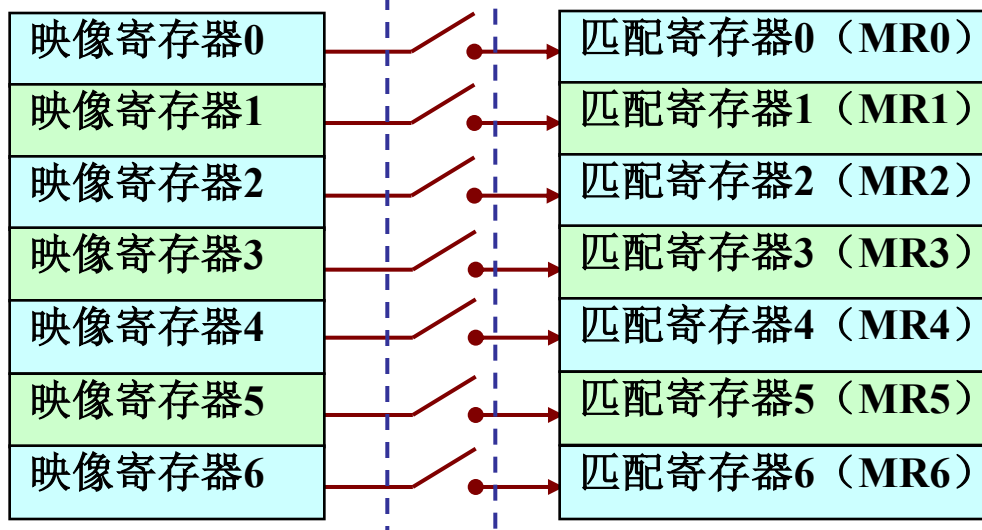
• PWM寄存器描述—锁存使能寄存器



在PWM运行模式下，修改PWM匹配寄存器时，新值并不直接写入到目标寄存器中，而是写入相应的映象寄存器中。在PWMMR0发生匹配并且PWMLER中的相应位置位时才能使修改值生效。

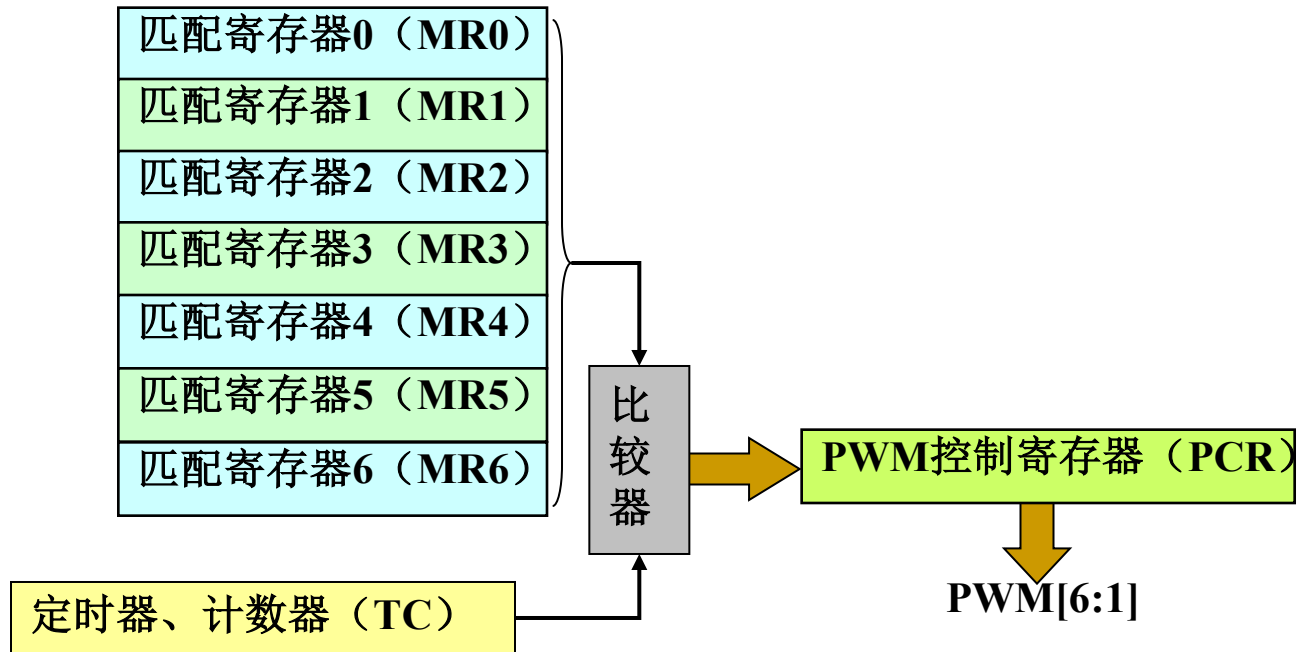
PWM寄存器描述—锁存使能寄存器

锁存使能寄存器(LER)



位	功能	描述	复位值
0	使能PWM 匹配0锁存	置位后，写入PWM匹配寄存器0（PWMMR0）的值，只有在PWMMR0发生匹配事件后才能生效。	0
6 : 1	使能PWM 匹配1~6锁存	功能同上，只是分别对应匹配1到匹配6。	0
7	保留	用户程序不要向该位写入1。	NA

• PWM寄存器描述—PWM控制寄存器



PWM控制寄存器用于使能并选择每个PWM通道的类型。

• PWM寄存器描述—PWM控制寄存器

PWMPCR位	功能	描述	复位值
1 : 0	保留	用户程序不要向这些位写入1。	NA
2	PWMSEL2	● 0: PWM2通道选择为单边沿控制模式。 ● 1: PWM2通道选择为双边沿模式。	0
3	PWMSEL3	功能同上，控制PWM3通道的输出模式。	0
4	PWMSEL4	功能同上，控制PWM4通道的输出模式。	0
5	PWMSEL5	功能同上，控制PWM5通道的输出模式。	0
6	PWMSEL6	功能同上，控制PWM6通道的输出模式。	0
8 : 7	保留	用户程序不要向这些位写入1。	NA
9	PWMENA1	为1时，使能PWM1输出；为0时禁止。	0
10	PWMENA2	为1时，使能PWM2输出；为0时禁止。	0
11	PWMENA3	为1时，使能PWM3输出；为0时禁止。	0
12	PWMENA4	为1时，使能PWM4输出；为0时禁止。	0
13	PWMENA5	为1时，使能PWM5输出；为0时禁止。	0
14	PWMENA6	为1时，使能PWM6输出；为0时禁止。	0
15	保留	用户程序不要向这些位写入1。	NA

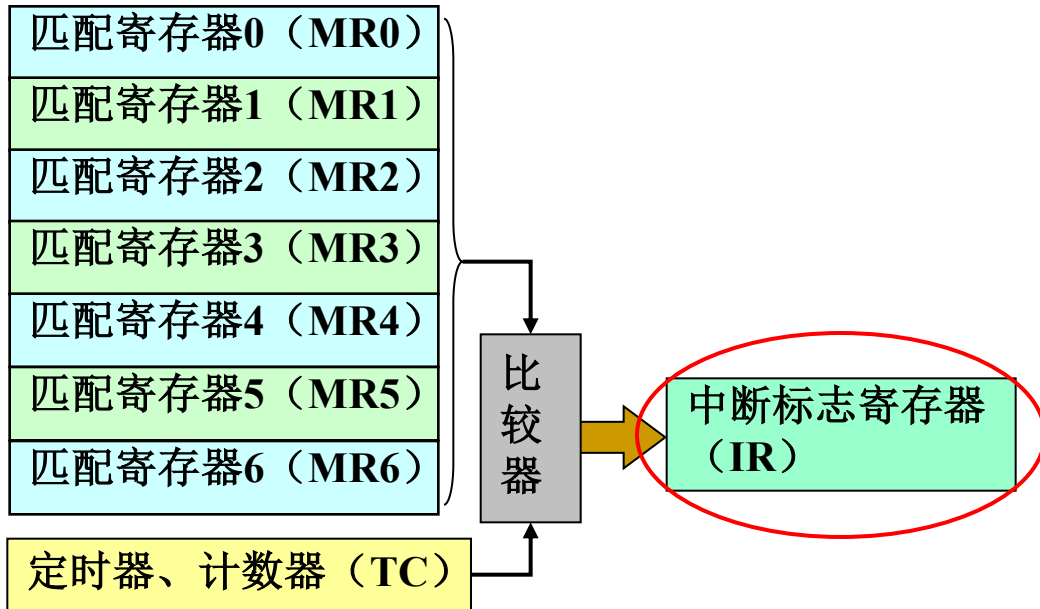
4.17 脉宽调制器

• PWM寄存器描述—PWM控制寄存器

说明：

- PWM1的双边沿输出与单边沿是相同的，因此，PWM1不具有双边沿PWM输出。
- 如果将PWM3和PWM5用作双边沿PWM输出，那么此时只能有2路双边沿PWM输出。
- 只有将PWM2、PWM4和PWM6作为双边沿输出时，才能够达到3路双边沿PWM输出。

寄存器描述—中断标志寄存器



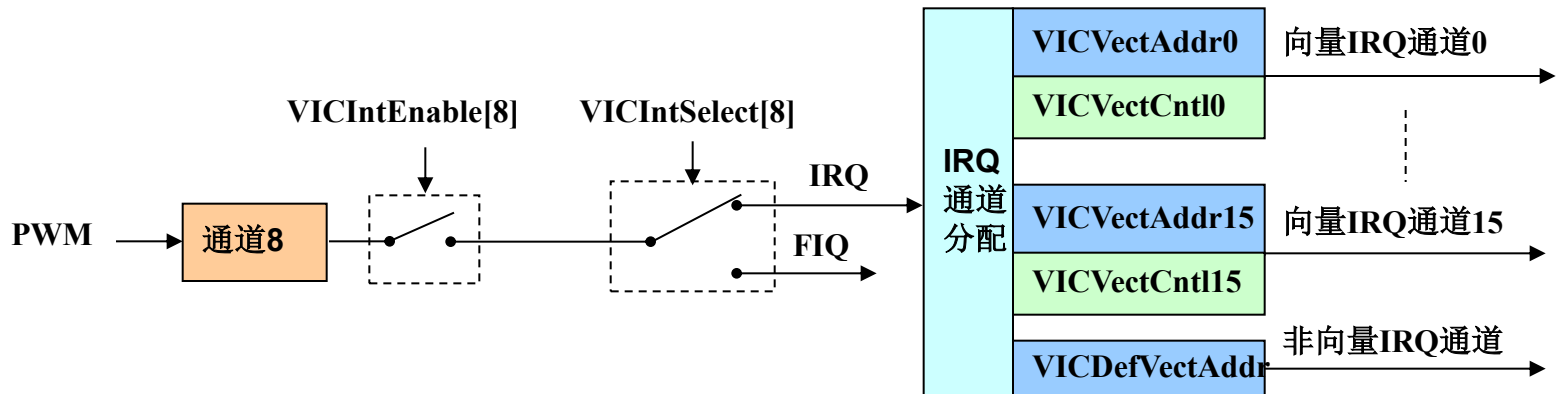
中断寄存器包含11个位。其中7个位用于匹配中断，其它的位保留。中断发生时，对应位将被置“1”。向对应的IR位写入1会复位中断，写入0无效。

位	功能	描述	位	功能	描述
0	PWMMR0	匹配 0 中断	4:7	—	保留
1	PWMMR0	匹配 1 中断	8	PWMMR4	匹配 4 中断
2	PWMMR0	匹配 2 中断	9	PWMMR5	匹配 5 中断
3	PWMMR0	匹配 3 中断	10	PWMMR6	匹配 6 中断

PWM中断

■ PWM中断与VIC的关系

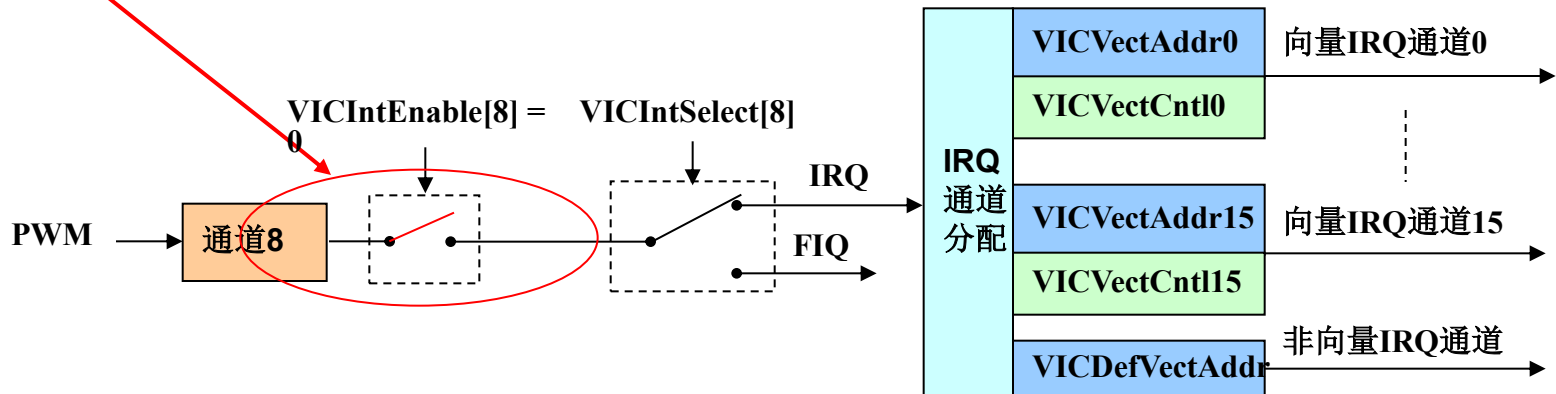
LPC2000系列ARM含有1个脉宽调制器(PWM)，可以产生7路匹配中断,PWM位于VIC的通道8。中断使能寄存器VICIntEnable的Bit8用来控制通道8的使能。



PWM中断

- PWM与VIC的关系

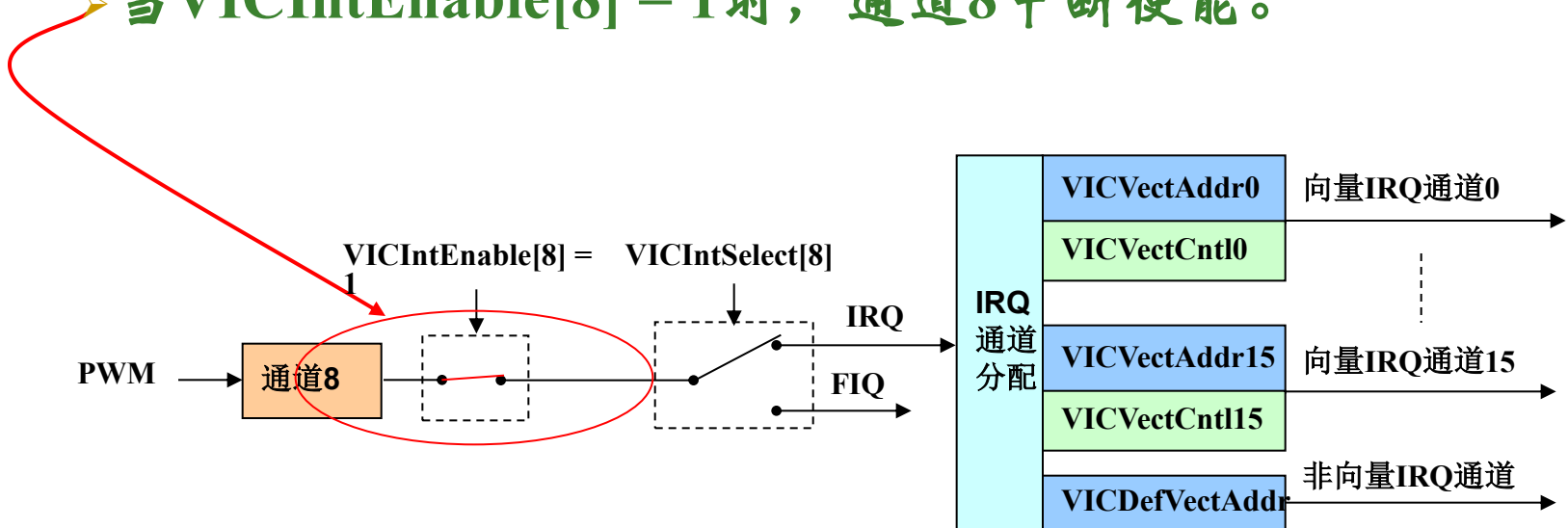
➤ 当 $VICIntEnable[8] = 0$ 时，通道8中断禁止；



PWM中断

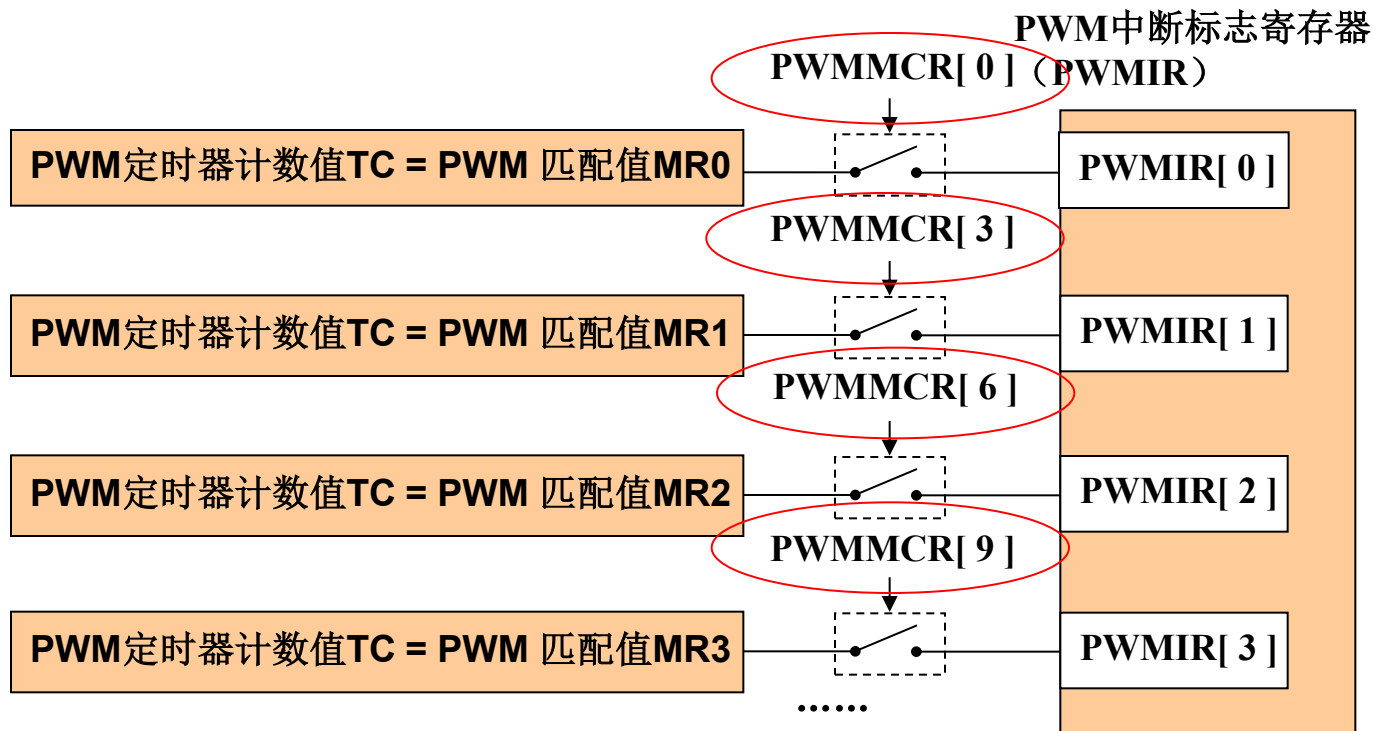
- PWM与VIC的关系

- 当 $VICIntEnable[8] = 0$ 时，通道8中断禁止；
- 当 $VICIntEnable[8] = 1$ 时，通道8中断使能。



PWM中断

PWM中断是匹配中断，PWM含有7个匹配寄存器，当发生匹配事件时，便触发中断。



匹配控制寄存器MCR控制匹配中断的使能，相关位为“1”时，中断使能。

4.17 脉宽调制器

■ 使用PWM的注意事项

- 所有的PWM输出频率都是相同的。
- 修改匹配寄存器之后，必须设置锁存使能寄存器中的相应位，否则匹配寄存器的值不能生效；
- 修改匹配寄存器时，不需要停止PWM定时器，以免产生不完整的PWM波形；
- 不使用PWM功能时，可将该部件作为一个标准的32位定时器使用；
- **$\text{PWMTTC计数频率} = F_{\text{pclk}} / (\text{PWMPR} + 1)$**

• PWM使用示例

操作流程

连接PWM输出
功能引脚



设置预分频值



设置匹配值



设置PWM
输出方式



启动PWM定时器

• PWM使用示例一 单边沿输出

PWM1输出50%占空比方波

```
Void PWM1Out(uint16 FREQ)
{
```

输出频率，单位Hz

```
PINSEL0 &= 0xFFFFF0C;
```

```
PINSEL0 |= 0x00000002;
```

设置引脚连接模块
连接PWM1输出

```
PWMPCR = 0x200;
```

使能PWM1输出

```
PWMMCR = 0x02;
```

MR0匹配后复位定时器

```
PWMPR = 0x00;
```

设置预分频值为0

```
PWMMR0 = Fpclk / FREQ;
```

设置PWM周期

```
PWMMR1 = PWMMR0 / 2;
```

设置占空比为50%

```
PWMLER = 0x03;
```

使能PWM匹配锁存

```
PWMTCR = 0x09;
```

使能PWM，启动定时器

```
}
```

• PWM使用示例一 双边沿输出

PWM2双边沿控制输出

```
Void PWM2Out(uint16 FREQ)
```

输出频率，单位Hz

```
{
```

```
PINSEL0 &= 0xFFFF3FFF;
```

```
PINSEL0 |= 0x00008000;
```

设置引脚连接模块连接PWM1输出

```
PWMPCR = 0x404;
```

设置PWM2双边沿输出

```
PWMMCR = 0x02;
```

MR0匹配后复位定时器

```
PWMPR = 0x00;
```

设置预分频值为0

```
PWMMR0 = Fpclk / FREQ;
```

设置PWM周期

```
PWMMR1 = PWMMR0 / 5;
```

设置前沿在周期的1/5处

```
PWMMR2 = PWMMR1 * 2;
```

设置后沿在周期的2/5处

```
PWMLER = 0x07;
```

使能PWM匹配锁存

```
PWMTCR = 0x09;
```

使能PWM，启动定时器

```
}
```



LPC2000系列ARM硬件结构

- 1.LPC2000系列简介
- 2.引脚描述
- 3.存储器寻址
- 4.系统控制模块
- 5.存储器加速模块 (MAM)
- 6.外部存储器控制器 (EMC)
- 7.引脚连接模块
- 8. GPIO
- 9. 向量中断控制器
- 10.外部中断输入
- 11.定时器0和定时器1
- 12. SPI接口
- 13. I²C接口
- 14. UART(0、1)
- 15. A/D转换器
- 16. 看门狗
- 17. 脉宽调制器(PWM)
- 18. 实时时钟

4.18 实时时钟

■ 特性

LPC2000系列微控制器内部集成了一个功能完整的实时时钟模块，它具有如下特性：

- 带有日历和时钟功能；
- 超低功耗设计；
- 提供秒、分、小时、日、月、年和星期；
- 可编程基准时钟分频器允许调节RTC以适应不同的晶振频率。

- RTC的英文全称是Real-Time Clock

RTC（实时时钟）功能：通常采用RTC 来提供可靠的系统时间，包括时、分、秒、和年、月、日等；而且要求在系统处于关机状态下它也能够正常工作（后备电池供电），它的外围也不需要太多的辅助电路，典型的的就是只需要一个高精度的32.768KHz 晶振和电阻电容等。

RTC应用：现在很多电子产品都有RTC功能，如电子日历（台式、壁式等）、手持数码产品（手机、电子词典、各种学习机、照相机、摄像机等）、电子计量仪表（电度表、燃气表、水表等）、家用电器（电视机、机顶盒、DVD等）等，应用非常广泛。

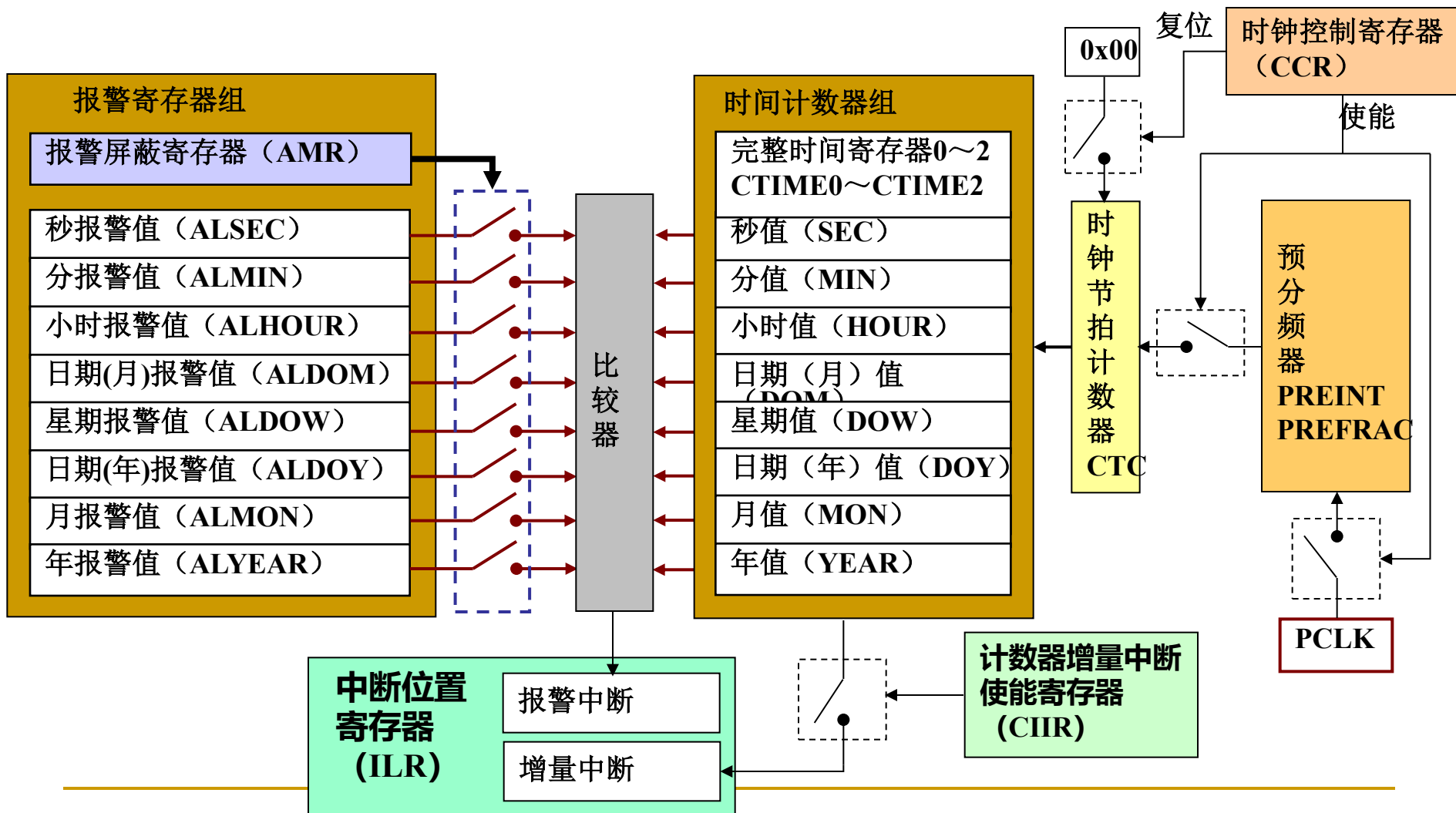
4.18 实时时钟

■ 描述

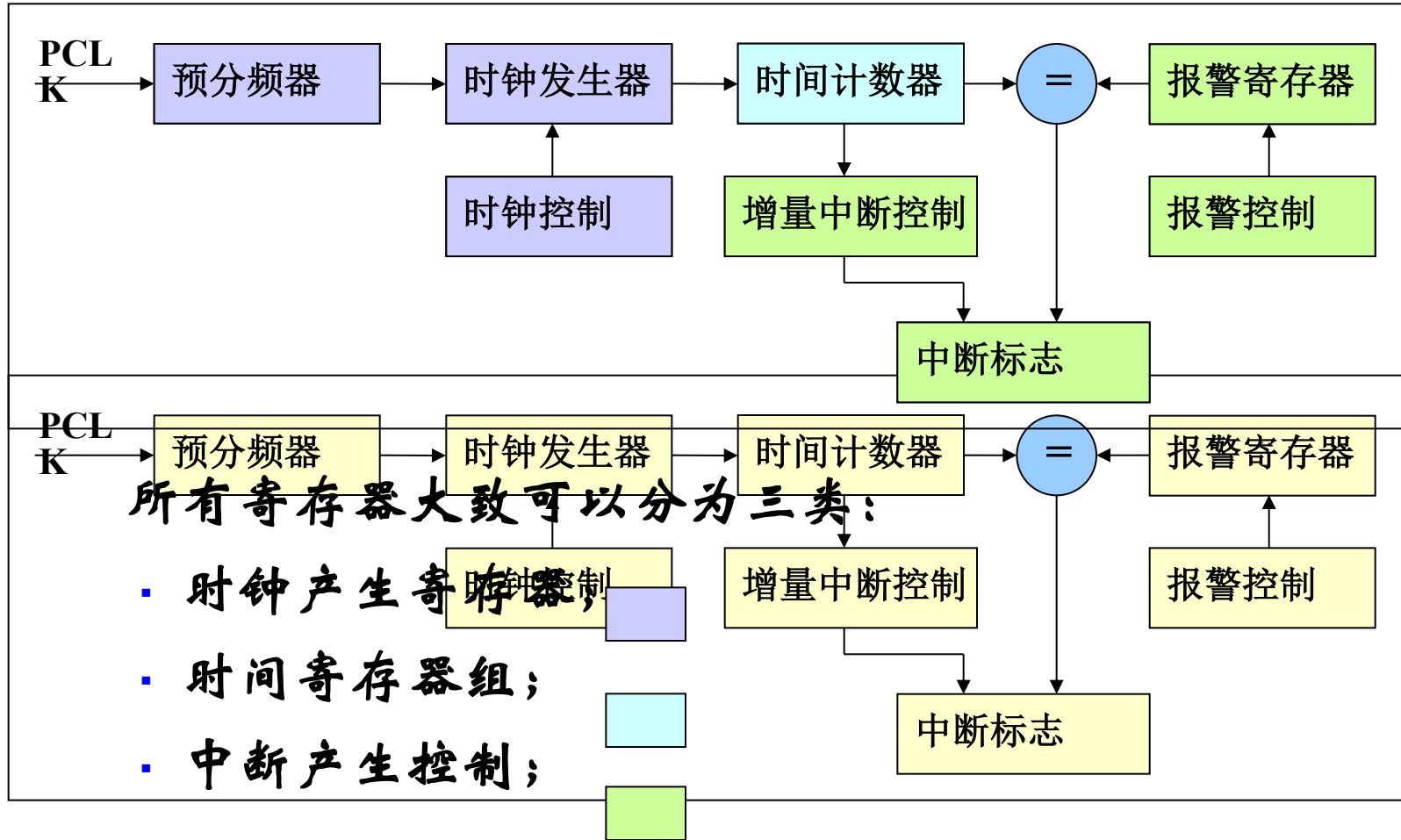
实时时钟(RTC)提供一套计数器在系统工作时对时间进行测量。RTC消耗的功率非常低,这使其适合于由电池供电的,CPU不连续工作(空闲模式)的系统。

注意: LPC22xx、LPC211x系列微控制器的RTC没有独立的时钟源,它使用的时钟频率是通过将Fpclk分频得到,所以CPU不能进入掉电模式。

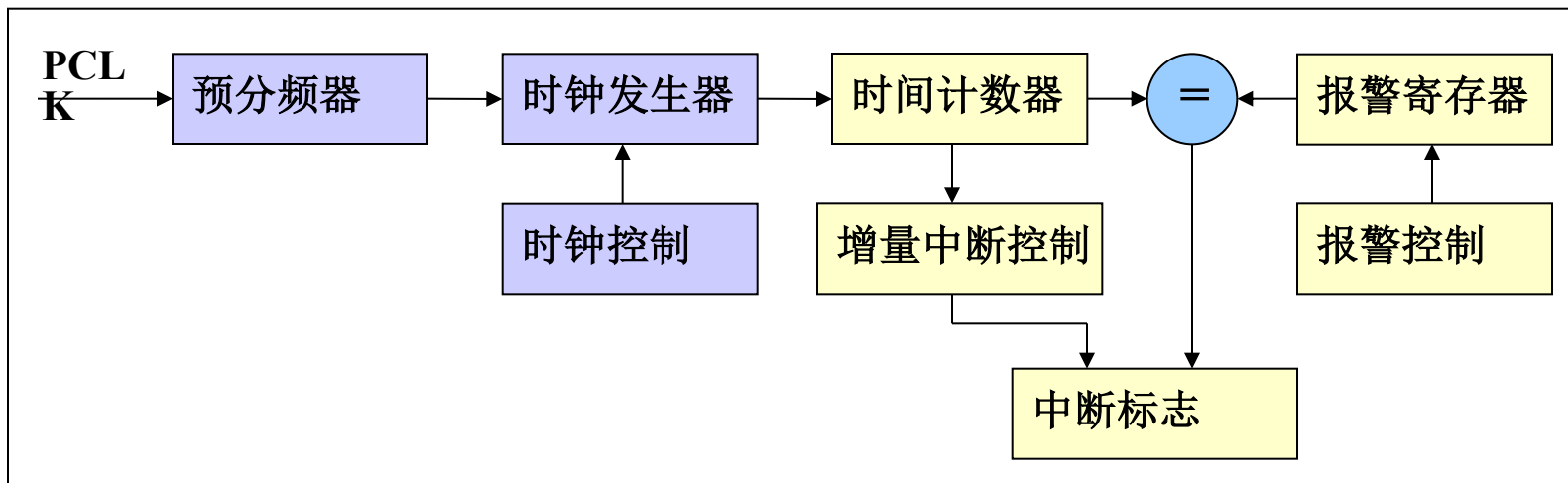
■ 实时时钟结构图



寄存器描述



寄存器描述—时钟产生寄存器组



时钟产生寄存器组控制产生RTC时间计数器需要的秒时钟信号。

名称	有效位	描述	访问
PREINT	13	预分频值，整数部分	读写
PREFRAC	15	预分频值，小数部分	读写
CCR	4	时钟控制寄存器	读写
CTC	15	时钟节拍计数器	只读

• 寄存器描述—预分频寄存器

名称	有效位	描述	访问
PREINT	13	预分频值，整数部分	读写
PREFRAC	15	预分频值，小数部分	读写
CCR	4	时钟控制寄存器	读写
CTC	15	时钟节拍计数器	只读

预分频器用于将任何频率高于65.536KHz的PCLK时钟分频产生32.768KHz的基准时钟。这样就可以不管外设时钟的频率为多少，RTC总是以正确的速率运行。

预分频器在四十八分频和四十八分频之间，因此有一个节拍一定是32768

名称	有效位	描述	访问
PREINT	13	预分频值，整数部分	读写
PREFRAC	15	预分频值，小数部分	读写
CCR	4	时钟控制寄存器	读写
CTC	15	时钟节拍计数器	只读

• 寄存器描述—预分频寄存器

名称	有效位	描述	访问
PREINT	13	预分频值，整数部分	读写
PREFRAC	15	预分频值，小数部分	读写
CCR	4	时钟控制寄存器	读写
CTC	15	时钟节拍计数器	只读

预分频整数部分为13位有效位，小数部分为15位有效位。

预分频整数部分的计算公式为：

$$\text{PREINT} = \text{int}(\text{PCLK} / 32768) - 1$$

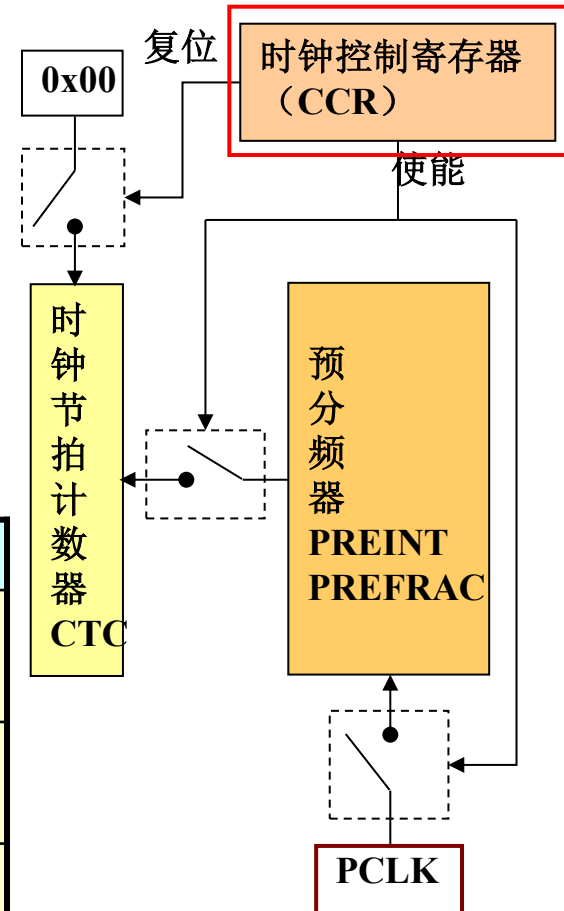
预分频小数部分的计算公式为：

$$\text{PREFRAC} = \text{PCLK} - ((\text{PREINT} + 1) \times 32768)$$

寄存器描述—时钟控制寄存器

时钟控制寄存器包含4位有效位，用来对时钟分频电路进行控制，包括启动RTC和复位时钟节拍计数器(CTC)等功能。

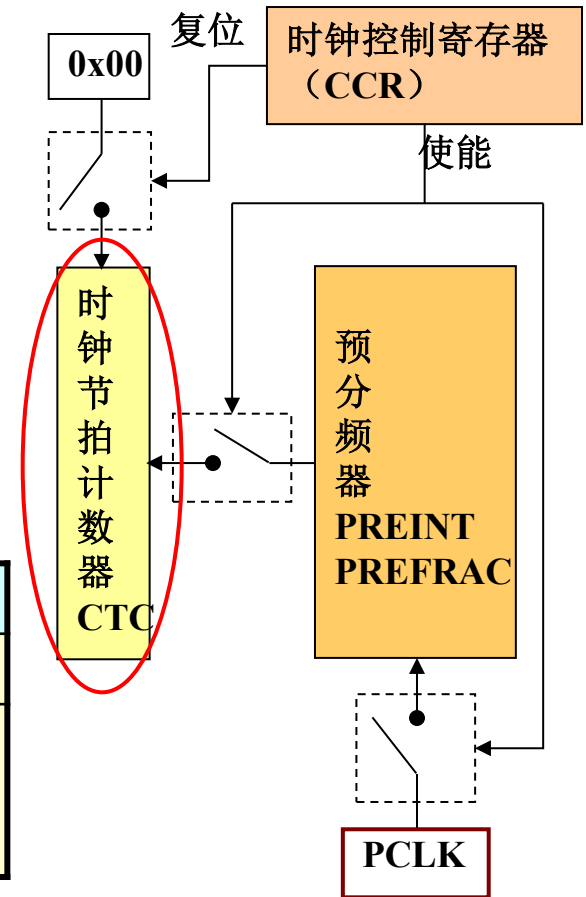
位	功能	描述
0	CLKEN	时钟使能。当该位为1时，时间计数器使能。为0时，时间计数器被禁止，这时可对其初始化。
1	CTCRST	CTC复位。为1时，时钟节拍计数器复位。在CCR的bit1变为0之前，它将一直保持复位状态。
3 : 2	CTTEST	测试使能。在正常操作中，这些位应当全为0。



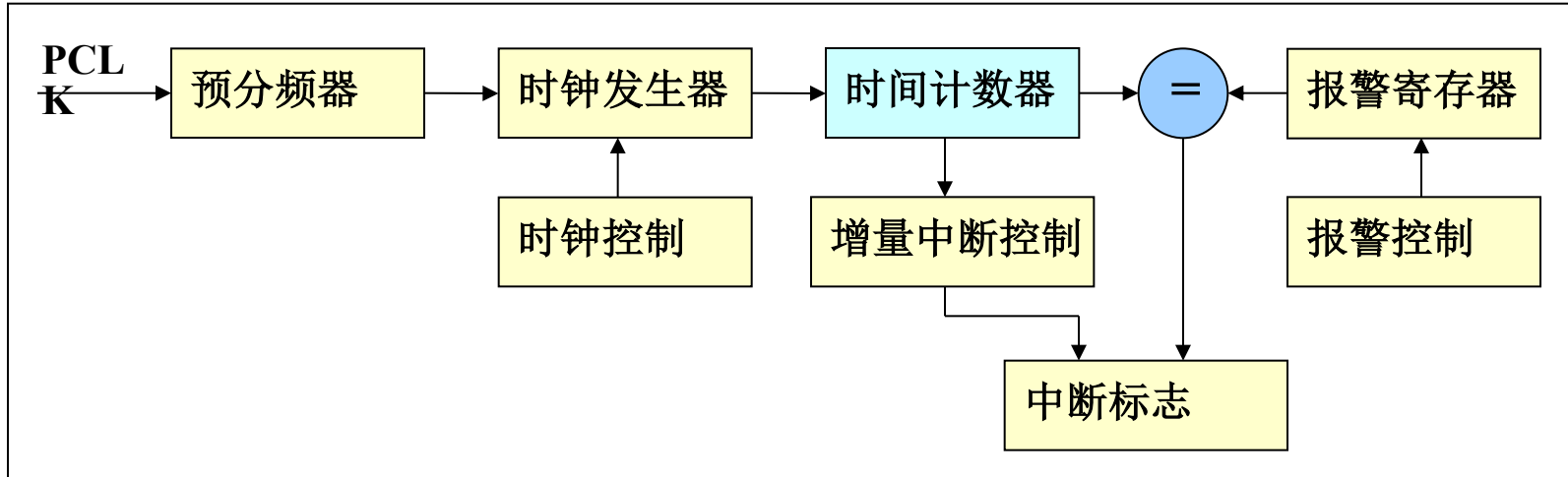
寄存器描述—时钟节拍计数寄存器

时钟节拍计数器对预分频器的输出时钟进行计数，用于产生秒的时钟节拍。它是一个只读寄存器，但它可通过时钟控制寄存器(CCR)复位为0。

位	功能	描述
0	保留	用户程序不要向该位写入1
15 : 1	时钟节拍计数器	对预分频器的时钟输出进行计数，每计满32768个时钟，该寄存器复位，并向时间计数器发送一个秒脉冲

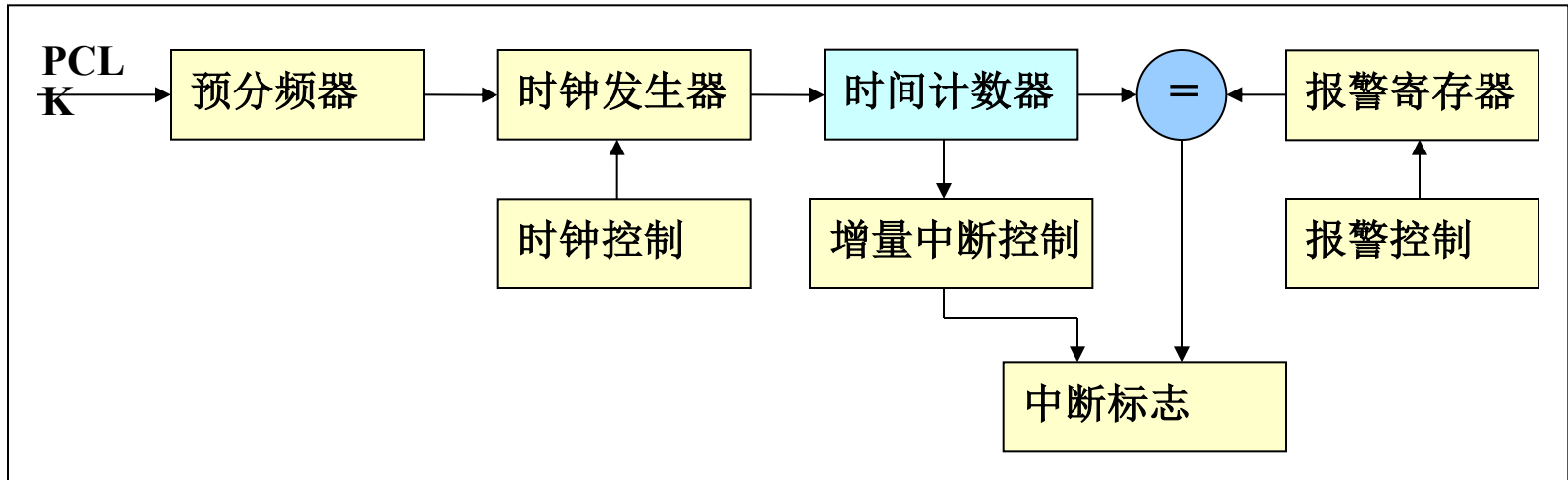


• 寄存器描述—时间计数器



时间计数器组中包含当前的时间，它们分为两类：
完整时间寄存器和分类时间寄存器。

寄存器描述—时间计数器



在**完整时间寄存器**中，时间以一个比较完整的格式存储，程序只需要3次读操作即可读出所有计数器值。这些寄存器为只读寄存器。

名称	有效位	描述
CTIME0	32	包含秒、分、时和星期。
CTIME1	32	包含日期（月）、月和年
CTIME2	32	包含日期（年）

寄存器描述—时间计数器

名称	有效位	描述
CTIME0	32	包含秒、分、时和星期。
CTIME1	32	包含日期（月）、月和年
CTIME2	32	包含日期（年）

CTIME0	31:27	26:24	23:21	30:16	15:14	13:8	7:6	5:0
功能	保留	星期	保留	小时	保留	分	保留	秒
取值范围	—	(0~6)	—	(0~23)	—	(0~59)	—	(0~59)

CTIME1	31:28	27:16	15:12	11:8	7:5	4:0
功能	保留	年	保留	月	保留	日期(月)
取值范围	—	(0~4095)	—	(1~12)	—	(1~28,29,30)

CTIME2	31:9	8:0
功能	保留	日期(年)
取值范围	—	(1~365,366)

这两个寄存器意义不同，(月)表示当日在当月中序号，(年)表示当日在当年中的序号

• 寄存器描述—时间计数器

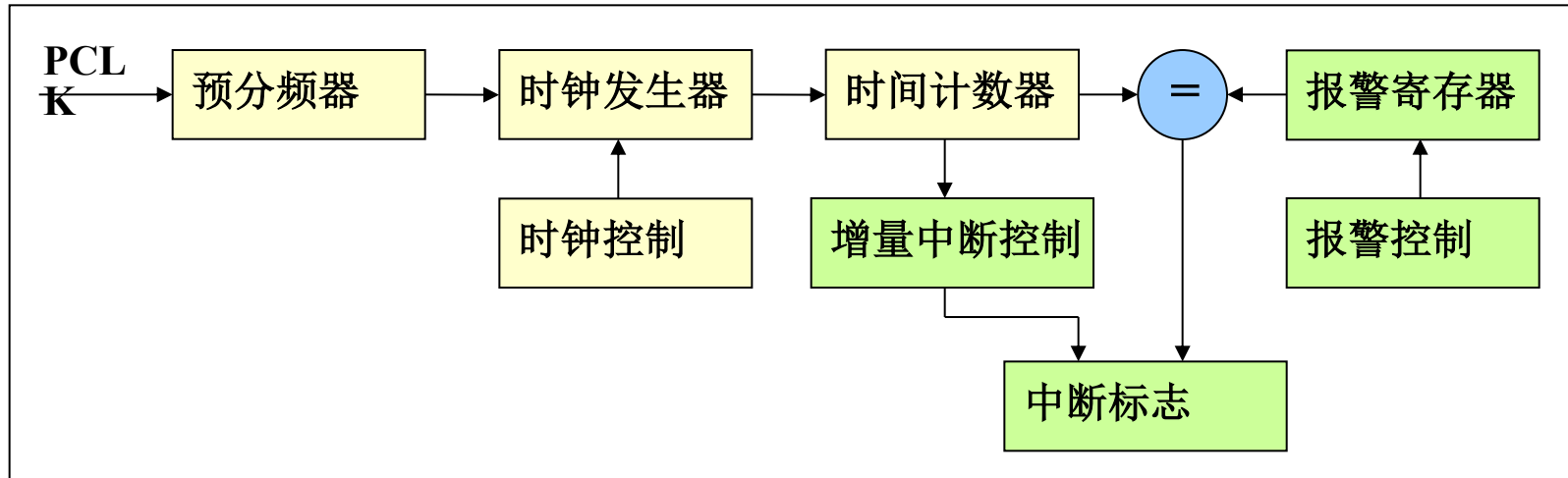
分类时间寄存器包含8个寄存器，所有寄存器都可读可写。

名称	有效位	描述
SEC	6	秒值。该值的范围为0~59。
MIN	6	分值。该值的范围为0~59。
HOUR	5	小时值。该值的范围为0~23。
DOM	5	日期（月）值。该值的范围为1~28,29,30或31（取决于月份以及是否为闰年）。
DOW	3	星期值。该值的范围为1~365（闰年为366）。
DOY	9	日期（年）值。该值的范围为1~365。

注意：这些日期的寄存器只能在适当的时间间隔处递增，而在定义的溢出点处复位。为了使这些值有意义，它们不能进行计算且必须正确初始化。

其中DOY寄存器需要单独初始化，也就是说该寄存器的值不会因为对年、月、日寄存器进行初始化而自动确定到一个正确的值。

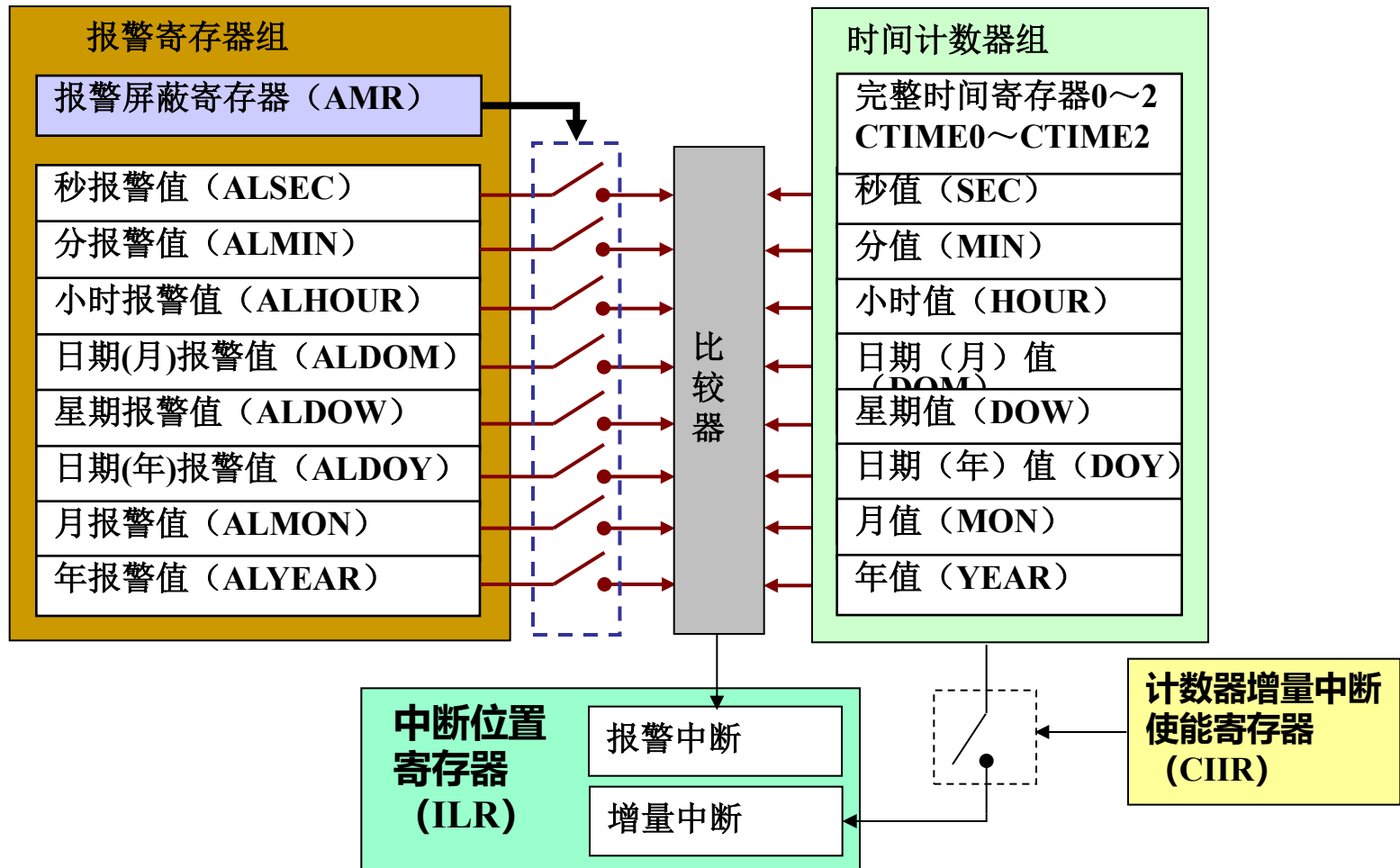
寄存器描述—中断产生寄存器



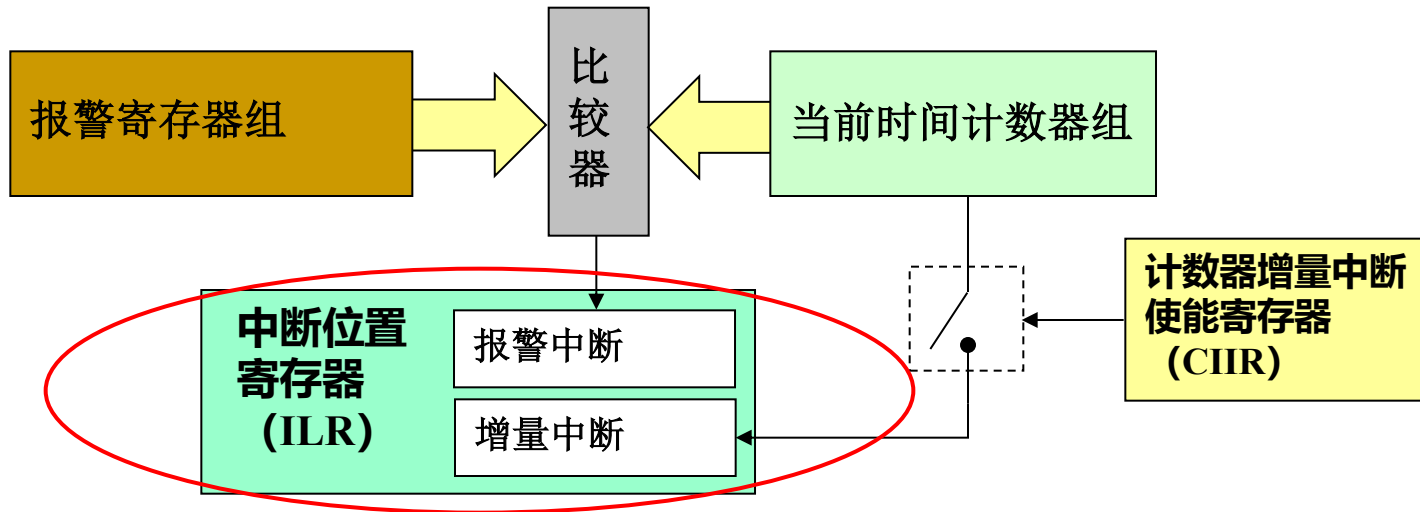
RTC部件的中断分为两类，一类为**时间计数器的增量中断**，由增量中断寄存器控制。另一类为**报警匹配产生的中断**，由报警屏蔽寄存器控制。

名称	描述	访问
ILR	中断位置寄存器	读写
CIIR	递增中断寄存器	读写
AMR	报警屏蔽寄存器	读写
报警寄存器组	设定报警时间	读写

实时时钟中断示意图



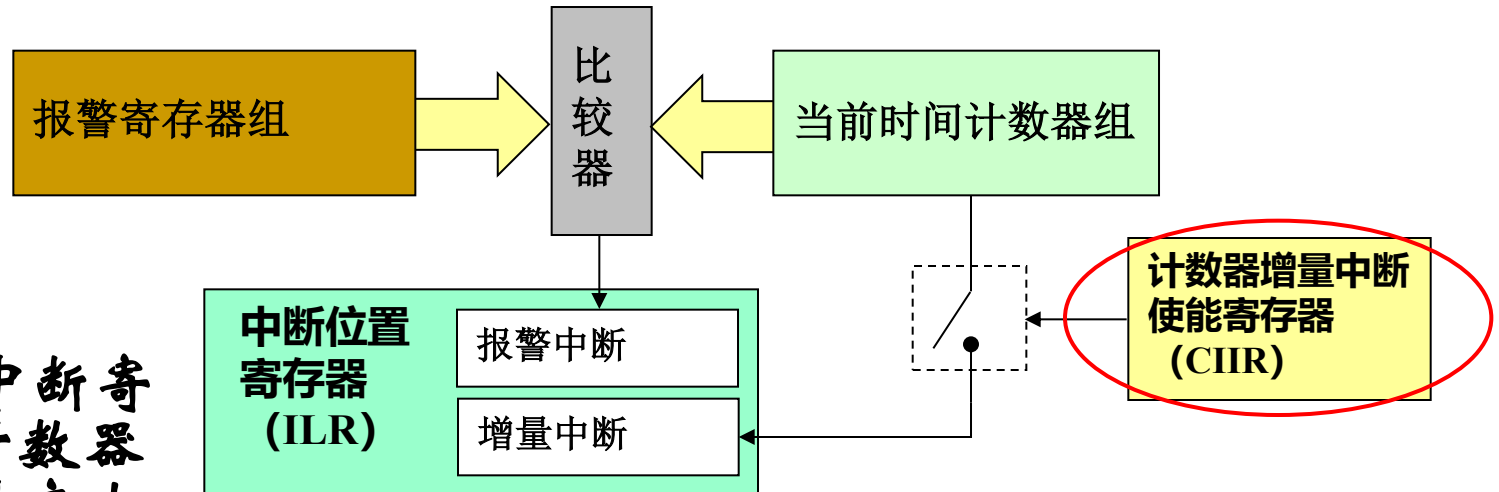
寄存器描述—中断标志寄存器



中断位置寄存器是一个2位的寄存器，它指出哪些模块产生中断，它实际上是一个中断标志寄存器。

位	功能	描述
0	RTCCIF	为1时，表示计数器增量中断模块产生了中断。 向该位写入1清除该中断标志。
1	RTCALF	为1时，表示报警寄存器产生了中断。 向该位写入1清除该中断标志位。

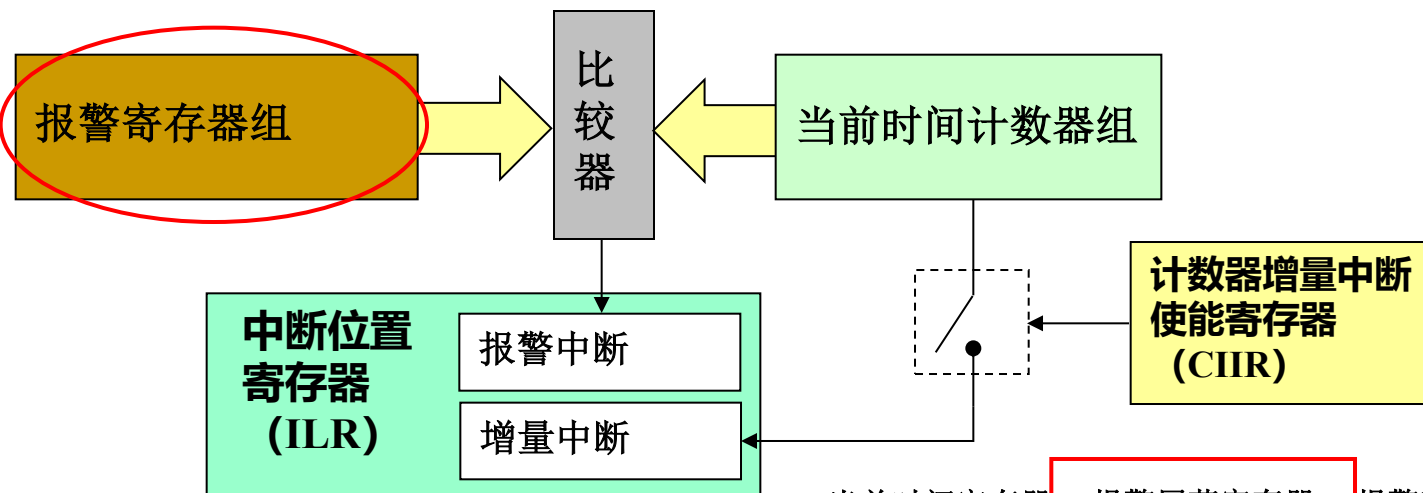
寄存器描述—增量中断使能寄存器



增量中断寄存器可使计数器每次增加时产生一次中断，比如设置秒增加中断为1，则每秒均产生一次中断。在清除增量中断标志之前，该中断一直保持有效。

CIIR	功能	描述
0	IMSEC	为1时，秒值的增加产生一次中断。
1	IMMIN	为1时，分值的增加产生一次中断。
2	IMHOUR	为1时，小时值的增加产生一次中断。
3	IMDOM	为1时，日期（月）的增加产生一次中断。
4	IMDOW	为1时，星期值的增加产生一次中断。
5	IMDOY	为1时，日期（年）的增加产生一次中断。
6	IMMON	为1时，月值的增加产生一次中断。
7	IMYEAR	为1时，年值的增加产生一次中断。

寄存器描述—报警屏蔽寄存器



报警屏蔽寄存器允许用户屏蔽任意的报警寄存器，被屏蔽的报警寄存器将不与时间计数器比较。

未被屏蔽的报警寄存器与时间计数器比较如果匹配，将产生中断。该中断只有在从不匹配到匹配时才发生，可以避免中断重复。

注意：如果所有屏蔽位都置位，报警将被禁止。

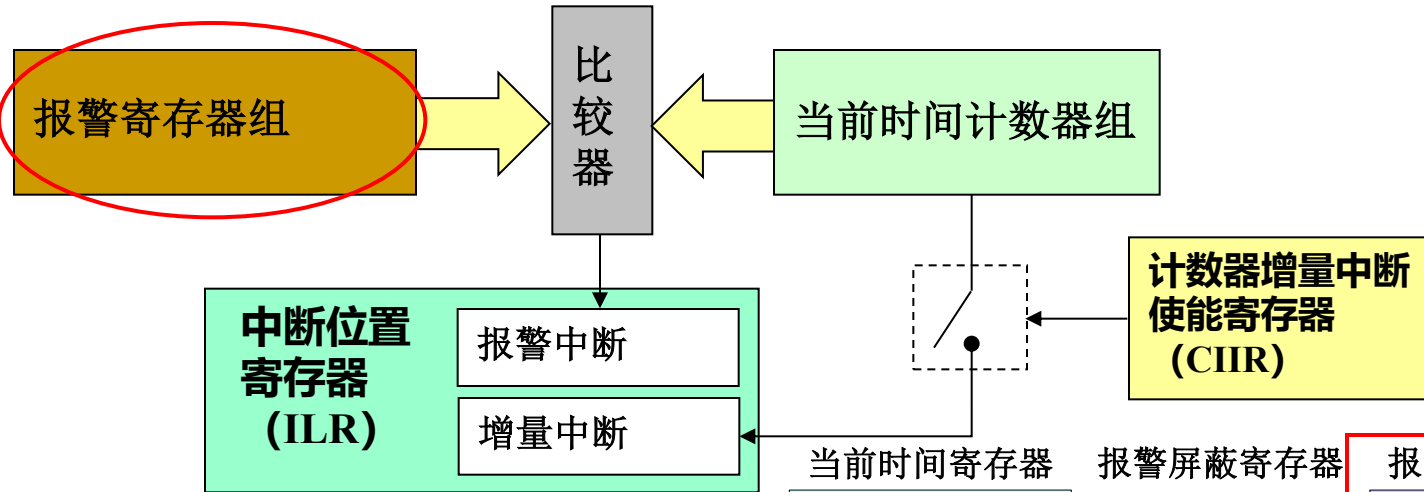
当前时间寄存器	报警屏蔽寄存器	报警时间寄存器
年值	AMRYEAR	年值
月值	AMRMON	月值
日期值（年）	AMRDOY	日期值（年）
星期值	AMRDOW	星期值
日期值（月）	AMRDOM	日期值（月）
小时值	AMRHOUR	小时值
分值	AMRMIN	分值
秒值	AMRSEC	秒值

寄存器描述—报警屏蔽寄存器

AMR	功能	描述
0	AMRSEC	为1时，秒值不与报警寄存器比较。
1	AMRMIN	为1时，分值不与报警寄存器比较。
2	AMR HOUR	为1时，小时值不与报警寄存器比较。
3	AMRDOM	为1时，日期（月）值不与报警寄存器比较。
4	AMRDOW	为1时，星期值不与报警寄存器比较。
5	AMRDOY	为1时，日期（年）值不与报警寄存器比较。
6	AMRMON	为1时，月值不与报警寄存器比较。
7	AMRYEAR	为1时，年值不与报警寄存器比较。



• 寄存器描述—报警屏蔽寄存器

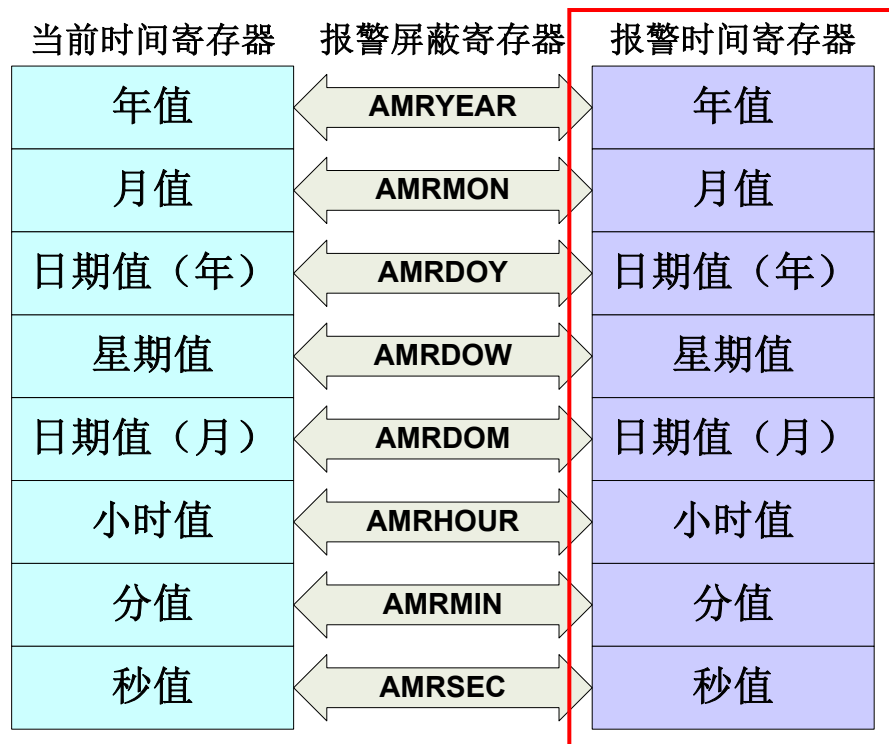


报警寄存器的值与时间计数器相比较，如果所有未被屏蔽的报警寄存器都与它们对应的时间计数器相匹配，那么将产生一次中断。向中断位置寄存器的bit1写入1清除中断。

当前时间寄存器	报警屏蔽寄存器	报警时间寄存器
年值	AMRYEAR	年值
月值	AMRMON	月值
日期值 (年)	AMRDOY	日期值 (年)
星期值	AMRDOW	星期值
日期值 (月)	AMRDOM	日期值 (月)
小时值	AMRHOUR	小时值
分值	AMRMIN	分值
秒值	AMRSEC	秒值

寄存器描述—报警寄存器组

名称	描述	访问
ALSEC	秒报警值	读写
ALMIN	分报警值	读写
ALHOUR	小时报警值	读写
ALDOM	日期（月）报警值	读写
ALDOW	星期报警值	读写
ALDOY	日期（年）报警值	读写
ALMON	月报警值	读写
ALYEAR	年报警值	读写



RTC中断

■ 概述

LPC2000系列ARM RTC具有两种类型的中断:

➤ 增量中断

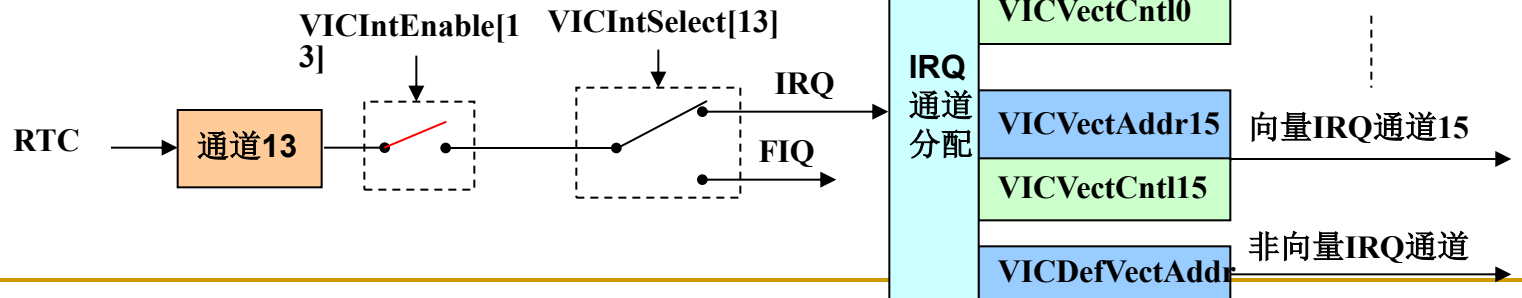
➤ 报警中断

用户可以通过读取中断位置寄存器 (ILR) 来区分中断类型。

RTC中断

■ RTC中断与VIC的关系

RTC处于VIC的通道13，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

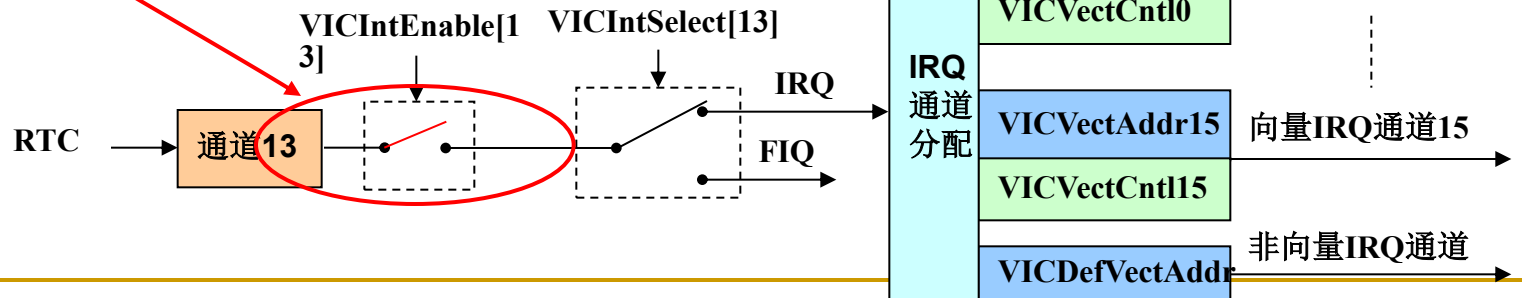


RTC中断

■ RTC中断与VIC的关系

RTC处于VIC的通道13，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

➤ 当VICIntEnable[13] = 0时，通道13中断禁止；

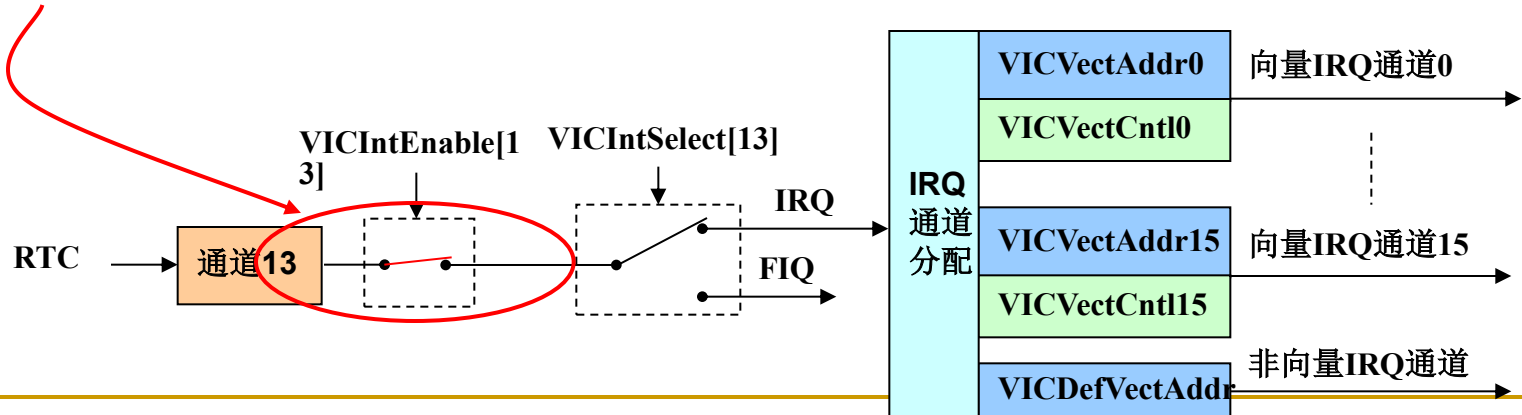


RTC中断

RTC中断与VIC的关系

RTC处于VIC的通道13，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能。

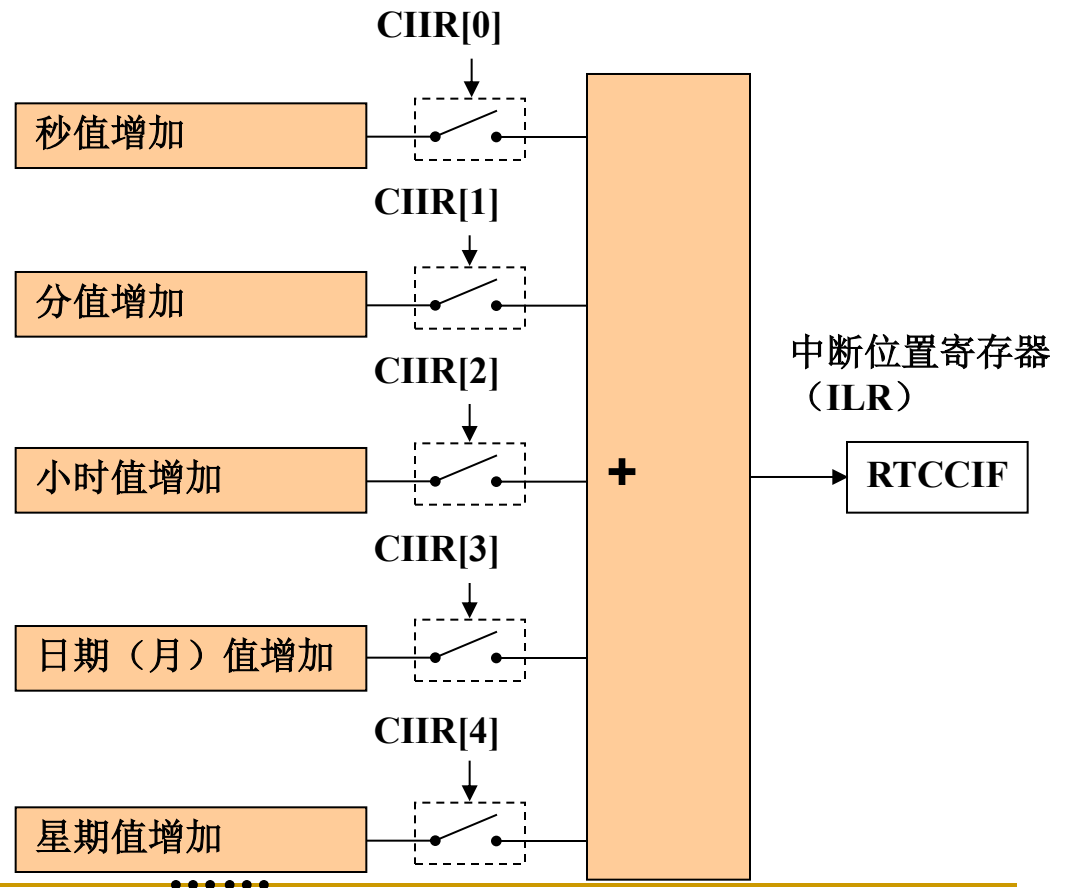
- 当 VICIntEnable[13] = 0 时，通道13中断禁止；
- 当 VICIntEnable[13] = 1 时，通道13中断使能；



RTC中断

• 增量中断

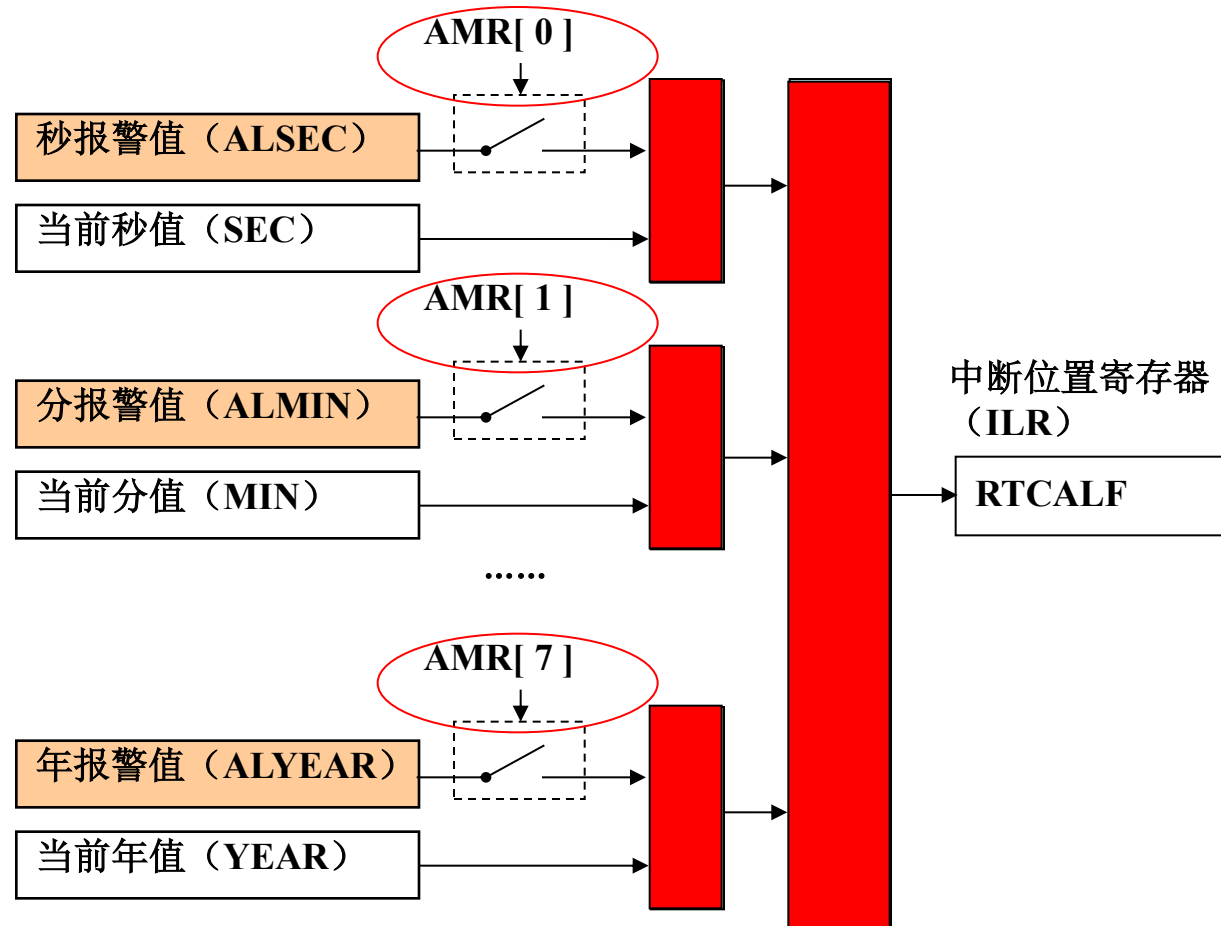
当秒、分、小时等增加时，触发增量中断。计数器增量中断寄存器CIIR[0:7]分别用来控制增量中断的使能。



RTC中断

报警中断

RTC的当前时间与报警时间进行比较，当相等时，便触发报警中断。



报警屏蔽寄存器用来屏蔽相关的报警时间。

4.18 实时时钟

■ 使用RTC的注意事项

- RTC部件使用VPB时钟，如果该时钟出现任何的异常都会导致时间值的偏移，所以芯片不能进入掉电模式；
- 芯片掉电后RTC不能保持寄存器的内容；
- 芯片复位后只有预分频寄存器被硬件初始化，其它寄存器的内容不受复位影响；
- RTC部件的闰年计算比较简单，当年计数器的最低两位为0时，认为该年为闰年，这在2100年时会出错。闰年影响2月份的日期数和年的日期数。

• RTC使用示例

操作流程

设置RTC基准
时钟分频器



初始化RTC时
钟值



报警中断设置



启动RTC



读取当前时间
或等待中断

RTC使用示例—RTC初始化示例

```
Void RTCIni(void)
```

```
{
```

```
PREINT = Fpclk/32768 - 1;
```

设置预分频寄存器

```
PREFRAC = Fpclk - (Fpclk/32768)*32768;
```

```
YEAR = 200;
```

设置年

```
MONTH = 2;
```

设置月

```
DOM = 19;
```

设置日（月）

```
DOW = 4;
```

设置星期

```
HOURL = 8;
```

设置小时

```
MIN = 30;
```

设置分

```
SEC = 0;
```

设置秒

```
CIIR = 0x01;
```

设置增量中断

```
CCR = 0x01;
```

启动RTC

```
}
```

• RTC使用示例—RTC定时报警设置

```
...  
ILR = 0x03;           // 清除RTC中断标志  
CIIR = 0x02;          // 设置分值增量中断  
ALHOUR = 12;          // 设置小时比较值  
ALMIN = 0;            // 设置分比较值  
ALSEC = 0;            // 设置秒比较值  
AMR = 0xF8;           // 设置报警屏蔽寄存器  
                        // 只允许时、分、秒比较  
...
```